

Opus Publica

RC1 Engineering Playbook

The Operational Execution of the Engineering Constitution

Version 1.0

Issued 09 August 2026

Document Type	Engineering Playbook (L3 in the Constitutional Hierarchy)
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	All engineers, reviewers, QA, on-call, operators, build agents
Classification	Mandatory — Operational Execution Authority
Governed By	Engineering Constitution Volume I (Ch 1-25) + Volume II (Ch 26-36)
Effective Date	09 August 2026
Supersedes	None (first issuance)
Review Cadence	Monthly, or on any Constitution amendment
Document ID	OP-PLAY-RC1-001

This Playbook converts every constitutional rule into a checklist, procedure, runbook, or gate that an engineer applies at the moment of implementation.

Chapter 1 — How to Use This Playbook

1.1 Opening: Why This Playbook Exists

The Engineering Constitution — Volume I and Volume II together — specifies what Opus Publica is, what contracts it honours, and what invariants it guarantees. The Constitution is 572 pages of normative requirements. An engineer facing a Tuesday morning task cannot consult 572 pages to discover how to proceed. The Engineering Playbook is the answer to that gap: it is the document the engineer consults at the moment of implementation, the document the reviewer consults at the moment of review, the document the certifier consults at the moment of certification, and the document the on-call engineer consults at three o'clock in the morning during an incident. Every constitutional rule is converted here into a checklist, a procedure, a runbook, or a gate. If the Constitution is the law, the Playbook is the operating manual.

The thesis of this Playbook is that operationalization is the bridge between specification and reality. A constitutional rule that is not operationalized — that does not appear as a checkable item in a checklist, a step in a procedure, or a trigger in a gate — is a rule that will be violated, not from malice but from the everyday pressure of implementation. The Playbook exists to ensure that no constitutional rule is merely aspirational. Every rule has an operational form, every operational form has an owner, and every owner is accountable for the rule's enforcement.

1.2 Background: Constitutional Context

The Engineering Playbook occupies Level 3 (L3) in the constitutional hierarchy of Opus Publica. The hierarchy is as follows: L0 is the Product Vision (what Opus Publica is and why it exists); L1 is the Engineering Constitution (the contracts, invariants, and verification regimes that bind every implementation); L2 is the Architecture Decision Records (the technology and design choices made within the Constitution's contracts); L3 is this Engineering Playbook (the operational checklists, runbooks, and gates that operationalize the Constitution); and L4 is the Implementation itself (the code, configurations, and prompts produced under the Constitution and Playbook). Each level is binding on the levels below it and is governed by the levels above it.

Constitution Reference: Volume I, Preamble §2 (The Constitutional Hierarchy)

The Playbook operationalizes two Foundational Principles that the Constitution establishes and that the Playbook MUST enforce at every operational gate: Foundational Principle I (the Principle of Runtime Truth) holds that no implementation is correct because it compiles or passes static analysis — every user-visible feature must be verified through runtime execution and objective evidence; Foundational Principle II (the Production Acceptance Gate) holds that no implementation may proceed to production release unless it satisfies ALL of the Engineering Constitution, the Engineering Playbook, Runtime Certification, and the Production Acceptance Criteria. These two principles are the operational contract of this Playbook: every procedure, checklist, and gate in the following chapters exists to enforce them.

The Playbook's Prime Directive

No implementation MAY proceed to production release unless it has passed every applicable checklist in this Playbook. The Production Acceptance Gate (Chapter 19) is the final gate; it SHALL NOT be bypassed, and it SHALL NOT be overridden below the Chief Systems Architect. The Chief Systems Architect SHALL NOT override a Runtime Certification failure — the implementation SHALL be corrected instead. This is the operational form of the Constitution's Production Acceptance Gate.

1.3 How the Playbook Is Organized

The Playbook is organized into seven parts, each addressing a distinct phase of the engineering lifecycle or a distinct operational discipline. The parts are sequential in the sense that an implementation passes through them in order, but they are also referenceable independently: an engineer investigating an incident consults Part V (Operational Ownership); an engineer preparing a release consults Part IV (Certification and Release); an engineer writing code consults Part II (Daily Engineering Disciplines).

Part	Chapters	Purpose	Primary Audience
I — The Engineering Lifecycle	1-2	How an implementation moves from Constitution clause to production release.	All engineers
II — Daily Engineering Disciplines	3-5	Branching, commits, pull requests, code review — the day-to-day operational practices.	All engineers, reviewers
III — Quality Assurance and Testing	6-8	The QA regime, test layers, environments, data, and runtime invariant monitoring.	QA engineers, implementers
IV — Certification and Release	9-12	The certification process, the five pillars, release, deployment, and rollback.	Release Certification Authority, release managers
V — Operational Ownership	13-16	On-call, operational governance execution, SLO management, incident command.	On-call engineers, operators
VI — Constitution Operationalization Matrices	17-18	Cross-references from every Constitution chapter to the Playbook section that	All engineers (reference)

		operationalizes it.	
VII — The Master Checklist Catalog	19-20	Every checklist and runbook referenced in the Playbook, collected for easy reference.	All engineers (reference)

1.4 How to Read Each Chapter

Every operational chapter in this Playbook follows a consistent structure designed for actionability. The structure is: (1) Purpose — what the chapter operationalizes and why; (2) Constitutional References — the Constitution chapters and rules the chapter enforces; (3) Procedure — the step-by-step actions an engineer takes; (4) Checklist — the checkable items that gate the procedure; (5) Roles and Ownership — who does what; (6) Gates — where the procedure blocks progress until satisfied; (7) Runbooks — the operational responses to failures and exceptions; (8) Evidence — what artifacts the procedure produces and where they are stored.

Chapters in Part VI (Matrices) and Part VII (Checklist Catalog) are reference chapters and do not follow the operational structure; they are organized as tables for lookup. The matrices map every Constitution chapter to the Playbook section that operationalizes it, and the checklist catalog collects every checklist in one place for easy reference during review and certification.

1.5 Operational Vocabulary

This Playbook uses the same RFC 2119 vocabulary as the Constitution (MUST, SHALL, SHOULD, MAY). In addition, the Playbook introduces operational terms that describe the enforcement level of each procedure:

Term	Meaning	Enforcement
GATE	A hard block. Progress cannot proceed until the gate is satisfied.	Automated (CI) or manual (reviewer/RCA).
CHECK	A checkable item in a checklist. Must be marked satisfied or N/A with justification.	Reviewer at review time; RCA at certification time.
PROCEDURE	A sequence of steps that MUST be followed in order.	Engineer; verified by reviewer.
RUNBOOK	A documented response to a specific failure or scenario.	On-call engineer; invoked during incidents.
ALERT	An automated signal that triggers human attention.	Monitoring system; routed per escalation matrix.
EVIDENCE	An artifact that proves a procedure	Stored in certification repository;

	was followed or a gate was satisfied.	retained per schedule.
--	---------------------------------------	------------------------

1.6 Limitations and When the Playbook Does Not Apply

Rigor demands acknowledging the limits of this Playbook. The Playbook is comprehensive but not exhaustive; it operationalizes the Constitution as it stands at RC1 and will require amendment as the Constitution evolves (per Chapter 36, Evolution Governance). The following limitations are acknowledged:

- **Constitutional supremacy.** The Playbook operationalizes the Constitution; it does not replace it. Where the Playbook and the Constitution conflict, the Constitution prevails, and the Playbook SHALL be corrected.
- **RC1 scope.** The Playbook's checklists and procedures are calibrated for RC1. RC2 and RC3 will introduce new operational scenarios (multi-tenancy, plugins, AI features) that require new Playbook chapters. The Playbook's review cadence (monthly) is designed to surface these gaps.
- **Team-size assumption.** The Playbook assumes a team of at least five engineers with defined roles. Smaller teams may consolidate roles but SHALL NOT consolidate the gates; the Production Acceptance Gate applies regardless of team size.
- **Automation maturity.** The Playbook's automation (CI gates, invariant monitors, reconciliation jobs) is specified but not yet implemented for every check. Where automation is not yet available, the check SHALL be enforced manually by the reviewer or RCA, and the gap SHALL be tracked as a Playbook automation debt item.
- **Editorial boundary.** The Playbook does not govern editorial decisions (what to publish, whom to assign as reviewer, what decision to render). It governs the engineering execution of editorial decisions. The boundary is enforced at the Constitution level (Chapter 1, Product Non-Goals).

When a scenario arises that the Playbook does not address, the engineer SHALL escalate to the Engineering Leadership Team. The Leadership Team SHALL either issue a Playbook amendment (through the Evolution Governance process) or issue a recorded precedent that addresses the scenario. Ad hoc deviation from the Playbook is forbidden; the Playbook is the operational authority, and deviations are tracked, not hidden.

1.7 Conclusion: The Path Forward

This Playbook is the operational execution of the Engineering Constitution. Its completion is the precondition for RC1 release: no feature MAY be released until it has passed the Production Acceptance Gate (Chapter 19), and no feature MAY reach the Gate until it has passed the Implementation Checklist (Chapter 19), the Review Checklist (Chapter 19), and the Certification Checklist (Chapter 19). The Playbook converts the Constitution's normative requirements into operational practice; the

Constitution's success or failure will be measured by whether the Playbook's gates are enforced without exception.

The path forward is: (1) Engineering Leadership operationalizes every checklist and procedure in this Playbook; (2) automation is built for every check that can be automated, with manual enforcement as the interim; (3) the first end-to-end implementation cycle is executed against the Playbook, and the Playbook is amended based on the lessons learned; (4) RC1 is released only after the Production Acceptance Gate has been exercised and passed. From that point forward, the Playbook is the daily operating manual of the engineering organization, reviewed monthly and amended through governance.

1.8 References

- Opus Publica RC1 Engineering Constitution, Volume I (Chapters 1-25), OP-CONST-RC1-001.
- Opus Publica RC1 Engineering Constitution, Volume II: The Eleven Missing Chapters (Chapters 26-36), OP-CONST-RC1-002.
- IETF RFC 2119 — Key words for use in RFCs to indicate requirement levels.
<https://datatracker.ietf.org/doc/html/rfc2119>
- Google SRE Workbook — SLI/SLO/error budget discipline. <https://sre.google/workbook/>
- NIST SP 800-92 — Guide to Computer Security Log Management.
<https://csrc.nist.gov/pubs/sp/800/92/final>

TABLE OF CONTENTS

Right-click the table below and select "Update Field" to refresh page numbers.

Chapter 2 — The Implementation Lifecycle

2.1 Purpose

This chapter operationalizes the end-to-end lifecycle of an implementation: from the selection of a Constitution clause to implement, through task definition, implementation, review, certification, release, and post-release ownership. It is the master lifecycle that every feature, every bug fix, and every operational change SHALL traverse. No implementation MAY skip a stage; no implementation MAY perform stages out of order. The lifecycle is the operational form of the Engineering Flow established in the Constitution's Preamble.

Constitution Reference: *Volume I, Preamble §6 (The New Engineering Flow); Volume I, Chapter 5.01 (Implementation Lifecycle)*

2.2 The Lifecycle Stages

The implementation lifecycle consists of eight stages, each with defined entry criteria, exit criteria, owner, and gate effect. The stages are sequential; the exit of one stage is the entry of the next. A stage's gate SHALL NOT be passed until its exit criteria are satisfied and its evidence is recorded.

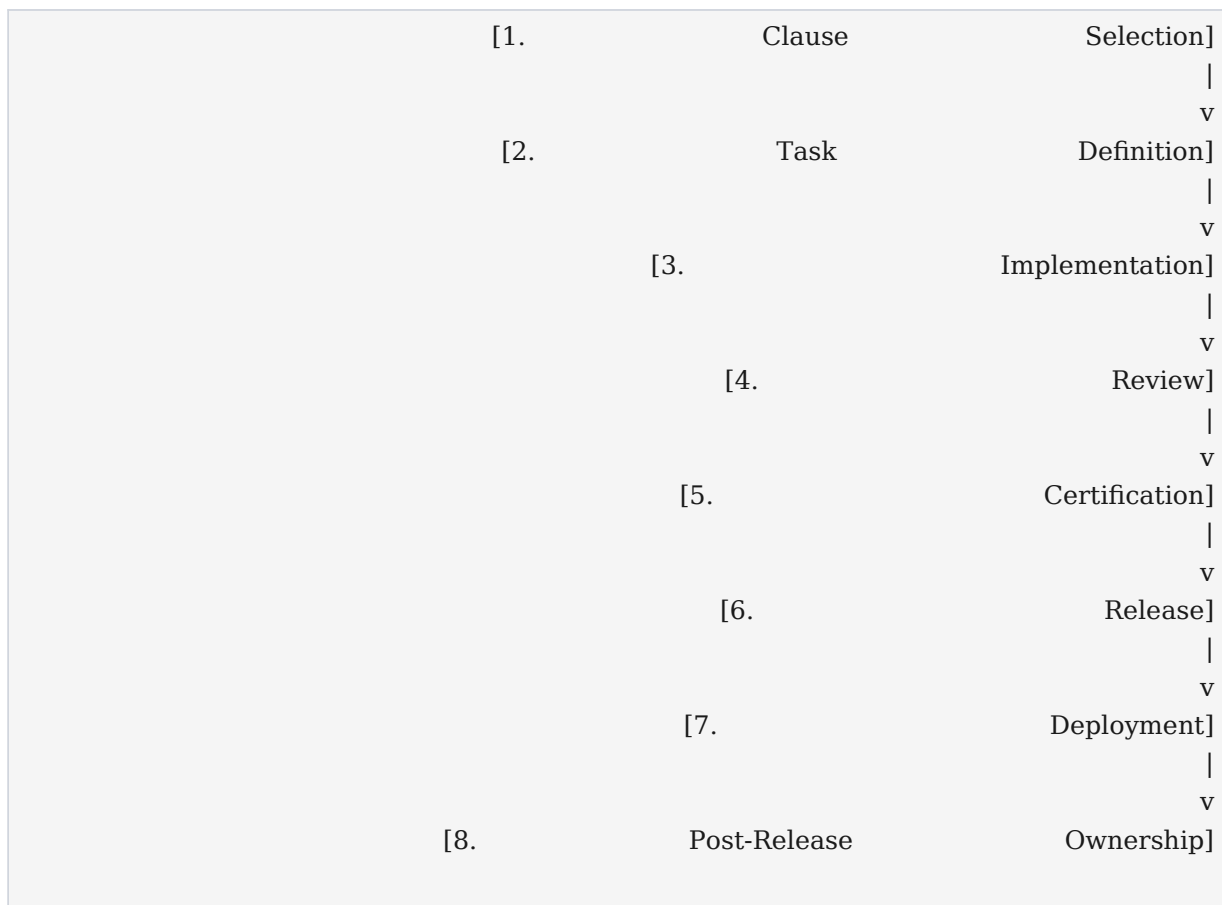


Figure 2.1: The Eight-Stage Implementation Lifecycle

Stage	Owner	Entry Criteria	Exit Criteria (Gate)
1. Clause Selection	Engineer + Tech Lead	Constitution chapter identified; no conflicting work in progress.	Constitution clause(s) cited by stable ID (e.g., PUB-001).
2. Task Definition	Tech Lead	Clause selected; acceptance criteria derivable from the clause.	Task ticket created with clause ref, ACs, owner, estimate.
3. Implementation	Engineer (+ build agent)	Task defined; branch created; environment ready.	Code complete; unit tests pass; local smoke passes.
4. Review	Reviewer (not the implementer)	PR opened; CI green; self-review done.	Reviewer approval; all checklist items satisfied.
5. Certification	QA + Release Certification Authority	PR merged to release branch; feature flag off.	Five pillars verified; artifact set complete; RCA sign-off.
6. Release	Release Manager	Certification complete; rollback tested.	Release artifact built, signed, staged.
7. Deployment	Release Manager + On-Call	Release staged; deployment window open; on-call paged.	Deployed; post-deploy verification passing; flag on.
8. Post-Release Ownership	Owning Team	Deployment complete; feature live.	Ownership transferred to on-call; telemetry observed for 72h.

2.3 Stage 1: Constitution Clause Selection

Before any implementation begins, the engineer and the tech lead SHALL identify the Constitution clause(s) the implementation satisfies. This is the operational form of Rule PROC-001 (no implementation without a corresponding Constitution clause). Clause selection is not optional; an implementation that cannot cite a clause is, by definition, out of process and SHALL be halted.

Constitution Reference: *Volume I, Chapter 3 §3.3.6 (PROC-001); Volume II, Chapter 23 (Knowledge Graph)*

Procedure: Clause Selection

Step 1. Identify the Constitution chapter Locate the chapter in Volume I or Volume II that governs the feature's domain (e.g., a Crossref deposit feature is governed by Volume I Chapter 14).

Step 2. Identify the specific clause or rule Within the chapter, locate the specific requirement, rule, or invariant the feature implements. Record its stable ID (e.g., EXT-001, INV-I-02, PUB-008).

Step 3. Verify no conflict with in-flight work Check the task tracker for in-flight work that touches the same clause. If conflict exists, coordinate with the other task's owner before proceeding.

Step 4. Record the clause reference in the task ticket The task ticket SHALL cite the Constitution chapter and the stable ID. A ticket without a clause reference SHALL NOT be created.

Step 5. Identify the Knowledge Graph node Per Volume II Chapter 23, the implementation SHALL be traceable. Record the requirement node ID that this implementation satisfies.

Checklist: Clause Selection (GATE)

- ☐ **CS-01** Constitution chapter identified and cited (e.g., 'Vol I Ch 14').
- ☐ **CS-02** Specific clause/rule/invariant cited by stable ID.
- ☐ **CS-03** No conflicting in-flight work identified.
- ☐ **CS-04** Task ticket created with clause reference.
- ☐ **CS-05** Knowledge Graph requirement node ID recorded.

2.4 Stage 2: Task Definition

Task definition converts a Constitution clause into a concrete, estimable, verifiable unit of work. The task ticket is the artifact that travels through the remaining stages; its acceptance criteria are the operational form of the Constitution clause's requirements.

Procedure: Task Definition

Step 1. Derive acceptance criteria from the clause Each acceptance criterion SHALL be testable, binary (pass/fail), and traceable to a clause sub-requirement. 'Mostly working' is not an acceptance state.

Step 2. Identify the applicable Playbook checklists Determine which checklists in this Playbook apply to the task (Implementation, Review, Certification, possibly Invariant). Record them in the ticket.

Step 3. Assign an owner The owner is the engineer accountable for the task end-to-end. The owner SHALL be a member of the team that owns the relevant Constitution chapter (Conway's Law, Volume I Chapter 2 §2.11).

Step 4. Estimate the work Estimates are ranges, not points. Record a best-case and a worst-case. Tasks exceeding one sprint SHALL be split.

Step 5. Identify dependencies Record upstream dependencies (other tasks, external services, schema changes) and downstream dependents.

Step 6. Record the task in the tracker The ticket SHALL contain: clause ref, ACs, owner, estimate, dependencies, applicable checklists.

Checklist: Task Definition (GATE)

- ☐ **TD-01** Acceptance criteria derived from Constitution clause.
- ☐ **TD-02** Each AC is testable and binary.

- ☐ **TD-03** Applicable Playbook checklists identified and recorded.
- ☐ **TD-04** Owner assigned (member of the chapter-owning team).
- ☐ **TD-05** Estimate recorded as best-case/worst-case range.
- ☐ **TD-06** Dependencies identified.
- ☐ **TD-07** Task ticket complete with all fields.

2.5 Stage 3: Implementation

Implementation is the stage at which code is written, tested locally, and prepared for review. Implementation SHALL follow the Daily Engineering Disciplines (Part II, Chapters 3-5) and the Coding Standards (Volume I Chapter 7). The build agent (Antigravity) MAY assist with implementation but its output is subject to the full review and certification regime (Rule PROC-005).

Constitution Reference: Volume I, Chapter 7 (Coding Standards); Volume I, Chapter 3 §3.3.6 (PROC-005: build-agent output reviewed)

Procedure: Implementation

Step 1. Create a branch per the branching model Follow Chapter 3 (Branching and Version Control).

Branch name SHALL encode the task ID and a short slug.

Step 2. Implement the feature Follow the Coding Standards (Vol I Ch 7). No magic strings, no fallback hacks, single implementation, single ownership.

Step 3. Write tests at every applicable layer Unit, integration, contract, and where applicable end-to-end. See Chapter 6 (Quality Assurance) for the test layers and ownership.

Step 4. Run the local smoke test Execute the local smoke test suite. All tests SHALL pass before opening a PR.

Step 5. Self-review the implementation Before opening a PR, review your own diff against the Review Checklist (Chapter 19). Fix issues found in self-review before requesting peer review.

Step 6. Open the pull request Follow Chapter 4 (Commits and Pull Requests). The PR description SHALL cite the Constitution clause, the task ticket, and the applicable checklists.

Checklist: Implementation (GATE to PR)

- ☐ **IM-01** Branch created per branching model (Ch 3).
- ☐ **IM-02** Coding Standards followed (Vol I Ch 7).
- ☐ **IM-03** Tests written at every applicable layer.
- ☐ **IM-04** Local smoke test passes (100%).
- ☐ **IM-05** Self-review completed; issues fixed.
- ☐ **IM-06** PR opened with required metadata (clause ref, ticket, checklists).
- ☐ **IM-07** CI pipeline green on the PR branch.

2.6 Stage 4: Review

Review is the gate at which a peer (not the implementer) verifies that the implementation satisfies the Constitution clause and the Playbook checklists. Review is mandatory; no PR MAY be merged without review approval. The review regime is specified in Chapter 5 (Code Review Regime).

Constitution Reference: *Volume I, Chapter 5.04 (Code Review); Volume I, Chapter 3 §3.3.6 (PROC-003, PROC-004)*

Procedure: Review

Step 1. Assign a reviewer The reviewer SHALL NOT be the implementer. For risk-class R1 features, two reviewers are required. See Chapter 5 for risk classification.

Step 2. Reviewer executes the Review Checklist The reviewer SHALL apply every applicable item in the Review Checklist (Chapter 19). Each item is marked satisfied (with evidence) or N/A (with justification).

Step 3. Reviewer cites the Constitution clause satisfied Per Rule PROC-004, the reviewer SHALL cite the Constitution clause and Playbook checklist item the implementation satisfies. A review approval without this citation is invalid.

Step 4. Address review comments The implementer SHALL address every blocking comment. Non-blocking comments SHOULD be addressed or explicitly deferred with a recorded ticket.

Step 5. Re-review after changes If the implementer's changes are substantive, the reviewer SHALL re-review. Trivial changes (whitespace, typo) do not require re-review.

Step 6. Approve and merge On approval, the PR is merged by the reviewer (not the implementer). The merge SHALL use the merge strategy specified in Chapter 3.

Checklist: Review (GATE to Merge)

- ☐ **RV-01** Reviewer assigned (not the implementer).
- ☐ **RV-02** Review Checklist (Ch 19) applied; all items satisfied or N/A.
- ☐ **RV-03** Constitution clause cited by reviewer (PROC-004).
- ☐ **RV-04** All blocking comments addressed.
- ☐ **RV-05** Idempotency contract verified (if API change).
- ☐ **RV-06** No fallback hacks introduced (Vol I Ch 7 §7.5).
- ☐ **RV-07** No magic strings introduced (Vol I Ch 7 §7.4).
- ☐ **RV-08** Approval recorded; PR merged by reviewer.

2.7 Stage 5: Certification

Certification is the gate at which runtime evidence is converted into a release decision. Certification is performed by the Release Certification Authority (RCA), a role distinct from the implementing team and

from QA. The certification regime is specified in Chapter 9 (The Certification Process) and the five pillars in Chapter 10.

Constitution Reference: *Volume I, Chapter 8 (Certification Standards); Volume II, Chapter 20 (Runtime Verification Constitution); Volume II, Chapter 22 (Certification Constitution)*

Procedure: Certification

Step 1. Merge to release branch The PR is merged to the release branch. The feature flag (if any) remains off.

Step 2. QA executes the test suite in production-equivalent env QA runs the full test suite in the staging/production-equivalent environment. All tests SHALL pass.

Step 3. QA produces the five-pillar evidence For each pillar (Runtime, Visual, Database, Storage, API), QA produces the evidence specified in Chapter 10.

Step 4. RCA reviews the artifact set The RCA reviews the complete certification artifact set (Chapter 9 §9.3). The RCA SHALL NOT sign off if any artifact is missing or incomplete.

Step 5. RCA verifies invariant monitors The RCA verifies that the feature does not violate any engineering invariant (Volume II Chapter 26). Invariant violations block certification absolutely.

Step 6. RCA signs off or rejects On sign-off, the RCA records the certification in the release record. On rejection, the RCA returns specific deficiencies to the implementing team.

Checklist: Certification (GATE to Release)

- ☐ **CT-01** PR merged to release branch; flag off.
- ☐ **CT-02** Full test suite passes in production-equivalent env.
- ☐ **CT-03** Runtime evidence captured (logs, traces, invariant monitors).
- ☐ **CT-04** Visual evidence captured (screenshots/recordings of every state).
- ☐ **CT-05** Database verification captured (before/after persisted state).
- ☐ **CT-06** Storage verification captured (artifacts, SHA-256 hashes, retrieval).
- ☐ **CT-07** API verification captured (request/response, contract tests).
- ☐ **CT-08** Idempotency tested (retries produce same end-state).
- ☐ **CT-09** Rollback tested (feature can be rolled back without data loss).
- ☐ **CT-10** No invariant violations during certification.
- ☐ **CT-11** RCA sign-off recorded in release record.

2.8 Stage 6: Release

Release is the stage at which the certified implementation is packaged, signed, and staged for deployment. Release is performed by the Release Manager and gated by the RCA's certification. The release regime is specified in Chapter 11 (Release and Deployment).

Procedure: Release

Step 1. Build the release artifact The release artifact is built from the release branch by the CI pipeline. The artifact SHALL be content-addressed and signed.

Step 2. Stage the release The release is staged in the production-equivalent environment, feature flag off, ready for deployment.

Step 3. Verify the staged release The Release Manager verifies the staged release is the certified artifact (hash match, signature match).

Step 4. Prepare the deployment plan The deployment plan SHALL specify: window, staged rollout strategy, rollback procedure, communication plan, on-call assignment.

Step 5. Brief the on-call The on-call engineer is briefed on the release, the rollback procedure, and the alert thresholds.

Checklist: Release (GATE to Deployment)

- ☐ **RL-01** Release artifact built, content-addressed, signed.
- ☐ **RL-02** Staged release hash matches certified artifact.
- ☐ **RL-03** Deployment plan documented (window, rollout, rollback, comms).
- ☐ **RL-04** On-call briefed.
- ☐ **RL-05** Rollback procedure tested in production-equivalent env.

2.9 Stage 7: Deployment

Deployment is the stage at which the release is promoted to production. Deployment SHALL follow the staged rollout strategy and SHALL be monitored throughout. The on-call engineer SHALL be present (paged) during deployment.

Procedure: Deployment

Step 1. Open the deployment window Deployment occurs within the defined window. Emergency deployments outside the window require RCA + Chief Systems Architect approval.

Step 2. Execute the staged rollout Deploy to the first stage (e.g., canary). Monitor for the defined observation period. If green, proceed to the next stage. If red, rollback.

Step 3. Enable the feature flag (if applicable) After deployment, enable the feature flag for the staged audience. Monitor for the defined observation period.

Step 4. Post-deploy verification Execute the post-deploy verification: runtime checks, invariant monitors, SLO dashboard. All SHALL be green.

Step 5. Communicate completion Notify the team and stakeholders of successful deployment.

Checklist: Deployment (GATE to Post-Release)

- ☐ **DP-01** Deployment window open; on-call present.
- ☐ **DP-02** Staged rollout executed; each stage observed.
- ☐ **DP-03** Feature flag enabled (if applicable); observed.
- ☐ **DP-04** Post-deploy verification passing.
- ☐ **DP-05** Invariant monitors green.
- ☐ **DP-06** SLO dashboard green.
- ☐ **DP-07** Communication sent.

2.10 Stage 8: Post-Release Ownership

Post-release ownership is the stage at which the implementation transitions from active development to operational ownership. The owning team carries the pager for the feature; the on-call rotation is updated; the feature is observed for the defined burn-in period (72 hours). Post-release ownership is the operational form of Principle 10 (You Build It, You Run It).

| Constitution Reference: Volume I, Chapter 2 §2.10 (Principle 10: You Build It, You Run It)

Procedure: Post-Release Ownership

Step 1. Transfer ownership to on-call The owning team's on-call rotation is updated to include the new feature. The on-call runbook is updated.

Step 2. Observe the burn-in period The feature is observed for 72 hours. Any anomaly (alert, SLO dip, invariant warning) is investigated.

Step 3. Conduct the post-release review At 72 hours, the owning team conducts a brief review: did the feature behave as expected? Were there anomalies? Is the SLO holding?

Step 4. Close the task ticket On successful burn-in, the task ticket is closed. The implementation is now in steady-state operational ownership.

Checklist: Post-Release Ownership

- ☐ **PR-01** On-call rotation updated to include the feature.
- ☐ **PR-02** On-call runbook updated.
- ☐ **PR-03** 72-hour burn-in observed; no SEV-1/SEV-2 incidents.
- ☐ **PR-04** Post-release review conducted.
- ☐ **PR-05** Task ticket closed.

2.11 Roles and Ownership

Role	Stages Owned	Authority	Boundaries
------	--------------	-----------	------------

Engineer	1, 2, 3, 8	Implements; self-reviews; observes burn-in.	Cannot approve own PR; cannot self-certify.
Tech Lead	1, 2	Validates clause selection; defines task.	Cannot override Constitution; cannot skip gates.
Reviewer	4	Approves or blocks merge.	Shall not be the implementer; shall cite clause.
QA Engineer	5	Produces five-pillar evidence.	Cannot sign off release; that is RCA's authority.
Release Certification Authority	5	Signs off certification; blocks release.	Cannot override invariant failure; cannot be overridden by CSA on runtime.
Release Manager	6, 7	Builds, stages, deploys.	Cannot deploy without RCA sign-off.
On-Call Engineer	7, 8	Monitors deployment; carries pager.	Can rollback; cannot change constitution.
Owning Team	8	Steady-state ownership.	Cannot abdicate on-call; cannot ignore alerts.

2.12 Gates Summary

The lifecycle establishes five hard gates. Each gate is a point at which progress SHALL NOT proceed until the gate's criteria are satisfied. Gates are enforced by the named owner and verified by the next stage's entry check.

Gate	After Stage	Enforced By	Blocks
Clause Selection Gate	1	Tech Lead	Task creation.
Task Definition Gate	2	Tech Lead	Implementation start.
Implementation Gate (PR)	3	CI + Engineer	Review start.
Review Gate (Merge)	4	Reviewer	Merge; certification start.
Certification Gate	5	RCA	Release start.
Release Gate	6	Release Manager	Deployment start.
Deployment Gate	7	On-Call + Post-deploy checks	Burn-in start.

Production Acceptance Gate	7	RCA + Chief Systems Architect	Feature declared live.
----------------------------	---	-------------------------------	------------------------

2.13 Evidence and Records

Every stage produces evidence that SHALL be stored in the certification repository (for certification-stage evidence) or in the task tracker (for earlier-stage evidence). The evidence is the artifact that the RCA reviews at certification and that an auditor reviews post-hoc. Evidence SHALL be retained per the retention schedule (Volume II Chapter 31 §31.8).

Stage	Evidence Produced	Storage	Retention
1. Clause Selection	Clause ref in task ticket.	Task tracker.	Lifetime of the ticket + 5 years.
2. Task Definition	Task ticket with ACs, owner, estimate.	Task tracker.	Lifetime + 5 years.
3. Implementation	Branch, commits, tests, PR.	Version control.	Lifetime of the repository.
4. Review	Review comments, approval record.	Version control.	Lifetime of the repository.
5. Certification	Five-pillar artifact set, RCA sign-off.	Certification repository.	Lifetime of the platform.
6. Release	Release artifact, hash, signature, deployment plan.	Release repository.	Lifetime of the platform.
7. Deployment	Deployment logs, post-deploy verification.	Observability platform.	90 days hot; 1 year cold.
8. Post-Release	Burn-in report, on-call runbook updates.	Task tracker + runbook repo.	Lifetime + 5 years.

Chapter 3 — Branching and Version Control

3.1 Purpose

This chapter operationalizes the branching and version-control regime that binds every line of code, configuration, schema migration, runbook, and infrastructure artifact committed to the Opus Publica repositories. Branching and version control are not a matter of personal taste; they are the substrate on which review, certification, release, rollback, and audit depend. A repository whose branching model is undisciplined cannot produce a defensible release artifact, cannot support a clean rollback, and cannot withstand an audit. The rules in this chapter convert Volume I Chapter 5.02 and Volume I Chapter 8 (Repository Constitution) into a set of operational steps, checklists, and gates that an engineer SHALL apply at every commit, every branch, and every merge.

The discipline specified here is deliberately trunk-based. Long-lived feature branches accumulate conflict, drift from main, hide work from reviewers, and make rollback a forensic exercise. Trunk-based development, with short-lived branches, atomic commits, and protected long-lived branches (main, release/*, hotfix/*), produces the inverse: small, reviewable, reversible units of change whose history is intelligible to a reviewer six months later. This chapter SHALL be read in conjunction with Chapter 4 (Commits and Pull Requests) and Chapter 5 (Code Review Regime); the three chapters form a single operational discipline that governs how change enters the codebase.

Constitution Reference: Volume I, Chapter 5.02 (Branching and Version Control); Volume I, Chapter 8 (Repository Constitution) — BR-*

3.2 Branching Model

Opus Publica SHALL adopt a trunk-based development model. The trunk (the main branch) is the single source of runtime truth; it SHALL always be deployable, always green on CI, and always reflect the most recent certified state of the platform. All work occurs on short-lived branches cut from main and merged back to main through the pull-request process specified in Chapter 4. Long-lived release branches and hotfix branches are permitted but SHALL follow the lifecycle rules in this section.

Branch names SHALL be machine-parseable and SHALL encode the task identifier and a short slug. The canonical format is op-<task-id>-<slug>, where <task-id> is the task tracker's stable identifier (for example, OP-1234) and <slug> is a kebab-case summary of no more than four words. The task ID is mandatory because it is the join key between the version-control history and the Knowledge Graph (Volume II Chapter 23); a branch without a task ID is not traceable, and an untraceable branch SHALL NOT be merged.

Branch Type	Naming Pattern	Lifespan	Allowed Operations
Trunk / main	main	Permanent	Fast-forward or squash-merge from feature branches; merge-commit

			from release/* and hotfix/*.
Feature branch	op-<task-id>-<slug>	Maximum 7 calendar days	Created from main; squash-merged back to main via PR; deleted on merge.
Release branch	release/<rc-id> (e.g., release/rc1.0)	Until release is retired	Cut from main at release freeze; receives cherry-picks of fixes only.
Hotfix branch	hotfix/<issue-id>-<slug>	Maximum 3 calendar days	Cut from release/* or main; merged back to BOTH via separate PRs.
Build-agent scratch	op-<task-id>-<slug>-abuild	Maximum 24 hours	Created by the build agent; never merged directly; output is rebased onto a feature branch by the engineer.

Branch age is a first-class invariant. A feature branch older than seven calendar days is, by definition, stale; the engineer SHALL either rebase it onto main and open (or reopen) the pull request, or close the branch and re-create the work as smaller units. The seven-day limit exists because conflict cost grows super-linearly with branch age; a branch two weeks old is no longer a code-review problem, it is a merge problem. The repository's CI SHALL report branch age daily and SHALL flag branches approaching the limit at five days.

3.3 What Is Version-Controlled

The repository is the single source of truth for everything required to build, test, deploy, operate, and audit the platform. If an artifact is required to reproduce a build, reproduce a runtime behaviour, respond to an incident, or pass an audit, that artifact SHALL be version-controlled. The Repository Constitution (Volume I Chapter 8) defines the canonical layout; this section enumerates what SHALL live in version control and what SHALL NOT.

Artifact Class	Version-Controlled?	Location	Notes
Application source code	Yes (mandatory)	/src	Single implementation per Volume I Ch 7 7.5.
Configuration (env, app, infra)	Yes (mandatory)	/config	Templated; secrets injected at deploy time, never in repo.
Schema migrations	Yes (mandatory)	/migrations	Forward-only; reversible

			via inverse migration.
Documentation (engineering)	Yes (mandatory)	/docs	Markdown; ADRs live in /docs/adr.
Architecture Decision Records	Yes (mandatory)	/docs/adr	Numbered; immutable once merged; see Volume I Ch 8.
Build-agent prompts	Yes (mandatory)	/.build/prompts	Subject to review like any other artifact.
Runbooks	Yes (mandatory)	/runbooks	On-call runbooks; reviewed quarterly.
Infrastructure-as-Code (Terraform/Pulumi)	Yes (mandatory)	/infra	State stored in remote backend, NOT in repo.
Feature flag definitions	Yes (mandatory)	/config/flags	Flag state is runtime; definitions are versioned.
Test fixtures (non-PII)	Yes (mandatory)	/tests/fixtures	Synthetic; PII-redacted.
Secrets (API keys, credentials)	NO - forbidden	Secret manager	Committing a secret is a SEV-1 incident; see 3.8.
Personally Identifiable Information	NO - forbidden	n/a	Use synthetic data; see Chapter 7.
Binaries larger than 1 MB	NO - use Git LFS	Object storage + LFS pointer	Repo bloat degrades clone/CI performance.
Build outputs (jars, wheels, dist/)	NO - forbidden	CI artifact store	Reproducible from source; never committed.

Secret Detection is Mandatory

A pre-commit hook SHALL scan every staged change for high-entropy strings and known secret patterns (AWS keys, private keys, connection strings, JWT secrets). A positive detection SHALL block the commit. If a secret is committed despite the hook, the secret SHALL be rotated immediately and the incident SHALL be filed as SEV-1 per Chapter 14. Git history is not a defense; secrets in history remain secrets in history.

3.4 Commit Conventions

Commits SHALL follow the Conventional Commits 1.0 specification. Conventional Commits are not aesthetic; they are the input to automated changelog generation, semantic version bumping, and release-note drafting. A repository whose commit history is unstructured cannot produce a defensible

changelog, and a release without a defensible changelog fails the Production Acceptance Gate (Chapter 19).

Every commit message SHALL consist of a header line, an optional body, and an optional footer. The header line SHALL be of the form `<type>(<scope>): <summary>`, where `<type>` is one of the enumerated types below, `<scope>` is the affected module (e.g., submissions, crossref, auth), and `<summary>` is a imperative-mood phrase of no more than 72 characters. The body, if present, SHALL explain the why of the change, not the what (the diff already shows the what). The footer, if present, SHALL carry metadata such as the task ticket reference and the BREAKING CHANGE marker.

Type	When to Use	Example
feat	A new user-visible feature.	feat(crossref): add deposit-status polling
fix	A bug fix for a user-visible defect.	fix(auth): correct OAuth redirect on expired session
docs	Documentation-only changes (engineering docs, ADRs, runbooks).	docs(adr): add ADR-0042 on Crossref retry policy
refactor	Code change that neither adds a feature nor fixes a bug.	refactor(submissions): extract validation into pure functions
test	Adding or correcting tests.	test(submissions): cover concurrent edit conflict
chore	Build, tooling, dependencies, CI config.	chore(ci): bump Python action to v5
perf	Performance improvement that does not change behaviour.	perf(search): index the DOI column
build	Changes to the build system or external dependencies.	build(deps): upgrade Flask to 3.0
ci	Changes to CI configuration files and pipelines.	ci(github-actions): add staging deploy job
revert	Revert a previous commit.	revert: feat(crossref): add deposit-status polling

The following constraints are mandatory. (1) The header line SHALL NOT exceed 72 characters; CI SHALL reject longer headers. (2) The body SHALL be wrapped at 100 characters. (3) The footer SHALL cite the task ticket, e.g., Refs: OP-1234. (4) Every commit SHALL be GPG-signed or SSH-signed; unsigned commits SHALL be rejected by the branch protection rule. (5) Commits SHALL be atomic: a single commit SHALL address a single concern. A commit that mixes a feature, a refactor, and a dependency bump is not atomic and SHALL be split. (6) A commit SHALL NOT leave the codebase in a non-building state; bisectability depends on this.

feat(crossref):	poll	deposit	status	until	terminal
Crossref deposits are asynchronous; until now the platform displayed a pending state indefinitely. This commit adds a background poller that queries the Crossref deposit-status endpoint at exponentially backed-off intervals until the deposit reaches a terminal state (success or failure).					
Refs:	OP-1234				
Constitution:	Vol	I	Ch	14	14.3 (Crossref Deposit)
Signed-off-by:	Jane		Engineer		<jane@opuspublica.com>

3.5 Merge Strategy

The merge strategy SHALL be selected by branch type, not by engineer preference. Feature branches are squash-merged to main so that main's history is a sequence of atomic, reviewable changes - one commit per PR, with the Conventional Commit header drawn from the PR title. Release branches receive merge commits so that the cherry-picks of fixes are individually identifiable in the release history. Hotfix branches are merged to BOTH their source branch and main, in that order, via two separate PRs. Trivial changes (whitespace, typos, dependency bumps with no behaviour change) MAY be fast-forwarded if CI permits and the change is one commit.

Branch Pair	Strategy	Rationale	Commit on Target
feature/* -> main	Squash merge	One reviewable commit per PR; clean main history.	1 (squashed)
release/* -> release/*	Merge commit	Cherry-picked fixes are individually traceable.	1 per fix
hotfix/* -> release/*	Merge commit	Hotfixes are individually traceable in release history.	1 per hotfix
hotfix/* -> main	Merge commit	Hotfixes flow back to main as merge commits with a backport reference.	1 per hotfix
release/* -> main	Merge commit (at retirement)	Release is folded back to main at end-of-life.	1 final merge
main -> main (trivial)	Fast-forward	Only when one commit and CI is green.	1

Before any merge, the source branch SHALL be rebased onto the target branch's latest tip and CI SHALL pass on the rebased branch. A merge that is not rebased SHALL be rejected by the branch protection rule. The reviewer (not the implementer) SHALL perform the merge; this is the operational form of Rule PROC-003 (separation of authorship and approval). The squash-merge commit message SHALL be the PR title, prefixed with the Conventional Commit type, followed by the PR number and the task ticket reference in the footer.

3.6 Branch Protection Rules

Branch protection is the operational enforcement of every rule in this chapter. Protection rules SHALL be configured on the repository hosting platform (GitHub, GitLab, or equivalent) and SHALL be reviewed quarterly by the Engineering Leadership Team. A repository whose long-lived branches are not protected is, by definition, out of process; protection SHALL NOT be disabled except by the Chief Systems Architect with a recorded reason, and SHALL be re-enabled within 24 hours.

Branch	Required Reviews	Required CI Checks	Sign-off	Force-push	Delete-on-merge
main	2 (R1/R2); 1 (R3)	lint, unit, integration, contract, SAST, secret-scan, docs-build	GPG or SSH signature required	Forbidden (always)	n/a (permanent)
release/*	2 (one SHALL be RCA)	lint, unit, integration, contract, SAST, secret-scan, smoke	GPG or SSH signature required	Forbidden (always)	n/a (until retired)
hotfix/*	2 (one SHALL be Release Manager)	lint, unit, integration, contract, SAST, secret-scan	GPG or SSH signature required	Forbidden (always)	Yes (after merge to both targets)
feature/*	1 (R3); 2 (R1/R2)	lint, unit, integration, SAST, secret-scan	Recommended	Allowed by author only	Yes (after squash-merge)

Procedure: Creating a Branch

Step 1. Verify the task ticket exists The task ticket SHALL exist in the tracker with a stable ID (e.g., OP-1234) and a Constitution clause reference. No ticket, no branch.

Step 2. Pull the latest main Run `git checkout main && git pull --ff-only`. The local main SHALL be at the remote tip before branching; stale main produces stale branches.

Step 3. Create the branch with the canonical name Run `git checkout -b op-<task-id>-<slug>`. The slug SHALL be kebab-case, no more than four words, and SHALL describe the change (e.g., `op-op-1234-crossref-poll`).

Step 4. Push the branch and set upstream Run `git push -u origin op-<task-id>-<slug>`. The branch SHALL be pushed before any commit so the host platform's branch-age tracking begins immediately.

Step 5. Verify the pre-commit hook is active The pre-commit hook (secret scan, lint, conventional-commit check) SHALL be installed locally. Run `git config core.hooksPath .githubhooks` if it is not.

Step 6. Open a draft PR (optional but recommended) For work that will exceed one session, open a draft PR immediately so CI runs on every push and the work is visible to the team.

Procedure: Committing Changes

Step 1. Stage atomic units of change Stage only the files and hunks that constitute a single concern. Use `git add -p` to stage hunks selectively. Mixed-concern commits SHALL be split.

Step 2. Verify the pre-commit hook passes The hook SHALL run secret-scan, lint, and conventional-commit validation. If the hook reports a secret, STOP, rotate the secret, and file a SEV-1.

Step 3. Write the commit message Use the Conventional Commits format. Header ≤ 72 chars; body wrapped at 100 chars; footer with Refs: `<task-id>` and Constitution: `<clause ref>`. See 3.4.

Step 4. Sign the commit Use `git commit -S` to sign with GPG or SSH. Unsigned commits SHALL be rejected at push time by the branch protection rule.

Step 5. Verify the commit leaves the codebase buildable Run the local smoke test. A commit that breaks the build breaks bisectability and SHALL be amended before pushing.

Step 6. Push the commit Run `git push`. CI SHALL run on the pushed commit; the engineer SHALL NOT open a PR until CI is green on the latest commit.

Procedure: Merging a PR

Step 1. Confirm all required reviews are recorded The PR SHALL have the required number of approvals for its risk class (see Chapter 4 4.4). The reviewer (not the implementer) SHALL perform the merge.

Step 2. Confirm all required CI checks are green Every required CI check SHALL be green on the latest commit of the PR branch. A yellow or red check blocks the merge; the implementer SHALL resolve it.

Step 3. Rebase the branch onto the latest main Run `git fetch origin && git rebase origin/main` on the PR branch. Resolve any conflicts. Push the rebased branch. CI SHALL re-run and SHALL be green.

Step 4. Select the merge strategy per 3.5 Squash-merge for feature branches; merge-commit for release/* and hotfix/*. The PR title SHALL be the squash-merge commit message.

Step 5. Confirm the squash-merge message is correct The squash message SHALL be the Conventional Commits header from the PR title, with the body and footer preserved. Do not accept the host platform's default if it omits the footer.

Step 6. Delete the source branch on merge For feature/* and hotfix/* branches, delete-on-merge SHALL be enabled. Stale branches accumulate drift and SHALL NOT be retained.

Step 7. Verify the merge commit is signed The merge commit SHALL be signed by the reviewer. Unsigned merge commits SHALL be reverted and re-merged with a signature.

Checklist: Branch and Commit (GATE)

- ☐ **BC-01** Branch name matches op-<task-id>-<slug> pattern.
- ☐ **BC-02** Branch is cut from the latest main; no stale base.
- ☐ **BC-03** Branch age is within the 7-day limit (feature) or 3-day (hotfix).
- ☐ **BC-04** Task ticket cited in the branch name and in every commit footer.
- ☐ **BC-05** Every commit uses Conventional Commits format; header <= 72 chars.
- ☐ **BC-06** Every commit is GPG/SSH signed.
- ☐ **BC-07** Every commit is atomic (single concern); codebase builds at every commit.
- ☐ **BC-08** Pre-commit hook passed (secret-scan, lint, conventional-commit).
- ☐ **BC-09** No secret, PII, or >1MB binary committed (verified by hook and by SAST).
- ☐ **BC-10** Merge strategy matches 3.5 table; reviewer (not author) performs the merge.

3.7 Roles and Ownership

Role	Authority	Boundaries
Engineer (Author)	Creates branches; commits; opens PRs; rebases.	Shall not approve own PR; shall not merge own PR; shall not disable hooks.
Reviewer	Approves PRs; performs the merge; signs the merge commit.	Shall not be the author; shall verify Conventional Commits compliance.
Tech Lead	Owens the repository's branching configuration; approves exceptions.	Cannot override branch protection; cannot extend branch age beyond 7 days.
Release Certification Authority	Approves merges to release/*; signs release merge commits.	Cannot bypass CI; cannot merge to release/* without RCA sign-off recorded.
Release Manager	Cuts release branches; merges hotfixes back to main.	Cannot cut a release branch without RCA certification; cannot disable delete-on-merge.

Chief Systems Architect	Sole authority to disable branch protection (with recorded reason).	Shall re-enable within 24 hours; shall not override an invariant failure.
-------------------------	---	---

3.8 Evidence and Records

Version control is itself the primary evidence store for Chapters 3, 4, and 5. The commit graph, the PR history, the CI logs, and the branch-protection configuration together constitute the auditable record of how every change entered the codebase. This evidence SHALL be retained for the lifetime of the platform and SHALL be exportable on demand for audit. The retention schedule in Volume II Chapter 31.8 governs the long-term retention of these artifacts.

Evidence	Source	Storage	Retention
Commit history (signed)	git log --show-signature	Version-control host; mirror to cold storage.	Lifetime of the platform.
Branch-protection configuration	Repo settings export	Infrastructure-as-Code in /infra/repos.	Lifetime of the platform.
CI run logs (per commit)	CI platform	CI platform hot; archive to object storage.	90 days hot; 1 year cold.
Secret-scan results	Pre-commit hook + CI job	CI platform; SEV-1 tickets for positives.	1 year for negatives; lifetime for positives.
Merge records	PR history on the host	Version-control host; mirror quarterly.	Lifetime of the platform.
Branch-age reports	Daily CI job	CI platform; escalation tickets for stale branches.	90 days.

Chapter 4 — Commits and Pull Requests

4.1 Purpose

This chapter operationalizes Volume I Chapter 5.03 (Commit and Pull-Request Conventions). A pull request is not a polite request for comment; it is the formal change-control record by which a Constitution clause becomes a runtime reality. Every PR SHALL carry the traceability metadata that links it to the Constitution clause it implements, the task ticket that authorized it, the acceptance criteria that verify it, the risk classification that scopes its review, and the rollback plan that contains it. A PR that lacks this metadata is not a PR; it is an uncontrolled change and SHALL be rejected at the gate.

The discipline specified here is the operational bridge between the branching regime of Chapter 3 and the review regime of Chapter 5. Branching governs how change is isolated; the PR governs how change is integrated; review governs how change is verified. An engineer who has read this chapter SHALL be able to open a PR that a reviewer can disposition in a single pass, with every required field present and every required gate documented.

Constitution Reference: Volume I, Chapter 5.03 (Commit and Pull-Request Conventions) — PR-*

4.2 PR Description Template

Every PR SHALL use the canonical description template below. The template is stored in the repository at `/.github/PULL_REQUEST_TEMPLATE.md` (or the host platform's equivalent) and is rendered automatically when a PR is opened. The engineer SHALL complete every section; sections that do not apply SHALL be marked N/A with a one-line justification. Empty sections are not permitted; an empty section is a section whose author has not thought about it.

```
## Summary
<One to three sentences describing what this PR changes and why.>

## Constitution Clause Reference
<Vol I/II chapter, section, and stable rule ID(s). e.g., Vol I Ch 14 14.3, EXT-001.>

## Task Ticket
<Task tracker ID, e.g., OP-1234. Link to the ticket.>

## Acceptance Criteria
<Bulleted list of binary, testable ACs derived from the clause. Each AC maps to a test.>

## Applicable Playbook Checklists
<List every checklist in this Playbook that applies. e.g., IM-*, RV-*, CT-*.>

## Risk Classification
<R1 / R2 / R3 - see 4.4. Justify the classification in one sentence.>
```

##	Testing	Summary
<Unit: X tests, Y% coverage of changed lines. Integration: ... Contract: ... E2E: ...>		
##	Screenshots /	Recordings
<For any user-visible change: before/after screenshots or short recordings of every state.>		
##	Breaking	Changes
<None / list. If any, include migration notes and downstream consumer impact.>		
##	Migration	Notes
<Schema migrations, config changes, feature-flag flips, data backfills. None if N/A.>		
##	Rollback	Plan
<Steps to revert this change in production. Include data-rollback if migration is involved.>		

4.3 PR Metadata Requirements

In addition to the description template, the PR SHALL carry structured metadata on the host platform. Structured metadata enables automated gating, release-note generation, and audit queries. The following fields SHALL be present on every PR; the host platform's labels, milestones, and linked-issue features are the mechanism.

Field	Required?	Example	Purpose
Task ticket link	Required (always)	OP-1234	Traceability from PR to authorized work.
Constitution clause label	Required (always)	clause:EXT-001	Traceability from PR to Constitution; enables Knowledge Graph link.
Risk class label	Required (always)	risk:R1 / risk:R2 / risk:R3	Drives reviewer count and SLA per 4.4 and 4.5.
Module label	Required (always)	module:crossref	Routes to CODEOWNERS; drives reviewer assignment.
Breaking-change label	Required if applicable	breaking-change	Triggers migration-review gate; alerts downstream consumers.
Feature-flag label	Required if applicable	flag:crossref-poll	Indicates the change is behind a flag; enables staged rollout.
Migration label	Required if applicable	migration:db	Triggers DBA review and

			migration-runbook gate.
Reviewer assignment	Required (always)	@owner/crossref	CODEOWNERS-driven; per 5.2.
Milestone	Required (always)	RC1.0	Maps PR to a release; enables release-note generation.

4.4 Risk Classification

Every PR SHALL carry a risk class. The risk class drives the number of required reviewers, the depth of the required testing, and the seniority of the approver. The engineer SHALL self-classify at PR open; the reviewer SHALL confirm or escalate the classification at first review. Misclassification (labeling an R1 change as R2) is itself a process violation and SHALL be recorded.

Risk Class	Criteria	Required Reviewers	Required Gates
R1 - Critical	Touches authn/authz, payments, PII, data integrity, infra, or any path governed by an engineering invariant (Vol II Ch 26). Any breaking change.	3 (one SHALL be a Principal Engineer or the Chief Systems Architect; one SHALL be the RCA)	Full test suite in prod-equiv env; invariant monitors observed for 24h; RCA sign-off before merge to release/*.
R2 - Standard	User-visible feature or fix that is not R1. Schema-additive migrations. New API endpoint. Configuration change with operational impact.	2 (one SHALL be a CODEOWNERS reviewer; both SHALL be senior to the author on the module)	Unit, integration, contract, SAST; smoke in prod-equiv env; rollback plan tested.
R3 - Trivial	Documentation-only, dependency bump with no behaviour change, typo fix, test-only addition. No user-visible behaviour change.	1 (CODEOWNERS reviewer)	Lint, unit, SAST; docs build if docs change.

4.5 PR Size Limits

PR size is a first-class quality signal. Large PRs are not thoroughly reviewed, regardless of reviewer skill; this is a finding replicated across the software engineering literature and confirmed by Opus Publica's own review history. Size limits are therefore mandatory and SHALL be enforced by CI on every PR.

Metric	R1 (Critical)	R2 (Standard)	R3 (Trivial)
Max lines changed (added+removed)	1,000 (with split justification)	500	200
Max files touched	50 (with split justification)	30	10
Max lines per file	400	300	100
Review SLA	24 hours	48 hours	12 hours

When an R1 or R2 PR exceeds its size limit, the engineer SHALL either split it into smaller PRs or include a documented split-justification in the PR description that explains why the change cannot be decomposed. The split-justification SHALL be reviewed by a Principal Engineer, who SHALL approve or reject the justification. Splitting guidance: prefer to split by concern (a refactor and a feature are two PRs); prefer to split by layer (an API change and its UI consumer are two PRs, the API first); prefer to split by file cluster (changes to unrelated modules are two PRs).

4.6 PR Lifecycle

A PR traverses a defined state machine. Each transition has an entry condition, an owner, and an evidence artifact. The host platform's PR states (draft, open, closed, merged) are mapped onto the operational states below; the operational states are the ones this Playbook governs.

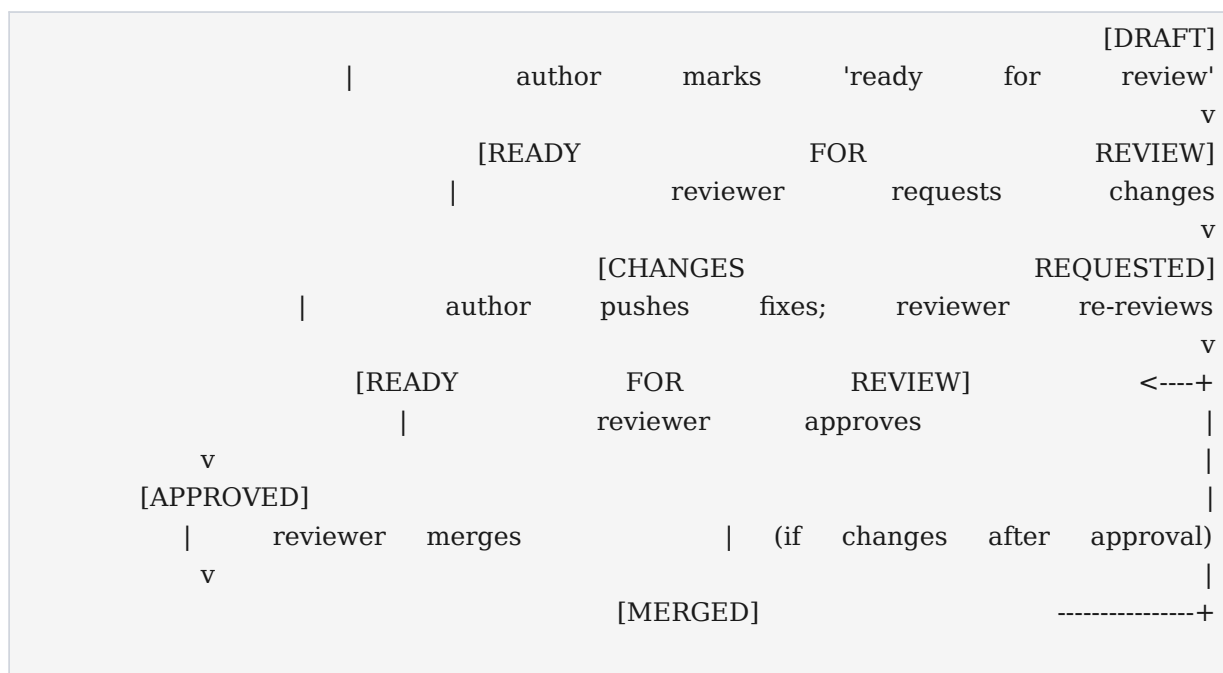


Figure 4.1: PR State Machine

State	Entry Condition	Owner	Evidence
DRAFT	PR opened; CI may be red; not yet ready for review.	Author	PR exists in draft state on the host platform.
READY FOR REVIEW	Author marks ready; CI is green; metadata complete.	Author	Ready-for-review label; CI green on latest commit.
CHANGES REQUESTED	Reviewer posts blocking comments; PR is not approved.	Reviewer	Review comments with classification per 5.4.
APPROVED	All required reviewers approve; all checks green.	Reviewer(s)	Approval records on the host platform.
MERGED	Reviewer (not author) merges per 3.5 strategy.	Reviewer	Merge commit on the target branch; source branch deleted.
CLOSED (without merge)	PR is abandoned or superseded; author or tech lead closes.	Author or Tech Lead	Closing comment with reason; linked superseding PR if any.

Procedure: Opening a PR

Step 1. Verify the implementation checklist is satisfied All items in IM-* (Chapter 2 2.5) SHALL be satisfied. The branch SHALL be rebased on the latest main; CI SHALL be green on the latest commit.

Step 2. Render the PR description template Open the PR on the host platform; the template at `/.github/PULL_REQUEST_TEMPLATE.md` SHALL render automatically. Complete every section; mark N/A with justification.

Step 3. Apply the required metadata Apply the labels and links per 4.3: clause label, risk-class label, module label, task ticket link, milestone. CI SHALL verify all required labels are present.

Step 4. Self-classify the risk Apply the risk-class label per 4.4. If R1, flag the PR for Principal Engineer and RCA review in the description.

Step 5. Run self-review Review your own diff against the Review Checklist (Chapter 5 5.3). Fix issues found in self-review before requesting peer review.

Step 6. Request review Assign the CODEOWNERS reviewer(s) per 4.4. Set the due date per the SLA in 5.5. Do NOT request review from the implementer (yourself) on any path.

Step 7. Verify CI is green on the latest commit CI SHALL be green before the reviewer is expected to begin. If CI is red, the review SLA does not start; the reviewer SHALL NOT review a red PR.

Procedure: Updating a PR After Review

Step 1. Read every review comment and classify it Comments are blocking, non-blocking, or nit per 5.4. Blocking comments SHALL be addressed; non-blocking comments SHOULD be addressed or explicitly deferred with a ticket; nits are at author discretion.

Step 2. Address blocking comments in the order posted Address each blocking comment in a dedicated commit. The commit message SHALL reference the comment (e.g., 'fix: address review comment on retry backoff').

Step 3. Reply to every comment Reply to every comment - blocking, non-blocking, or nit - with the action taken ('fixed in <commit>' / 'deferred to OP-XXXX' / 'agreed, will follow up'). An unanswered comment is an incomplete review response.

Step 4. Rebasing onto the latest main (if main has moved) If main has moved since the PR was opened, rebase. Resolve conflicts. Push. CI SHALL re-run and SHALL be green before re-requesting review.

Step 5. Re-request review Re-request review from the original reviewer(s). If the changes are substantive (new behaviour, new tests), the reviewer SHALL re-review in full. Trivial changes (typo, whitespace) do not require full re-review.

Step 6. Do not force-push over reviewed commits without notice If a rebase rewrites history, post a comment stating the rebase happened and what changed. Reviewers SHALL NOT be expected to re-review a silent force-push.

Procedure: Merging a PR

Step 1. Verify all approvals are recorded and current All required reviewers SHALL have approved the latest commit. An approval on an earlier commit is stale if the author has pushed substantive changes since.

Step 2. Verify all required CI checks are green Every required CI check SHALL be green on the latest commit. The reviewer SHALL NOT merge a PR with a yellow or red check.

Step 3. Verify the PR description and metadata are complete Every section of the template SHALL be complete; every required label SHALL be present. The reviewer SHALL NOT merge a PR with an empty section.

Step 4. Verify the merge strategy per 3.5 Squash-merge for feature branches; merge-commit for release/* and hotfix/*. The reviewer (not the author) SHALL perform the merge.

Step 5. Verify the merge commit is signed The reviewer SHALL sign the merge commit with GPG/SSH. The host platform SHALL verify the signature; unsigned merge commits SHALL be reverted.

Step 6. Delete the source branch For feature/* and hotfix/* branches, delete-on-merge SHALL be enabled. The reviewer SHALL verify the branch is deleted after merge.

Step 7. Update the task ticket Move the task ticket to the next state (e.g., 'In Certification' or 'Done'). Link the merge commit in the ticket for traceability.

Checklist: Pull Request (GATE)

- ☐ **PR-01** PR description template complete; every section filled or marked N/A with justification.
- ☐ **PR-02** Constitution clause reference cited by stable ID (e.g., EXT-001).
- ☐ **PR-03** Task ticket linked; acceptance criteria present and testable.
- ☐ **PR-04** Risk classification applied and justified (R1/R2/R3).
- ☐ **PR-05** Module label and CODEOWNERS reviewer(s) assigned per 4.4.
- ☐ **PR-06** PR size within limits (4.5) or split-justification approved by a Principal Engineer.
- ☐ **PR-07** All required CI checks green on the latest commit.
- ☐ **PR-08** Self-review completed; Review Checklist (5.3) applied by the author.
- ☐ **PR-09** Testing summary present; coverage of changed lines reported.
- ☐ **PR-10** Screenshots/recordings present for every user-visible state.
- ☐ **PR-11** Rollback plan present and tested in production-equivalent env (R1/R2).
- ☐ **PR-12** Merge performed by a reviewer (not the author); merge commit signed; source branch deleted.

4.7 Roles and Ownership

Role	Authority	Boundaries
Author	Opens the PR; classifies risk; addresses comments; updates the ticket.	Cannot approve own PR; cannot merge own PR; cannot self-classify R1 down to R2.
Reviewer	Approves, requests changes, or rejects; performs the merge.	Shall not be the author; shall cite the Constitution clause satisfied (PROC-004).
CODEOWNERS Reviewer	Mandatory reviewer for the module; routes to domain expertise.	Shall not delegate without recording; shall not approve outside own module.
Principal Engineer	Approves split-justifications; second reviewer for R1.	Cannot override Constitution; cannot approve without RCA on R1.
Release Certification Authority	Approves merges to release/*; verifies certification readiness.	Cannot certify a PR with an open blocking comment; cannot bypass invariant monitors.
Tech Lead	Owens the PR template; owns the CODEOWNERS file; closes stale PRs.	Cannot override branch protection; cannot extend the SLA beyond 2x.

4.8 Evidence and Records

The PR is itself an evidence record. The PR description, the review comments, the approvals, the CI runs, and the merge commit together constitute the audit trail for every change. This evidence SHALL be retained for the lifetime of the platform and SHALL be exportable on demand. The host platform's retention SHALL be supplemented by a quarterly export to cold storage, because host platforms have been known to lose history in incidents.

Evidence	Source	Storage	Retention
PR description and metadata	Host platform	Host platform; quarterly export to object storage.	Lifetime of the platform.
Review comments and approvals	Host platform	Host platform; quarterly export.	Lifetime of the platform.
CI run artifacts (per commit)	CI platform	CI platform hot; archive per Chapter 6 6.4.	90 days hot; 1 year cold.
Merge commit (signed)	git log	Version-control host; mirror to cold storage.	Lifetime of the platform.
Risk classification record	PR label and description	Host platform; quarterly export.	Lifetime of the platform.
Rollback-plan record	PR description 9	Host platform; mirrored to runbook repo if invoked.	Lifetime of the platform + 5 years after rollback.

Chapter 5 — Code Review Regime

5.1 Purpose

This chapter operationalizes Volume I Chapter 5.04 (Code Review) and the review-related procedures of Volume I Chapter 3 3.3.6 (PROC-003, PROC-004, PROC-005). Code review is the gate at which a peer, who is not the implementer, verifies that the implementation satisfies the Constitution clause the PR claims to satisfy. Review is not a stylistic preference; review is the operational form of the Principle of Runtime Truth applied to the moment of merge. A review that approves without verifying the clause, without running the changed code, and without applying the checklist is not a review; it is a rubber stamp, and a rubber stamp is a process violation that SHALL be recorded.

The regime specified here applies to every PR regardless of authorship, including PRs authored by the build agent (Antigravity). Build-agent output is subject to the FULL review regime; the reviewer SHALL verify that the build agent has not introduced fallback hacks, magic strings, duplicated logic, or any of the anti-patterns enumerated in 5.7. The reviewer SHALL verify the implementation matches the Constitution clause, not just the prompt that was given to the build agent. This is the operational form of Rule PROC-005.

Constitution Reference: *Volume I, Chapter 5.04 (Code Review); Volume I, Chapter 3 3.3.6 (PROC-003, PROC-004, PROC-005) — PROC-003 / PROC-004 / PROC-005*

5.2 Reviewer Assignment

Reviewer assignment SHALL be driven by the CODEOWNERS file maintained at `/.github/CODEOWNERS` (or the host platform's equivalent). The CODEOWNERS file maps every path in the repository to one or more owning teams or individuals; the host platform SHALL automatically request review from the owners of every path touched by the PR. The reviewer SHALL NOT be the implementer; this is the operational form of Rule PROC-003 (separation of authorship and approval). Self-review is required before requesting peer review (Chapter 4 4.6) but does not satisfy the peer-review requirement.

Risk Class	Reviewer Count	Required Expertise	SLA
R1 - Critical	3 (one Principal Engineer or CSA; one RCA; one CODEOWNERS reviewer)	Deep domain expertise on the affected module; familiarity with the governing Constitution chapter and the engineering invariants in play.	24 hours from ready-for-review.
R2 - Standard	2 (both CODEOWNERS reviewers; both senior to the author on the	Working knowledge of the affected module and its Constitution chapter.	48 hours from ready-for-review.

	module)		
R3 - Trivial	1 (CODEOWNERS reviewer)	Familiarity with the module's conventions.	12 hours from ready-for-review.

When the CODEOWNERS reviewer is unavailable (vacation, illness, conflict of interest), the tech lead SHALL assign a deputy with documented domain expertise. A PR SHALL NOT be reviewed by an engineer who lacks the expertise to evaluate it; an unqualified reviewer is worse than no reviewer because it creates the illusion of oversight. Reviewer expertise is verified by the tech lead at quarterly CODEOWNERS review.

5.3 The Review Checklist

The master review checklist is the operational form of Volume I Chapter 5.04. Every reviewer SHALL apply every applicable item on every PR. The checklist is grouped by concern; the groups are applied in the order below because each group's verification depends on the prior group's. A reviewer who skips a group is not performing a review; the reviewer SHALL mark each item satisfied (with evidence) or N/A (with justification).

Group A - Constitution Traceability

- ☐ **RV-A1** PR cites the Constitution chapter and stable rule ID it implements.
- ☐ **RV-A2** Each acceptance criterion maps to a clause sub-requirement.
- ☐ **RV-A3** The implementation actually satisfies the cited clause (verified by reading the code, not the PR description).
- ☐ **RV-A4** If a build agent authored the code, the implementation matches the Constitution clause, not just the prompt (PROC-005).

Group B - Coding Standards (Vol I Ch 7)

- ☐ **RV-B1** Single implementation; no duplicated logic (Vol I Ch 7 7.5).
- ☐ **RV-B2** No magic strings or magic numbers; named constants used (Vol I Ch 7 7.4).
- ☐ **RV-B3** No fallback hacks that mask failure (Vol I Ch 7 7.5); errors are surfaced, not swallowed.
- ☐ **RV-B4** Idempotency contract honoured where applicable (retries produce same end-state).
- ☐ **RV-B5** Code is readable; naming is intention-revealing; comments explain why, not what.

Group C - Error Handling and Resilience

- ☐ **RV-C1** All error paths are tested; failure modes are explicit, not silent.
- ☐ **RV-C2** Retries are bounded and use exponential backoff with jitter where applicable.
- ☐ **RV-C3** External calls have timeouts; no unbounded waits.

- ☐ **RV-C4** Partial-failure modes are handled; the system does not enter an inconsistent state.

Group D - Testing

- ☐ **RV-D1** Unit tests cover the changed lines and the new branches.
- ☐ **RV-D2** Integration tests cover the new cross-module interactions.
- ☐ **RV-D3** Contract tests cover any API change (consumer and provider sides).
- ☐ **RV-D4** End-to-end tests cover the user-visible behaviour (R1/R2 only).
- ☐ **RV-D5** Tests are deterministic; no flaky tests introduced (see 6.7).

Group E - Security

- ☐ **RV-E1** No secrets committed; secret-scan is green.
- ☐ **RV-E2** Input validation on every trust boundary; no SQL injection, XSS, or SSRF vectors.
- ☐ **RV-E3** Authn/authz enforced on every new endpoint; least privilege applied.
- ☐ **RV-E4** SAST is green; findings triaged or suppressed with recorded justification.
- ☐ **RV-E5** PII handling compliant with Vol II Ch 30 (Privacy Constitution); no PII in logs.

Group F - Observability

- ☐ **RV-F1** Structured logging on every new code path; log level appropriate.
- ☐ **RV-F2** Metrics emitted for every new user-visible operation.
- ☐ **RV-F3** Tracing spans added for cross-service calls.
- ☐ **RV-F4** Alerts defined for new failure modes; alert thresholds sane.

Group G - Documentation

- ☐ **RV-G1** Public API documentation updated (OpenAPI / GraphQL schema).
- ☐ **RV-G2** Runbook updated or added for new operational surface.
- ☐ **RV-G3** ADR created or updated for any architectural decision.
- ☐ **RV-G4** Changelog entry generated by Conventional Commits is accurate.

Group H - Performance

- ☐ **RV-H1** No N+1 queries introduced; queries are batched or cached as appropriate.
- ☐ **RV-H2** Memory and CPU impact assessed for hot paths; no unbounded growth.
- ☐ **RV-H3** Performance regression tests run for R1/R2 changes; no regression > 5%.

5.4 Comment Classification

Every review comment SHALL be classified by the reviewer. Classification determines whether the comment blocks merge and what response is required. The host platform's review features (resolve, request changes, approve) SHALL be used to record classification; a comment without classification is not a review comment, it is a chat message, and chat messages do not gate merges.

Type	Definition	Blocks Merge?	Response Required
Blocking	The PR as written violates a Constitution clause, a Coding Standard, a security requirement, or has a correctness or data-integrity bug.	Yes - PR SHALL NOT be merged until addressed or explicitly waived by the CSA.	Author SHALL address in a new commit; reviewer SHALL re-review the fix.
Non-blocking	The PR is correct but could be improved; the improvement is worth doing but not worth blocking the merge.	No - PR MAY be merged with the comment recorded.	Author SHALL address or explicitly defer with a recorded ticket (e.g., 'OP-XXXX').
Nit	Style, wording, or cosmetic preference with no operational impact.	No - PR MAY be merged with the nit ignored.	Author MAY address at discretion; no ticket required if ignored.
Question	Reviewer does not understand the code; author SHALL clarify.	Yes if unresolved - the reviewer SHALL NOT approve a PR they do not understand.	Author SHALL clarify in the code, in a comment, or in the PR description.

5.5 Review SLA

Review SLA is the maximum elapsed time from ready-for-review to first substantive reviewer response (approval, request-changes, or question). The SLA is per risk class. Missing the SLA is not a violation in itself, but it triggers an escalation that SHALL be resolved; a PR that sits in ready-for-review beyond 2x SLA is a process failure and SHALL be raised at the next Engineering Leadership stand-up.

Risk Class	Review SLA	Escalation at 1x SLA	Escalation at 2x SLA
R1 - Critical	24 hours	Notify tech lead; reassign if reviewer unavailable.	Notify Principal Engineer and CSA; PR is blocking the release; cannot proceed.
R2 - Standard	48 hours	Notify tech lead; deputy	Notify Engineering

		reviewer assigned.	Leadership; raise at stand-up; review is blocking the milestone.
R3 - Trivial	12 hours	Notify tech lead; reassign.	Auto-merge after second reviewer (R3-only) confirms trivial; record the SLA miss.

5.6 Build-Agent Authored Code Review

Code authored by the build agent (Antigravity) is subject to the FULL review regime specified in this chapter. There is no reduced regime for build-agent output; the reviewer SHALL apply every applicable item in 5.3. In addition, the reviewer SHALL apply the build-agent-specific verifications below. These verifications exist because build agents are prone to specific failure modes that human authors are not: they generate plausible-looking code that satisfies the prompt but not the underlying Constitution clause; they introduce fallback hacks to make tests pass; they duplicate logic because they do not have global context; they hard-code values that should be configurable.

■ **Constitution Reference: Volume I, Chapter 3 3.3.6 (PROC-005: build-agent output reviewed) — PROC-005**

Build-Agent-Specific Review Checklist (in addition to 5.3)

- ☐ **BA-01** Implementation matches the Constitution clause, not just the build-agent prompt (PROC-005).
- ☐ **BA-02** No fallback hacks introduced to make tests pass (Vol I Ch 7 7.5); tests test real behaviour.
- ☐ **BA-03** No magic strings or hard-coded values that should be configurable (Vol I Ch 7 7.4).
- ☐ **BA-04** No duplicated logic; existing utilities were reused, not reinvented (Vol I Ch 7 7.5).
- ☐ **BA-05** No 'TODO: fix later' or 'temporary workaround' comments without a linked ticket.
- ☐ **BA-06** Error handling is real, not 'return None on any exception' style.
- ☐ **BA-07** Tests are not over-mocked to the point of testing nothing (anti-pattern: testing mocks).
- ☐ **BA-08** Build-agent prompt is committed in /.build/prompts and reviewed alongside the code.

5.7 Review Anti-Patterns

The following review anti-patterns are forbidden. A reviewer who exhibits them is not performing a review and SHALL be coached by the tech lead. An author who exhibits them in self-review is not performing a self-review. The detection column lists how the anti-pattern is observed; the engineering leadership SHALL monitor for these patterns in the quarterly review-process audit.

Anti-Pattern	Why Forbidden	Detection
--------------	---------------	-----------

Rubber-stamp approval	Creates the illusion of oversight without the substance; invariants are violated.	Approval within minutes of assignment; no comments; reviewer did not run the code.
LGTM-with-comments	Approval with unresolved blocking comments; comments are ignored.	Approval recorded while blocking comments are unresolved.
Approving own PR	Violates PROC-003 (separation of authorship and approval).	Author and approver are the same GitHub/GitLab account.
Style-only review	Review addresses nits but not correctness, security, or Constitution traceability.	All comments are nits; no Group A/C/E items checked.
Trusting the prompt, not the clause	Build-agent output satisfies the prompt but not the Constitution clause (PROC-005).	Reviewer did not read the cited Constitution clause; clause not opened during review.
Approving without running	Violates the Principle of Runtime Truth; static review cannot verify runtime behaviour.	Reviewer did not run the changed code locally or in the preview env.
Burying blocking comments in nits	Blocking comments are missed; merge proceeds with a defect.	Blocking comment buried among 30 nits; not classified per 5.4.
Reviewing unfamiliar module	Reviewer lacks expertise to evaluate; creates illusion of oversight.	Reviewer is not in CODEOWNERS for any touched path; no deputy recorded.

Procedure: Conducting a Review

Step 1. Read the PR description and the cited Constitution clause Open the PR; read the description; open the cited Constitution chapter and rule in a side window. The reviewer SHALL NOT proceed without the clause open. (PROC-004: the reviewer cites the clause satisfied.)

Step 2. Run the changed code locally or in the preview env Check out the PR branch; run the local smoke test; exercise the new behaviour manually. The Principle of Runtime Truth applies to review: the reviewer SHALL verify runtime behaviour, not just static code.

Step 3. Apply the Review Checklist (5.3) group by group Apply Groups A through H in order. Mark each item satisfied (with evidence) or N/A (with justification). Do NOT skip groups.

Step 4. Classify every comment per 5.4 Every comment SHALL be blocking, non-blocking, nit, or question. Use the host platform's review features to record classification.

Step 5. Post the review If blocking comments exist: request changes. If only non-blocking/nits: approve with comments. If you do not understand the code: post questions and do NOT approve.

Step 6. Cite the Constitution clause satisfied (PROC-004) In the approval comment, the reviewer SHALL cite the Constitution clause and the Playbook checklist items the implementation satisfies. An approval without this citation is invalid.

Step 7. Re-review after author fixes When the author pushes fixes, re-review. If the changes are substantive, re-apply the full checklist. If trivial, verify the fix and re-approve.

Step 8. Perform the merge per 3.5 On final approval, the reviewer (not the author) performs the merge per the merge-strategy table. Verify the merge commit is signed.

Procedure: Responding to Review Comments

Step 1. Read every comment and note its classification Blocking comments SHALL be addressed; non-blocking comments SHOULD be addressed or deferred with a ticket; nits are at discretion; questions SHALL be answered.

Step 2. Address blocking comments first Address each blocking comment in a dedicated commit referencing the comment. Do not bundle multiple blocking fixes into one commit; atomicity applies.

Step 3. Reply to every comment Reply with the action taken: 'fixed in <commit>' / 'deferred to OP-XXXX' / 'agreed, will follow up' / 'clarification: ...'. An unanswered comment is an incomplete response.

Step 4. Resolve comments only after the reviewer confirms The author SHALL NOT resolve a blocking comment themselves; the reviewer SHALL resolve it after verifying the fix. Resolving your own blocking comment is a process violation.

Step 5. Push the fixes and re-request review Push the commits; re-request review from the original reviewer. Rebase if main has moved.

Step 6. Post a summary comment If the changes are substantive, post a summary of what changed since the last review so the reviewer can re-review efficiently.

Checklist: Code Review (GATE to Merge)

- ☐ **RV-01** Reviewer is not the author (PROC-003); CODEOWNERS reviewer assigned.
- ☐ **RV-02** Constitution clause cited by reviewer in the approval (PROC-004).
- ☐ **RV-03** Reviewer ran the changed code locally or in preview env (Principle of Runtime Truth).
- ☐ **RV-04** Review Checklist Group A (Traceability) applied; all items satisfied or N/A.
- ☐ **RV-05** Review Checklist Group B (Coding Standards) applied; no fallback hacks, no magic strings.
- ☐ **RV-06** Review Checklist Group C (Error Handling) applied.
- ☐ **RV-07** Review Checklist Group D (Testing) applied; tests are deterministic.
- ☐ **RV-08** Review Checklist Group E (Security) applied; SAST green; no secrets.
- ☐ **RV-09** Review Checklist Group F (Observability) applied.
- ☐ **RV-10** Review Checklist Group G (Documentation) applied.
- ☐ **RV-11** Review Checklist Group H (Performance) applied (R1/R2 only).

- ❑ **RV-12** Every review comment classified per 5.4 (blocking / non-blocking / nit / question).
- ❑ **RV-13** All blocking comments addressed; reviewer (not author) resolved them.
- ❑ **RV-14** If build-agent authored: BA-01 through BA-08 verified (PROC-005).
- ❑ **RV-15** Merge performed by reviewer per 3.5; merge commit signed.

5.8 Roles and Ownership

Role	Authority	Boundaries
Reviewer	Applies the checklist; approves, requests changes, or rejects; merges.	Shall not be the author; shall cite the clause; shall not skip groups.
CODEOWNERS Reviewer	Mandatory reviewer for the module's paths.	Shall not delegate without recording; shall not review outside own module.
Principal Engineer	Required reviewer for R1; approves split-justifications.	Cannot override Constitution; cannot waive an invariant.
Release Certification Authority	Verifies certification readiness; approves merges to release/*.	Cannot certify a PR with an open blocking comment.
Tech Lead	Owens the CODEOWNERS file; coaches reviewers; owns the quarterly review audit.	Cannot serve as the sole reviewer on R1; cannot extend SLA beyond 2x.
Chief Systems Architect	Sole authority to waive a blocking comment (with recorded reason).	Shall NOT waive an invariant violation; the implementation SHALL be corrected.

5.9 Evidence and Records

The review process produces evidence that SHALL be retained for the lifetime of the platform. The review comments, the approvals, the cited clauses, and the checklist results together constitute the auditable record that an implementation was reviewed against its Constitution clause. This evidence SHALL be exportable on demand and SHALL be mirrored to cold storage quarterly; the host platform is a convenience, not a system of record.

Evidence	Source	Storage	Retention
Review comments (classified)	Host platform	Host platform; quarterly export to object storage.	Lifetime of the platform.
Approval records (with clause citations)	Host platform	Host platform; quarterly export.	Lifetime of the platform.

Checklist results (per PR)	Reviewer's checklist	PR comment or review tool; mirrored quarterly.	Lifetime of the platform.
Reviewer assignment log	CODEOWNERS + host platform	Host platform; quarterly export.	Lifetime of the platform.
SLA reports	CI job (daily)	CI platform; escalation tickets for misses.	90 days hot; 1 year cold.
Quarterly review-process audit	Tech Lead	Engineering Leadership archive.	Lifetime of the platform + 5 years.
Build-agent prompt + output (PROC-005)	Build agent	/.build/prompts + PR history.	Lifetime of the platform.

Chapter 6 — Quality Assurance

6.1 Purpose

This chapter operationalizes Volume I Chapter 5.05 (Quality Assurance), Volume I Chapter 19 (Testing Constitution), and Volume II Chapter 20 (Runtime Verification Constitution). Quality assurance is not a department; it is a discipline that every engineer SHALL apply at every layer of the test pyramid, on every PR, at every release gate, and through every certification. The discipline specified here is the operational form of the Principle of Runtime Truth applied to verification: no implementation is correct because its tests are written; the implementation is correct because its tests are written, run, pass, and demonstrate the Constitution clause at runtime.

The QA regime binds the test pyramid (6.2), the test layers and their owners (6.3), the environments in which tests run (6.4, expanded in Chapter 7), the data on which tests operate (6.5), the gates at which tests block progress (6.6), and the maintenance regime that keeps tests trustworthy over time (6.7). An engineer who has read this chapter SHALL be able to write tests at the right layer, run them in the right environment, gate the right decision on their result, and maintain them as the system evolves.

Constitution Reference: Volume I, Chapter 5.05 (Quality Assurance); Volume I, Chapter 19 (Testing Constitution); Volume II, Chapter 20 (Runtime Verification Constitution) — QA-* / TEST-* / RV-*

6.2 The Test Pyramid

Opus Publica SHALL adopt the test pyramid as its test-architecture doctrine. The pyramid is not aesthetic; it reflects the economic reality that fast, isolated, deterministic tests at the base catch the majority of regressions at the lowest cost, while slow, integrated, non-deterministic tests at the apex catch integration regressions that the base cannot. A pyramid that is inverted (many end-to-end tests, few unit tests) is slow, flaky, and expensive; a pyramid that is a single layer (only unit tests, no integration) misses integration regressions and is a false sense of safety.

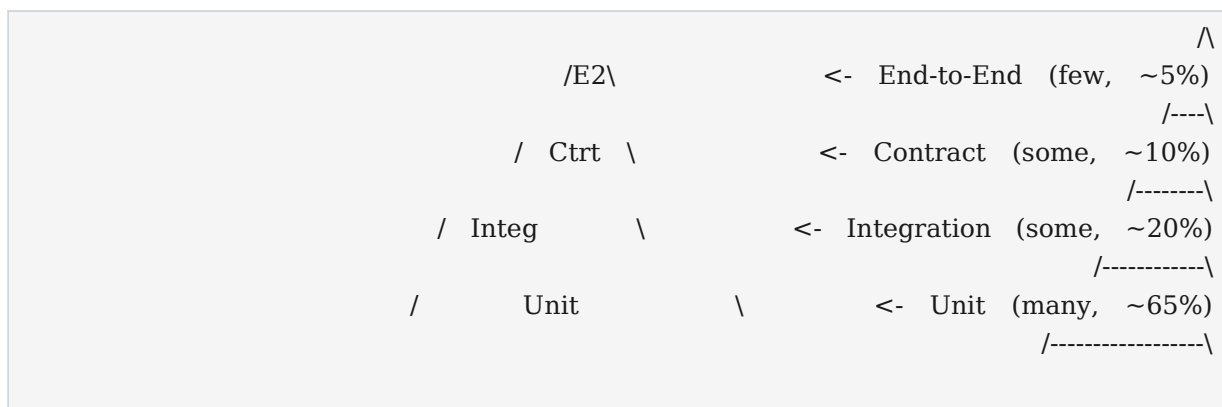


Figure 6.1: The Test Pyramid (target ratios by test count)

The target ratios are by test count, not by line count. A team with 1,000 unit tests, 300 integration tests, 150 contract tests, and 75 end-to-end tests is approximately on ratio. A team with 200 unit tests and 400

end-to-end tests is inverted and SHALL rebalance. The ratios are enforced at quarterly review by the QA Lead; a team out of ratio SHALL produce a rebalancing plan.

6.3 Test Layers and Ownership

Every test layer has a defined owner, a defined toolset, a defined coverage target, and a defined gating effect. A test without an owner is an orphan; an orphan test rots and SHALL be deleted per 6.7. The table below is the authoritative mapping; deviations SHALL be approved by the QA Lead and recorded in the team's ADR.

Layer	Definition	Owner	Tools	Coverage Target	Gating
Unit	Tests a single function or class in isolation; mocks collaborators.	Implementing engineer	pytest, unittest, vitest, jest	85% line; 75% branch on changed code	Merge gate
Integration	Tests a module with its real collaborators (DB, cache, queue) but not external services.	Implementing engineer	pytest, testcontainers	70% line on module boundaries	Merge gate
Contract	Tests that a service honours its published contract (consumer and provider sides).	Service-owning team	Pact, Spring Cloud Contract	100% of published contract endpoints	Merge gate + release gate
End-to-End	Tests a user-visible flow through the full system in a production-equivalent env.	QA engineer	Playwright, Cypress, Selenium	100% of critical user journeys (enumerated in /tests/e2e/journeys.md)	Release gate
Performance	Tests that the system meets SLOs under load (latency, throughput, resource use).	QA + Service-owning team	k6, Locust, JMeter	Every P0 user operation; regression < 5%	Release gate

Load	Tests sustained behaviour at expected peak load for a defined duration.	QA + SRE	k6, Locust	Every release; 1-hour soak at 2x expected peak	Release gate (R1 only)
Mutation	Tests that mutate the code to verify the test suite catches the mutation.	QA engineer	mutmut, Stryker, cosmic-ray	Mutation score $\geq 60\%$ on critical modules	Monthly gate (CI)
Chaos	Tests resilience by injecting failures (network, process, dependency).	SRE + QA	Chaos Mesh, Gremlin, Litmus	Every critical dependency has a chaos test	Quarterly gate
Accessibility	Tests that the UI meets WCAG 2.1 AA.	QA + Frontend engineer	axe-core, Pa11y, Lighthouse	100% of new screens; 0 critical violations	Merge gate (UI changes)
Security - SAST	Static analysis of source for vulnerabilities.	Security engineer	Semgrep, Bandit, CodeQL	0 high/critical findings; medium triaged	Merge gate
Security - DAST	Dynamic analysis of a running service for vulnerabilities.	Security engineer	OWASP ZAP, Burp Suite	Every release; 0 high/critical findings	Release gate
Security - SCA	Software composition analysis of dependencies for known CVEs.	Security engineer	Dependabot, Snyk, OSV-scanner	0 high/critical CVEs; medium triaged within 30 days	Merge gate + daily CI
Golden Snapshot	Compares a deterministic output (render, API response, document) to a stored golden.	Service-owning team	Approval tests, jest-image-snapshot	Every deterministic output has a golden; changes reviewed	Merge gate
Visual	Compares	QA + Frontend	Percy,	Every UI screen;	Merge gate (UI

Regression	rendered UI against a baseline image to detect unintended visual changes.	engineer	Chromatic, Playwright snapshots	< 0.1% pixel diff tolerated	changes)
------------	---	----------	---------------------------------	-----------------------------	----------

6.4 Test Environments

Tests run in environments appropriate to their layer. Unit tests run on the developer's laptop and in CI. Integration tests run in CI with testcontainers. End-to-end, performance, and security tests run in a production-equivalent staging environment. Chapter 7 specifies the environment topology, provisioning, access control, data seeding, drift detection, and production-equivalence requirements in depth; this section provides the summary mapping.

Environment	Purpose	Data	Access	Reset Policy
Local (developer laptop)	Unit tests; integration tests with testcontainers; smoke checks.	Synthetic; locally generated fixtures.	Engineer (own laptop).	Per-run; ephemeral.
CI (ephemeral runner)	All merge-gate tests; build; SAST; secret-scan.	Synthetic; generated in pipeline.	CI service account (read-only on prod).	Per-job; runner torn down after.
Staging	End-to-end tests; performance; DAST; release-gate tests.	Scrubbed production-like; refreshed weekly.	QA, engineers, release manager.	Per-release; reset on demand.
Production-equivalent (pre-prod)	Final certification tests; load; chaos (controlled).	Full-scale scrubbed production mirror.	QA, RCA, SRE; break-glass for others.	Per-release; drift-monitored against prod.
Production	Post-deploy verification; runtime invariant monitors; canary.	Real production data.	On-call + SRE only; break-glass per Chapter 13.	Never reset; only mutated by deploy.

6.5 Test Data Management

Test data is a first-class engineering concern. Tests that depend on stale, incomplete, or PII-laden data are unreliable and a compliance liability. The following rules govern test data at Opus Publica. (1) Synthetic data SHALL be used wherever possible; the platform SHALL maintain a synthetic-data

generator that produces realistic-but-fake submissions, users, journals, and Crossref deposits. (2) Fixtures SHALL be version-controlled alongside the tests that use them. (3) PII SHALL be redacted before any production-derived data enters staging; the redaction pipeline SHALL be tested itself. (4) Staging data SHALL be refreshed weekly from scrubbed production. (5) Test data SHALL be retained only for the duration of its need; long-lived staging data accumulates drift and SHALL be reset per the schedule in 6.4.

Data Class	Source	Redaction Required?	Refresh Cadence	Retention
Unit test fixtures	Synthetic generator	No (synthetic)	Per-test (generated in code)	Lifetime of the test.
Integration test data	Testcontainers + synthetic seed	No (synthetic)	Per-run (ephemeral)	Per-run.
Contract test data	Pact broker	No (synthetic)	On contract publish	Lifetime of the contract.
E2E test data (staging)	Scrubbed production mirror	Yes (mandatory)	Weekly	Until next refresh.
Performance test data	Scrubbed production at scale	Yes (mandatory)	Per-release	Until next release.
Chaos test data	Production-equivalent env	Yes (mandatory)	Per-release	Until next release.
Production data in tests	FORBIDDEN	n/a	n/a	n/a - never used in tests.

6.6 Gating Policy

Tests are not valuable unless they gate a decision. The gating policy specifies which tests block which decisions, on what pass criteria, and with what evidence. A test that does not gate a decision is a test that can be ignored; a test that can be ignored is a test that will rot. The table below is the authoritative mapping; deviations SHALL be approved by the RCA and recorded in the release record.

Gate	Tests Included	Pass Criteria	Enforced By
Merge gate (PR-to-main)	Unit, integration, contract (changed modules), SAST, SCA, secret-scan, accessibility (UI changes), golden snapshot, visual regression (UI).	100% pass; 0 high/critical SAST/SCA findings; coverage targets met.	CI (automated); reviewer confirms before merge.
Release gate (release/* certification)	End-to-end (all critical journeys), performance	100% pass; 0 high/critical findings; RCA reviews	RCA sign-off; release manager cannot proceed

	(regression < 5%), load (1-hour soak), DAST, chaos (controlled), full SAST + SCA.	evidence.	without it.
Production gate (deploy to production)	Post-deploy smoke; runtime invariant monitors; SLO dashboard green for burn-in (72 hours for R1; 24 hours for R2).	100% pass; no invariant violations; SLO holding.	On-call + RCA; rollback if any check fails.

6.7 Test Maintenance

Tests are not written once and trusted forever. Tests rot: the code they test evolves, the data they depend on drifts, the dependencies they mock change. The maintenance regime below is the operational form of Volume I Chapter 19 (Testing Constitution) applied over time.

Flaky tests - tests that pass and fail without any code change - are a first-class quality problem. A flaky test erodes trust in the entire suite: if one test is flaky, engineers begin to suspect all tests, and the suite's value collapses. Opus Publica SHALL operate a three-strikes policy on flaky tests: (1) on the first observed flake, the test is marked flaky in CI and the owning team is notified; (2) on the second flake within 30 days, a ticket is filed and the test is quarantined (runs but does not gate); (3) on the third flake, the test is disabled and the ticket is escalated. A disabled test SHALL be fixed or deleted within 30 days; a disabled test older than 30 days SHALL be deleted by the QA Lead.

Test deletion is permitted and encouraged. A test that tests deleted code SHALL be deleted. A test that tests behaviour that has been superseded SHALL be deleted. A test that has been disabled for 30 days SHALL be deleted. A test that is so thoroughly mocked it tests nothing SHALL be rewritten or deleted. The instinct to retain tests because someone might need them is wrong; version control retains them, and the suite retains only what is trustworthy.

Procedure: Writing Tests for a Feature

Step 1. Identify the test layers applicable to the feature Every feature has unit tests. Features that cross module boundaries have integration tests. Features that change a published API have contract tests. Features with user-visible behaviour have end-to-end tests. R1 features have performance tests. Identify and record the applicable layers.

Step 2. Write the unit tests first (test-driven where feasible) Unit tests are written alongside the implementation, ideally before. The unit tests SHALL cover every branch of the changed code; coverage on changed lines SHALL be $\geq 85\%$.

Step 3. Write the integration tests for cross-module behaviour Integration tests exercise the real collaborators (DB, cache) but mock external services. Integration tests SHALL cover every new module boundary.

Step 4. Write the contract tests for any API change If the feature changes a published API, publish the new contract to the Pact broker and verify both consumer and provider sides.

Step 5. Write or extend the end-to-end test for user-visible behaviour End-to-end tests cover the critical user journeys. If the feature introduces a new journey, add a test. If it modifies an existing journey, update the test.

Step 6. Add performance tests for R1 features R1 features SHALL have a performance test that asserts the SLO. Run the test in staging; record the baseline.

Step 7. Run the full suite locally and in CI All tests SHALL pass locally before opening a PR. CI SHALL be green before requesting review. Flaky tests SHALL be reported per 6.7.

Step 8. Update the test ownership in CODEOWNERS Every test file SHALL have an owner in CODEOWNERS. Orphan tests are deleted.

Procedure: Handling a Flaky Test

Step 1. Confirm the flake is real Re-run the test in isolation. If it passes, re-run 10 times. If it fails at least once in 10 runs, it is flaky; file a ticket.

Step 2. Mark the test flaky in CI Tag the test with @flaky (or the framework equivalent). CI SHALL run it but SHALL not gate on it. Notify the owning team in the ticket.

Step 3. Investigate the root cause Flakes are usually: time-dependent logic, network races, shared state, non-deterministic ordering, or external-service dependency. Identify the cause; record it in the ticket.

Step 4. Fix the root cause Do NOT add a retry-and-hope. Fix the underlying non-determinism. If the cause is an external service, mock it in CI and run the real version in a dedicated nightly job.

Step 5. Quarantine if the flake recurs (strike 2) If the test flakes again within 30 days, quarantine it (runs but does not gate). Update the ticket with the second occurrence.

Step 6. Disable if the flake recurs again (strike 3) On the third flake, disable the test. Escalate the ticket. The owning team SHALL fix or delete within 30 days.

Step 7. Delete if disabled > 30 days The QA Lead SHALL delete any test disabled for more than 30 days. The test SHALL be re-written from scratch if the behaviour is still required.

Checklist: QA (GATE to Certification)

- ☐ **QA-01** Test layers identified and written for the feature (6.3).
- ☐ **QA-02** Unit test coverage on changed lines $\geq$ 85%.
- ☐ **QA-03** Integration tests cover new module boundaries.
- ☐ **QA-04** Contract tests cover any published API change (consumer + provider).

- ☐ **QA-05** End-to-end tests cover critical user journeys.
- ☐ **QA-06** Performance tests present for R1 features; no regression > 5%.
- ☐ **QA-07** Accessibility tests pass (WCAG 2.1 AA, 0 critical) for UI changes.
- ☐ **QA-08** SAST and SCA are green; 0 high/critical findings.
- ☐ **QA-09** DAST run in staging; 0 high/critical findings (R1/R2).
- ☐ **QA-10** No flaky tests introduced; existing flaky tests handled per 6.7.
- ☐ **QA-11** Test data is synthetic or scrubbed; no PII in tests (6.5).
- ☐ **QA-12** Test ownership recorded in CODEOWNERS; no orphan tests.

6.8 Roles and Ownership

Role	Authority	Boundaries
Implementing Engineer	Writes unit, integration, and (where applicable) contract tests.	Cannot certify own tests; cannot disable a flaky test without a ticket.
QA Engineer	Writes end-to-end, performance, and visual-regression tests; owns the test pyramid ratios.	Cannot sign off release; cannot waive a coverage target.
Security Engineer	Owens SAST, DAST, SCA configuration and triage.	Cannot suppress a finding without recorded justification; cannot waive a critical.
SRE	Owens load, chaos, and production-gate tests; owns invariant monitors.	Cannot run chaos in production without RCA sign-off.
QA Lead	Owens the test pyramid doctrine; owns the flaky-test regime; deletes orphan tests.	Cannot extend the 30-day deletion window; cannot override a release gate.
Release Certification Authority	Signs off the release gate; verifies evidence.	Cannot certify with an open high/critical finding; cannot bypass an invariant.

6.9 Evidence and Records

Test evidence is the artifact the RCA reviews at certification. Every test run SHALL produce evidence: a result, a coverage report, a performance baseline, a finding list. The evidence SHALL be stored in the certification repository for release-gate tests and in the CI platform for merge-gate tests, with quarterly export to cold storage. Volume II Chapter 20 (Runtime Verification Constitution) specifies the long-term retention; this section provides the operational summary.

Evidence	Source	Storage	Retention
Unit/integration test results + coverage	CI	CI platform; quarterly export.	90 days hot; 1 year cold.
Contract test results (Pact)	Pact broker	Pact broker; mirrored to object storage.	Lifetime of the contract.
End-to-end test recordings	CI / staging	Object storage; linked from release record.	Lifetime of the release + 1 year.
Performance baseline + regression report	k6 / Locust	Performance dashboard; archived per release.	Lifetime of the platform (for trend analysis).
SAST/DAST/SCA findings + triage	Security tools	Security findings platform; SEV tickets for criticals.	Lifetime of the platform + 5 years.
Flaky-test quarantine + deletion log	CI + ticket tracker	CI platform; ticket tracker.	1 year.
Release-gate evidence (full suite in prod-equiv)	RCA package	Certification repository.	Lifetime of the platform.

Chapter 7 — Test Environments and Data

7.1 Purpose

This chapter extends Chapter 6 with a deeper treatment of test environments and the data that flows through them. It operationalizes Volume I Chapter 5.05 (Quality Assurance) as it pertains to environments, and the 12-Factor methodology's Factor X (Dev/Prod Parity), which holds that development, staging, and production environments SHALL be as similar as possible. Dev/prod parity is not a luxury; it is the precondition for the Principle of Runtime Truth. A test that passes in an environment unlike production proves nothing about production; a certification granted in an environment unlike production is not a certification, it is a guess.

The discipline specified here binds the environment topology (7.2), the provisioning regime (7.3), access control (7.4), data seeding and refresh (7.5), drift detection (7.6), and the production-equivalence requirements that gate certification (7.7). An engineer who has read this chapter SHALL be able to provision a staging environment, seed it with scrubbed data, verify its parity with production, and present it to the RCA for certification with confidence that what passes there will pass in production.

Constitution Reference: Volume I, Chapter 5.05 (Quality Assurance); 12-Factor App Methodology, Factor X (Dev/Prod Parity) — ENV-* / FACTOR-X

7.2 Environment Topology

Opus Publica SHALL maintain a defined topology of environments, each with a purpose, a region, a size, a set of backing services, and a parity level. The topology is provisioned by Infrastructure-as-Code (7.3); ad-hoc environments are forbidden. The diagram below shows the relationship between environments and the flow of a release through them.

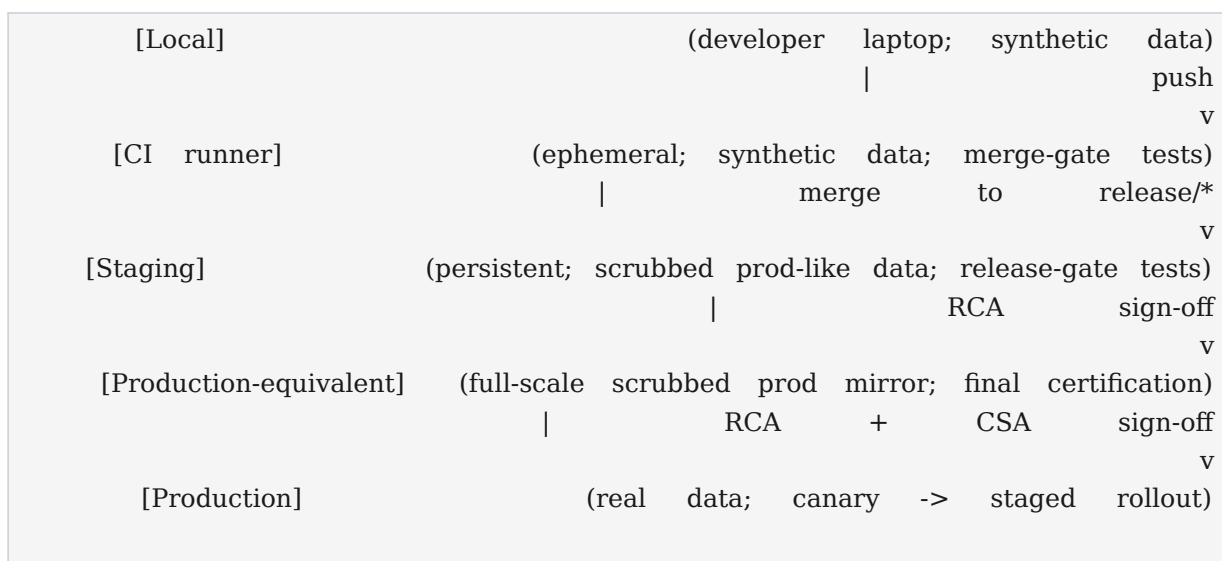


Figure 7.1: Environment Topology and Release Flow

Environment	Region	Size	Backing Services	Parity Level
Local	n/a (laptop)	1 instance of each service, ephemeral	SQLite/Postgres-local, Redis-local, mocks for external APIs	Low (synthetic data; mocked externals).
CI runner	CI region	Ephemeral; per-job	Testcontainers: Postgres, Redis, S3-mock, mocks for external APIs	Low-medium (real backing services, mocked externals).
Staging	Primary cloud region	1 instance of each service, persistent	Real Postgres, Redis, S3; mocks for billable external APIs (Crossref test prefix)	Medium (real backing services; test-prefix externals).
Production-equivalent	Primary cloud region (separate account)	Production-scale (replica of prod sizing)	Real Postgres, Redis, S3; real external APIs (test credentials); production-shape data	High (production-scale; production-shape data; real externals on test credentials).
Production	Primary cloud region (production account)	Production-scale, multi-AZ	Real Postgres, Redis, S3; real external APIs (production credentials)	Full (this IS production).

7.3 Environment Provisioning

Every environment SHALL be provisioned by Infrastructure-as-Code (Terraform or Pulumi). Manual changes to any environment are forbidden; a manual change is an uncontrolled change, and an uncontrolled change to staging is a parity violation that SHALL be detected by the drift detector (7.6) and reverted. The IaC source SHALL live in /infra in the repository; the state SHALL live in a remote backend (S3 + DynamoDB lock, or equivalent) and SHALL NOT be committed.

Provisioning time targets are mandatory. An environment that takes hours to provision is an environment that engineers will avoid rebuilding, and an environment that is not rebuilt accumulates drift. The targets below are enforced; an environment that exceeds its target SHALL be optimized by the SRE team.

Environment	Provisioning Target	Teardown Policy	Owner
Local	5 minutes (one command)	Per-session; torn down by	Implementing engineer.

	make env-up)	developer.	
CI runner	< 10 minutes (per job)	Per-job; torn down by CI.	SRE (CI platform owner).
Staging	< 30 minutes (full rebuild)	Persistent; reset on demand per 6.4; full rebuild quarterly.	SRE + QA.
Production-equivalent	< 60 minutes (full rebuild)	Persistent; reset per-release; full rebuild monthly.	SRE + RCA.
Production	n/a (never rebuilt; only mutated by deploy)	Never torn down; mutated by deploy and IaC apply.	SRE + CSA (sole authority).

Procedure: Provisioning a Staging Environment

Step 1. Verify the IaC source is at the release branch tip The IaC source SHALL be at the release branch tip. Run git checkout release/<rc-id> && git pull. The IaC version SHALL match the release version being certified.

Step 2. Verify the remote state is current Run terraform init (or pulumi login). The remote state SHALL be the source of truth; local state is forbidden.

Step 3. Run a plan and review the diff Run terraform plan -out=plan.tfplan. Review the diff. If the diff includes destructive changes (resource replacement), escalate to the SRE and the RCA.

Step 4. Apply the plan Run terraform apply plan.tfplan. The apply SHALL be run from CI, not from a laptop, so the actor and the diff are recorded.

Step 5. Run the post-provisioning smoke test Run the staging smoke test suite. All tests SHALL pass. If any fail, the environment SHALL be torn down and re-provisioned; do NOT patch forward.

Step 6. Seed the data per 7.5 Run the data-seeding pipeline. Verify the seed completed and the data scrubbing report is green (0 PII detected).

Step 7. Run the drift detector against production per 7.6 The drift detector SHALL report the staging configuration matches production (within the allowed deltas). Any drift SHALL be remediated before certification.

Step 8. Record the provisioning in the release record Record the IaC version, the apply timestamp, the actor (CI service account), and the smoke-test result in the release record.

7.4 Environment Access Control

Access to each environment SHALL be granted on a least-privilege basis. Production access is restricted to on-call and SRE; break-glass access is available but SHALL be logged, alerted, and reviewed. The break-glass procedure is the operational form of Volume I Chapter 13 (Operational Ownership); it is invoked

only when production access is required outside the normal rotation (an incident, an emergency deployment).

Environment	Who Can Access	Access Method	Break-Glass Procedure
Local	Engineer (own laptop)	Direct	n/a
CI runner	CI service account; engineers via CI logs	CI platform; logs are read-only	n/a
Staging	QA, engineers, release manager	SSO + role; SSH via bastion for SRE	Tech Lead approves; logged.
Production-equivalent	QA, RCA, SRE	SSO + role; SSH via bastion	Principal Engineer approves; logged; RCA notified.
Production	On-call + SRE only	SSO + role + MFA; SSH via bastion; all actions logged	Break-glass: CSA or designated deputy approves; on-call invokes; incident ticket filed within 1 hour; reviewed within 24 hours.

7.5 Data Seeding and Refresh

Staging and production-equivalent environments SHALL be seeded with scrubbed production-like data. The seeding pipeline SHALL: (1) export production data; (2) scrub PII per the Privacy Constitution (Volume II Chapter 30); (3) scale the data to the environment's size; (4) load it into the environment; (5) verify the scrub by running the PII-detection scanner on the seeded data; (6) produce a scrub report that the RCA reviews at certification. Scrubbed data is not optional; production data in staging without scrubbing is a SEV-1 privacy incident.

The refresh cadence is weekly for staging and per-release for production-equivalent. A staging environment whose data is older than 14 days is stale and SHALL be refreshed before any release-gate test runs. The data pipeline SHALL be idempotent: a re-run produces the same end-state, and a partial failure does not leave the environment in an inconsistent state.

Environment	Seed Source	Scrub Method	Refresh Cadence	Verification
Staging	Production export (anonymized)	PII redaction pipeline (Volume II Ch 30); deterministic fakes for names, emails, ORCIDs	Weekly (Sundays); on-demand before release-gate tests	PII scanner run post-seed; report attached to release record.

Production-equivalent	Production export (full-scale, anonymized)	Same pipeline as staging, applied at production scale	Per-release (before certification)	PII scanner + scale verification; RCA reviews.
CI runner	Synthetic generator	n/a (synthetic)	Per-job	Tests pass.
Local	Synthetic generator (make seed)	n/a (synthetic)	Per-session	Smoke test passes.

Procedure: Refreshing Test Data

Step 1. Verify the data-export job is current The production data export SHALL be no older than 24 hours. If older, trigger a fresh export and wait for completion.

Step 2. Run the PII scrubbing pipeline Run the scrub pipeline against the export. The pipeline SHALL redact names, emails, ORCIDs, IP addresses, and any other PII enumerated in Volume II Chapter 30. The redaction SHALL be deterministic so re-runs are reproducible.

Step 3. Run the PII scanner on the scrubbed output The scanner SHALL detect any residual PII. If the scanner reports any finding, STOP; file a SEV-1 privacy ticket; do NOT proceed.

Step 4. Scale the data to the target environment Staging receives a sampled subset (e.g., 10% of production volume, capped at a fixed size). Production-equivalent receives the full production-shape data.

Step 5. Load the data into the target environment Run the load pipeline. The load SHALL be idempotent; a re-run produces the same end-state. The load SHALL be transactional; a partial failure rolls back.

Step 6. Verify the load Run the environment's smoke test suite. Verify row counts are within expected bounds. Verify the PII scanner is green on the loaded data.

Step 7. Produce the scrub report and attach to the release record The scrub report SHALL include: export timestamp, scrub pipeline version, PII scanner version, scanner findings (0 expected), scale target, load verification. The RCA SHALL review this report at certification.

7.6 Environment Drift Detection

Drift between staging and production is the silent killer of certification. An environment that drifts - different DB version, different config flag, different library version - certifies a system that is not the system that will run in production. Drift SHALL be detected daily and remediated before any release-gate test runs. The drift detector compares the staging IaC state to the production IaC state and reports any delta outside an allowed allow-list (e.g., environment-specific secrets, environment-specific sizing).

The drift detector SHALL run as a daily CI job. On detecting drift, the job SHALL: (1) file a ticket; (2) notify the SRE and the QA Lead; (3) block any in-progress release-gate test run with a clear message. The drift

SHALL be remediated by re-applying the IaC to staging (or to production, if production drifted from the IaC source - which is itself a SEV-2 incident).

Drift Class	Detection Method	Severity	Remediation
DB version mismatch	Compare staging DB version to prod DB version	Blocking	Upgrade staging to match prod (or prod to match IaC).
Library version mismatch	Compare dependency lock files	Blocking	Reconcile to the IaC-pinned version.
Config flag mismatch	Compare feature-flag states	Blocking (except allowed deltas)	Reconcile; investigate the cause of the drift.
Sizing mismatch (CPU/RAM/disk)	Compare IaC resource specs	Allowed (staging may be smaller)	Record the delta; verify it is in the allow-list.
Secret rotation lag	Compare secret versions in the secret manager	Warning (staging may lag by ≤ 7 days)	Rotate staging secrets if lag > 7 days.
IaC source divergence	Compare staging IaC ref to prod IaC ref	Blocking	Reconcile staging to the release branch IaC tip.

7.7 Production-Equivalent Requirements

An environment SHALL be considered production-equivalent for certification purposes only if it satisfies ALL of the following criteria. The RCA SHALL verify each criterion before signing off the release gate; an environment that fails any criterion SHALL NOT be used for certification, and the certification SHALL NOT proceed until the environment is remediated.

- **Infra class.** Same infrastructure class (same cloud provider, same instance family, same storage class, same network topology).
- **DB version.** Same database version (major and minor) and same schema migration state as production (or as production will be after the release).
- **Config parity.** Same configuration as production, minus secrets (which are environment-specific) and minus environment-specific overrides recorded in the allow-list (7.6).
- **Data scale.** Same data volume scale as production (within an order of magnitude; production-equivalent is NOT a 1% sample - it is a full-scale mirror).
- **External integrations.** Same external API integrations, on test credentials (e.g., Crossref test deposit prefix). Real externals, not mocks.
- **Observability parity.** Same observability stack (same metrics, same logs, same traces, same alerts) so that certification observes what production will observe.
- **Drift-free.** Drift detector is green (7.6); no blocking drift between the env and production.

- **Security posture.** Same IAM and network policy shape as production (roles may differ, but the policy structure SHALL be the same).

Checklist: Environment Readiness (GATE)

- ☐ **ER-01** Environment provisioned from IaC at the release branch tip (7.3).
- ☐ **ER-02** Provisioning time within target (7.3); smoke test green.
- ☐ **ER-03** Data seeded from scrubbed production export; PII scanner green (7.5).
- ☐ **ER-04** Data refresh within cadence (staging $\leq$ 14 days; prod-equiv per-release).
- ☐ **ER-05** Drift detector green; no blocking drift vs production (7.6).
- ☐ **ER-06** Access control configured per 7.4; break-glass procedure documented.
- ☐ **ER-07** Production-equivalent criteria met (7.7) for release-gate certification.
- ☐ **ER-08** Same infra class, DB version, config parity, data scale, externals, observability.
- ☐ **ER-09** Scrub report produced and attached to release record.
- ☐ **ER-10** RCA has reviewed and approved the environment for certification.

7.8 Roles and Ownership

Role	Authority	Boundaries
Implementing Engineer	Uses local and CI environments; reports env issues.	Cannot modify staging or higher without SRE approval.
QA Engineer	Uses staging and production-equivalent; runs release-gate tests.	Cannot modify env config; cannot disable the drift detector.
SRE	Owens env provisioning, IaC, drift detection, access control.	Cannot provision production without CSA approval; cannot disable drift detection.
Security Engineer	Owens PII scrubbing pipeline, secret rotation, IAM policy shape.	Cannot waive a PII scanner finding; cannot suppress a drift alert.
Release Certification Authority	Verifies env readiness; signs off release gate.	Cannot certify on a non-production-equivalent env; cannot bypass 7.7.
Chief Systems Architect	Sole authority for break-glass production access approval.	Shall NOT approve break-glass for non-emergencies; shall review every invocation.

7.9 Evidence and Records

Environment evidence is the artifact that proves the certification was run in an environment that resembles production. Without this evidence, certification is a guess; with it, certification is a defensible decision. Every release-gate run SHALL produce the evidence below, stored in the certification repository and retained for the lifetime of the platform.

Evidence	Source	Storage	Retention
IaC apply record (version, timestamp, actor, diff)	CI / Terraform Cloud	CI platform; mirrored to certification repo.	Lifetime of the platform.
Smoke test result (post-provisioning)	CI	CI platform; linked from release record.	90 days hot; 1 year cold.
Data scrub report (export ts, scrub version, scanner findings)	Scrub pipeline	Certification repository.	Lifetime of the platform.
Drift detector report (staging vs prod)	Daily CI job	CI platform; tickets for any drift.	90 days hot; 1 year cold.
Production-equivalence verification (7.7)	RCA checklist	Certification repository.	Lifetime of the platform.
Break-glass invocation log (if any)	IAM platform + incident ticket	Security incident platform; reviewed within 24 hours.	Lifetime of the platform + 5 years.
Access control review (quarterly)	SRE	Engineering Leadership archive.	Lifetime of the platform + 5 years.

Chapter 8 — Runtime Invariant Monitoring

8.1 Purpose

This chapter operationalizes Volume II Chapter 26 (Engineering Invariants). The Constitution declares thirty-eight engineering invariants — identity invariants, metadata invariants, retrieval invariants, consistency invariants, lifecycle invariants, audit invariants, and preservation invariants — each of which holds at every moment the platform is in operation. A declared invariant that is not monitored is an invariant that will silently drift into violation, and a violation that is not detected is a violation that propagates until a user-visible failure exposes it. The purpose of this chapter is to convert every declared invariant into an operational monitor, a daily review ritual, a reconciliation job, and an incident response procedure. Invariant monitoring is the operational immune system of the platform; this chapter is the manual for that immune system.

Constitution Reference: Volume II, Chapter 26 (Engineering Invariants) — INV-I-01 ... INV-V-01 (the thirty-eight invariants)

The scope of this chapter is the day-to-day operation of the invariant regime: what the on-call engineer looks at every morning, how an alert is triaged and responded to, what reconciliation jobs run in the background and how their failures are handled, how a monitor itself is monitored for liveness, and how the invariant catalogue evolves as the platform evolves. The chapter does not restate the invariants themselves — those are normative in Volume II Chapter 26 — nor does it restate the certification use of invariants (Chapter 9 §9.4) or the failure-mode classification of invariant violations (Chapter 12 §12.5). It assumes those and operates them.

8.2 The Invariant Dashboard

The Invariant Dashboard is the single pane of glass through which the on-call engineer and the engineering leadership observe the invariant state of the platform. It is hosted in the observability platform (Grafana equivalent) and is the first URL every on-call engineer opens at the start of a shift. The dashboard's authority is operational, not advisory: a red panel on the dashboard SHALL be treated as an active incident until proven otherwise, and no release SHALL proceed while any SEV-1 invariant panel is red.

The dashboard is organized into seven panels, one per invariant category, plus an eighth meta-panel that monitors monitor liveness (§8.6). Each panel presents, for every invariant in the category: the invariant ID (e.g., INV-I-01), the human-readable statement, the current pass/fail status (green / amber / red), the most recent check timestamp, the monitor that produced the status, and a deep link to the monitor's runbook. Historical trend lines are preserved for ninety days so that slow drift is visible alongside sudden failure.

Panel	Invariants	What It Shows	Refresh
-------	------------	---------------	---------

Identity	INV-I-01 ... INV-I-04	Per-article canonical identity, DOI binding, ORCID binding, ROR binding.	Every 5 minutes.
Metadata	INV-M-01 ... INV-M-06	Published immutability, audit append-only, schema conformance, cross-reference integrity.	Every 5 minutes.
Retrieval	INV-R-01 ... INV-R-04	DOI resolution, artifact retrievability, OAI-PMH exposure, long-tail accessibility.	Every 15 minutes.
Consistency	INV-C-01 ... INV-C-08	Orphan storage, transition legality, referential integrity across subsystems.	Every 15 minutes.
Lifecycle	INV-L-01 ... INV-L-05	Published-has-DOI, deposited-has-receipt, preserved-has-copy, etc.	Every 30 minutes.
Audit	INV-A-01 ... INV-A-04	Every transition audited, audit chain verifiable, audit tamper-evident, audit retention honoured.	Every 30 minutes.
Preservation	INV-V-01 ... INV-V-03	Artifacts never lost, fixity verifiable, geographic redundancy intact.	Every hour.
Meta	MON-01 – MON-08	Liveness of every monitor above; blind-spot indicator.	Every 5 minutes.

The daily review ritual is the operational discipline that converts the dashboard from a passive display into an active control. Every on-call engineer SHALL conduct the Daily Invariant Dashboard Review at the start of the shift, every working day, without exception. The review is brief (target five minutes) but mandatory. A missed review is itself a recorded process violation, because it means the dashboard was not observed and any overnight drift went undetected. The procedure is specified in §8.3 below.

8.3 Invariant Monitor Operations

Every invariant in the catalogue has a corresponding monitor: a small, focused program that periodically evaluates the invariant's predicate against live platform state and emits a pass/fail signal into the

dashboard and the alert router. Each monitor has a named owner (the team accountable for the subsystem the invariant constrains), a defined cadence (the maximum interval between checks), and a defined alert routing (the channel(s) paged on failure). The table below enumerates the representative monitors; the full register is maintained in the Invariant Operations Catalogue (a living document under version control) and SHALL be reconciled against Volume II Chapter 26 monthly.

Invariant	Monitor	Cadence	Owner	What It Checks	Alert Routing
INV-I-01	mon-canonical-article	5 min	Identity Team	Every published article has exactly one canonical record in the article service; no duplicate canonical IDs exist.	PagerDuty: identity-oncall.
INV-I-02	mon-doi-binding	5 min	Identity Team	Every published article is bound to exactly one Crossref-registered DOI; no article has zero or >1 DOIs.	PagerDuty: identity-oncall; #eng-invariants.
INV-M-01	mon-published-immutable	5 min	Metadata Team	No published article's metadata has been mutated outside the correction/retraction workflow.	PagerDuty: metadata-oncall; CSA notified within 1 hour.
INV-M-02	mon-audit-append-only	5 min	Audit Team	The audit log is strictly append-only; no row in the audit table has been updated or deleted since creation.	PagerDuty: audit-oncall; CSA notified within 1 hour.
INV-R-01	mon-doi-resolution	15 min	Retrieval Team	Every DOI registered in the last 90 days resolves through	PagerDuty: retrieval-oncall; #eng-invariants.

				doi.org to the Opus Publica landing page within 30 seconds.	
INV-R-02	mon-artifact-retrievable	15 min	Retrieval Team	Every artifact referenced by a published article is retrievable via its content-addressed handle within 30 seconds.	PagerDuty: retrieval-oncall; #eng-invariants.
INV-C-01	mon-no-orphan-storage	15 min	Storage Team	No object exists in the artifact store that is not referenced by at least one article record.	PagerDuty: storage-oncall; #eng-invariants.
INV-C-07	mon-transition-legality	15 min	Lifecycle Team	Every state transition recorded in the last hour is legal per the lifecycle state machine (Vol I Ch 5.01).	PagerDuty: lifecycle-oncall; #eng-invariants.
INV-L-01	mon-published-has-doi	30 min	Lifecycle Team	Every article in the 'published' state has a non-null DOI binding; none are missing.	PagerDuty: lifecycle-oncall; #eng-invariants.
INV-A-01	mon-every-transition-audited	30 min	Audit Team	Every state transition recorded in the lifecycle service has a corresponding entry in the audit log within 60 seconds.	PagerDuty: audit-oncall; CSA notified within 1 hour.

INV-A-03	mon-audit-chain-verifiable	30 min	Audit Team	The hash chain over the audit log is verifiable end-to-end; no broken links detected.	PagerDuty: audit-oncall; CSA notified within 1 hour.
INV-V-01	mon-artifacts-never-lost	60 min	Preservation Team	Every artifact in the preservation inventory is present in both geographic replicas and passes SHA-256 fixity check.	PagerDuty: preservation-oncall; CSA notified within 1 hour.

8.4 Invariant Violation Response

An invariant violation is, by definition, a violation of a constitutional rule. The Constitution does not permit invariant violations to be deferred, batched, or triaged into a backlog. Every violation SHALL be responded to with a severity-appropriate procedure, and every response SHALL be recorded. The severity classification below is the operational overlay on top of the general incident severity classification in Chapter 12 §12.5; it is specialized to invariant violations because the response geometry differs (an invariant violation implies a constitutional breach, which has notification and remediation obligations beyond those of a generic incident).

Severity	Definition	Response Time	Notification	Authority
SEV-1	Invariant violation that compromises the integrity of the scholarly record (e.g., INV-M-01 published-immutability breach; INV-A-03 audit chain break; INV-V-01 artifact loss).	Page on-call immediately; ack within 5 minutes; mitigation within 30 minutes.	CSA notified within 1 hour; Engineering Leadership notified; incident commander assigned.	Incident Commander; CSA may halt any release.
SEV-2	Invariant violation that compromises correctness of a subsystem but not	Page on-call within 15 minutes; mitigation within 4 hours.	Engineering Leadership notified; owning team engaged; CSA	Owning team; RCA may block certification.

	the scholarly record (e.g., INV-C-01 orphan storage; INV-L-01 missing-DOI for one article).		notified in the daily invariant digest.	
SEV-3	Invariant monitor in a degraded state (e.g., partial failure where the invariant still holds but the monitor cannot prove it).	Ticket created within 1 business day; remediation within 5 business days.	Recorded in the Invariant Operations Catalogue; reviewed in the weekly engineering meeting.	Owning team.

Runbook: Invariant Violation (SEV-1)

Detect

The monitor fires and pages the on-call engineer via PagerDuty. The page SHALL include the invariant ID, the monitor output (the offending rows or evidence), the dashboard deep link, and the link to this runbook. The on-call engineer acks within 5 minutes; if no ack, the page escalates to the secondary on-call and then to Engineering Leadership.

Triage

The on-call engineer confirms the violation is real (not a monitor bug) by manually inspecting the offending rows. If the violation is a monitor false positive, the engineer records it as a SEV-3 monitor defect and closes the SEV-1. If the violation is real, the engineer assigns an incident commander (often themselves) and declares the incident in the incident channel (#eng-incidents).

Contain

The engineer takes immediate containment action to prevent propagation: for an immutability breach, freeze writes to the affected article(s); for an audit chain break, freeze the audit log; for an artifact loss, halt any in-progress deletion or replication jobs. The goal of containment is to bound the blast radius, not to remediate.

Remediate

The engineer identifies the root cause (per the postmortem process in Chapter 12 §12.8) and applies a corrective action that restores the invariant to a passing state. Remediation SHALL be the smallest change that restores the invariant; speculative changes are forbidden during a SEV-1.

Verify

The engineer re-runs the offending monitor manually and confirms it passes. The dashboard panel returns to green. The engineer confirms that no downstream monitor is now red as a side-effect of the remediation.

Postmortem

A blameless postmortem SHALL be authored within 5 business days (Chapter 12 §12.8). The postmortem SHALL specifically address: why the invariant was violable in the first place; why the monitor did not catch it sooner (if applicable); what structural change will prevent recurrence. The CSA reviews every SEV-1 invariant postmortem personally.

Runbook: Invariant Violation (SEV-2)**Detect**

The monitor fires and pages the on-call engineer. The page routing is to the owning team's on-call, not to the global on-call. The on-call engineer acks within 15 minutes.

Triage

The engineer confirms the violation is real and assesses scope: how many records affected, which subsystems impacted, whether any user-visible behaviour is wrong. The triage note is recorded in the incident ticket.

Contain

Containment is best-effort within the subsystem; the broader platform continues to operate. The engineer records the containment action and timestamp.

Remediate

The engineer fixes the underlying cause. If remediation requires more than 4 hours, the engineer SHALL escalate to SEV-1 with CSA notification, because a SEV-2 that exceeds its mitigation window is, by definition, a SEV-1.

Verify

The engineer re-runs the monitor and confirms pass. The dashboard panel returns to green.

Postmortem

A short-form postmortem (300-600 words) SHALL be authored within 5 business days. The RCA reviews all SEV-2 invariant postmortems in the weekly certification review.

8.5 Reconciliation Jobs

Monitors answer the question 'is the invariant true right now?'. Reconciliation jobs answer the complementary question 'has the invariant ever been silently false, and have external systems drifted out of sync with internal state?'. A reconciliation job is a heavier, less-frequent process that compares internal state to an external system-of-record (or to a redundant internal copy) and reports any divergence. Reconciliation is the defence against drift that monitors cannot detect because both sides of the comparison have moved together.

Reconciliation Job	What It Reconciles	Cadence	Owner	On-Failure Action
--------------------	--------------------	---------	-------	-------------------

Crossref reconciliation	Internal DOI registry against Crossref's records (deposited vs. registered; metadata match).	Daily	Identity Team	Diff is filed as a SEV-2; deposits are replayed for any DOI Crossref lacks; metadata corrections filed for any divergence.
Storage reconciliation	Artifact store inventory against the article-service references (both directions: orphan storage, dangling references).	Daily	Storage Team	Orphans are flagged for review (not auto-deleted); dangling references are SEV-2 tickets against the article service.
Audit chain verification	End-to-end hash-chain verification of the audit log from genesis to the current head.	Daily	Audit Team	Any broken link is SEV-1 (INV-A-03); CSA notified within 1 hour.
OAI-PMH self-harvest	The platform harvests its own OAI-PMH endpoint and compares against internal article state.	Weekly	Retrieval Team	Divergence indicates a publication-pipeline bug; SEV-2 ticket filed.
DOI resolution probe	Probes every DOI registered in the last 90 days through the public doi.org resolver.	Weekly	Retrieval Team	Unresolvable DOIs are SEV-2; metadata/landing-page mismatches are SEV-2.
ORCID link verification	Verifies every ORCID binding still resolves to the same ORCID record (no silent re-assignment).	Weekly	Identity Team	Stale bindings are flagged for re-authentication; SEV-3 unless the author is actively published, in which case SEV-2.
Preservation receipt reconciliation	Compares preservation receipts (LOCKSS/Clockss-	Weekly	Preservation Team	Missing receipts are SEV-1 (INV-V-01); CSA notified within 1 hour.

	equivalent) against the preservation inventory.			
--	---	--	--	--

8.6 The Meta-Monitor

A monitor that fails silently is worse than no monitor at all, because it produces false confidence. The Meta-Monitor (panel MON-01 in the dashboard) exists to detect monitor failure: it periodically executes a synthetic check against each monitor and verifies that the monitor responds within its declared cadence with a well-formed result. A monitor that has not produced a check within 2× its declared cadence is declared blind, and the meta-monitor raises a SEV-2 alert: the corresponding invariant is now unmonitored, and unmonitored is operationally equivalent to unknown.

When the meta-monitor raises a blind-spot alert, the response is identical to a SEV-2 invariant violation: the owning team is paged, mitigation begins within 4 hours, and the gap is recorded in the Invariant Operations Catalogue. If the blind spot persists beyond 24 hours, the alert escalates to SEV-1 with CSA notification, because a 24-hour unmonitored invariant is a constitutional breach regardless of whether the invariant itself has drifted. The meta-monitor's own liveness is verified by a tertiary heartbeat job that publishes a 'heartbeat-of-heartbeats' counter every five minutes; if the counter stalls, the on-call is paged.

8.7 Invariant Amendment

The invariant catalogue is not immutable; it evolves with the platform. A new feature may introduce a new invariant (e.g., a new external integration introduces a new binding invariant); a postmortem may reveal that an existing invariant is too weak (e.g., it checks hourly but drift can occur in minutes); a constitutional amendment (per Volume II Chapter 36) may introduce or remove an invariant. The amendment procedure ensures that every change to the invariant catalogue is deliberate, reviewed, and traceable, and that the operational regime (monitors, reconciliation jobs, runbooks) is updated in lockstep with the constitutional change.

Constitution Reference: Volume II, Chapter 36 (Evolution Governance)

No engineer MAY unilaterally add, remove, or weaken an invariant. All amendments SHALL be proposed in writing (an Invariant Amendment Proposal, or IAP), reviewed by the Engineering Leadership Team, and approved by the CSA. The IAP SHALL include: the textual change to Volume II Chapter 26; the operational impact (new monitors, modified cadences, new runbooks); the implementation plan; and the rollback plan if the amendment proves unsound. The IAP SHALL cite the Constitution clause that the amendment operationalizes or amends.

Procedure: Daily Invariant Dashboard Review

Step 1. Open the Invariant Dashboard Navigate to the canonical dashboard URL (bookmarked in the on-call runbook). Confirm the page loaded within 5 seconds; if not, file a SEV-3 observability ticket.

Step 2. Verify meta-panel green Confirm MON-01 (the meta-monitor panel) is entirely green. If any sub-panel is amber or red, follow the meta-monitor blind-spot procedure (§8.6) before continuing.

Step 3. Scan each invariant panel Walk the seven invariant panels (Identity, Metadata, Retrieval, Consistency, Lifecycle, Audit, Preservation). For each, confirm the panel header is green. Any amber or red panel SHALL be triaged per §8.4.

Step 4. Review overnight alerts Open the PagerDuty incident list for the last 24 hours. Confirm every invariant-related incident has an ack and a status (investigating / mitigated / resolved). Any un-acked incident is escalated immediately.

Step 5. Review reconciliation job results Open the reconciliation job dashboard. Confirm every job that ran overnight completed; investigate any failure per §8.5.

Step 6. Record the review Post a brief review note in #eng-invariants: 'Invariant review for <date>: all green' or list any open items with status. The note is the evidence that the review was conducted.

Procedure: Responding to an Invariant Alert

Step 1. Acknowledge the page Ack in PagerDuty within the severity-defined window (5 min SEV-1, 15 min SEV-2). If you cannot ack, the page escalates; do not let it escalate without intention.

Step 2. Confirm the violation is real Open the monitor's deep link. Manually inspect the offending evidence (rows, log lines, fixity report). If the alert is a false positive, file a SEV-3 monitor defect and close the incident; do not leave a false positive open as if it were real.

Step 3. Declare severity and assign commander Confirm or adjust the severity per §8.4. For SEV-1, assign an incident commander (Chapter 12 §12.7) and declare in #eng-incidents.

Step 4. Execute the runbook Follow the SEV-1 or SEV-2 runbook above. Contain first; remediate second; verify third.

Step 5. Notify per the severity matrix SEV-1: notify CSA within 1 hour and Engineering Leadership immediately. SEV-2: notify Engineering Leadership and the owning team; CSA is notified in the daily digest.

Step 6. Author the postmortem Within 5 business days, author the postmortem (Chapter 12 §12.8). For SEV-1, the CSA reviews personally; for SEV-2, the RCA reviews in the weekly certification review.

Checklist: Invariant Health (Daily)

- ☐ **IH-01** Invariant Dashboard URL loaded within 5 seconds.
- ☐ **IH-02** Meta-monitor panel (MON-01) entirely green.
- ☐ **IH-03** All seven invariant panels scanned; none red.
- ☐ **IH-04** Amber panels triaged; tickets filed or closed.

- ❑ **IH-05** PagerDuty overnight incidents reviewed; all acked.
- ❑ **IH-06** Reconciliation jobs from last 24h all completed.
- ❑ **IH-07** Any reconciliation failures triaged per §8.5.
- ❑ **IH-08** Review note posted in #eng-invariants.
- ❑ **IH-09** Open SEV-3 monitor defects reviewed for progression.
- ❑ **IH-10** Daily review evidence retained for 90 days.

8.8 Roles and Ownership

Role	Authority	Boundaries
On-Call Engineer	Conducts the daily review; acks and triages alerts; executes containment.	Cannot weaken an invariant; cannot close a SEV-1 without CSA acknowledgement.
Owning Team	Owens the monitors, reconciliation jobs, and runbooks for their invariant category.	Cannot amend the invariant itself (Volume II Ch 36); cannot disable a monitor without an approved IAP.
Incident Commander	Coordinates SEV-1 response; assigns tasks; communicates with stakeholders.	Cannot override the CSA; cannot declare an incident resolved until the monitor passes.
Release Certification Authority	Reviews invariant state as a certification input (Ch 9 §9.4).	Can block certification on any invariant failure; cannot override a runtime failure.
Chief Systems Architect	Notified of every SEV-1 within 1 hour; reviews every SEV-1 postmortem.	Can halt any release; cannot override a runtime certification failure (Constitution Prime Directive).

8.9 Evidence and Records

The invariant regime produces evidence continuously: monitor outputs, reconciliation job reports, alert pages, acks, incident tickets, and postmortems. Each artifact is the proof that the regime is operating and the proof an auditor requires to attest that the platform honours its constitutional invariants. Evidence SHALL be retained per the schedule below; no evidence SHALL be deleted before its retention elapses, even when the underlying data has rolled off the live dashboard.

Evidence	Storage	Retention	Consumer
Monitor outputs (pass/fail + payload)	Observability platform (hot); object store (cold).	Hot: 90 days; Cold: 5 years.	On-call; RCA at certification; external

			auditor.
Reconciliation job reports	Object store; indexed by job + date.	5 years.	Owning teams; RCA; external auditor.
PagerDuty incidents	PagerDuty (exported weekly to object store).	Lifetime of the platform.	On-call; CSA; postmortem authors.
Daily review notes	#eng-invariants archive (Slack export to object store).	90 days hot; 2 years cold.	On-call; Engineering Leadership.
Postmortems	Postmortem repository (version-controlled).	Lifetime of the platform.	All engineers; CSA; external auditor.
Invariant Amendment Proposals (IAPs)	Engineering Leadership repository (version-controlled).	Lifetime of the platform.	All engineers; CSA; external auditor.

Chapter 9 — The Certification Process

9.1 Purpose

This chapter operationalizes the certification regime established by Volume I Chapter 5.06 (Certification Process), Volume I Chapter 8 (Certification Standards), and Volume II Chapter 22 (Certification Constitution). Certification is the constitutional gate at which an implementation is converted from 'merged code' to 'releasable artifact'. It is the operational expression of Foundational Principle II (the Production Acceptance Gate): no implementation MAY proceed to production release unless it has been certified, and no certification MAY be issued without a complete, evidence-backed artifact set reviewed by the Release Certification Authority. The purpose of this chapter is to specify, in operational terms, what certification is, who performs it, what artifacts it requires, how it is signed off, when it must be re-done, where its evidence lives, and what anti-patterns SHALL be detected and refused.

Constitution Reference: *Volume I, Chapter 5.06 (Certification Process); Volume I, Chapter 8 (Certification Standards); Volume II, Chapter 22 (Certification Constitution)*

The certification regime rests on a single asymmetry that the engineer SHALL internalize: the Release Certification Authority (RCA) can stop a release but cannot override a runtime failure. The CSA cannot override the RCA on a runtime matter; the RCA cannot override the runtime; and the runtime cannot be argued with, only re-implemented. This asymmetry is the operational form of the Constitution's Prime Directive. Every procedure in this chapter honours it.

9.2 The Seven Certification Levels

Volume II Chapter 22 establishes seven certification levels, each representing a distinct layer of confidence in the implementation. The levels are cumulative in the sense that each level presupposes the lower levels; an implementation cannot be certified at the Publication level without first being certified at the Security level, and so on. The certification artifact set (§9.3) is the same across all levels, but the depth of evidence and the rigour of review increase with the level.

Level	Definition	Owner	Gate Effect
1. Development	The implementation compiles, passes unit tests, and meets the Coding Standards (Vol I Ch 7).	Implementing engineer	Allows the PR to be opened for review.
2. Engineering	The implementation passes code review by a peer (not the implementer); the Review Checklist (Ch 19) is	Reviewer	Allows the PR to be merged to the release branch.

	satisfied.		
3. Operational	The implementation passes the full test suite in a production-equivalent environment; observability is in place; runbooks are written.	QA Engineer	Allows the implementation to enter certification.
4. Security	The implementation passes the security review (Vol I Ch 12); threat model updated; no outstanding vulnerabilities at the SEV-1/SEV-2 level.	Security Engineer	Allows the implementation to proceed to publication-level review.
5. Publication	The five-pillar evidence (Chapter 10) is complete; the artifact set (§9.3) is complete; invariants hold.	QA Engineer + RCA	Allows the RCA to commence sign-off review.
6. Release	The RCA has signed off; the release artifact is built, signed, staged, and rollback-tested.	Release Certification Authority	Allows the Release Manager to schedule deployment.
7. Production	The implementation is deployed, post-deploy verification passes, the Production Acceptance Gate (Chapter 19) is satisfied.	RCA + Chief Systems Architect	Declares the implementation live; transfers ownership to on-call.

9.3 The Certification Artifact Set

Per Volume I Chapter 8 §8.3, ten mandatory artifacts SHALL be produced and assembled before the RCA may commence sign-off review. The artifact set is the evidence pack; absence of any single artifact is grounds for immediate rejection without further review. The RCA SHALL verify the presence and adequacy of each artifact; adequacy means the artifact actually supports the claim that the corresponding pillar or property holds, not merely that a file with the right name exists.

#	Artifact	What It Demonstrates	Produced By
1	Constitution clause reference	The implementation cites the Constitution clause(s) it satisfies, by stable ID.	Implementing engineer (Stage 1).

2	Runtime execution log	Logs and traces from the production-equivalent environment showing the feature executing.	QA Engineer.
3	Invariant monitor outputs	Evidence that no invariant is violated by the feature (Chapter 8).	QA Engineer (via Invariant Dashboard export).
4	Visual evidence	Screenshots or recordings of every user-visible state across the required browsers.	QA Engineer.
5	Database verification	Before/after queries against the replica showing persisted state change is correct.	QA Engineer.
6	Storage verification	Artifact listing, SHA-256 hashes, retrieval test for every artifact the feature produces.	QA Engineer.
7	API verification	Request/response recordings, contract test results, idempotency test results.	QA Engineer.
8	Idempotency test results	Evidence that retrying the operation produces the same end-state (Vol I Ch 7 §7.9).	QA Engineer.
9	Rollback test results	Evidence that the feature can be rolled back without data loss (Vol I Ch 5.08).	QA Engineer.
10	RCA sign-off	The RCA's signed certification that the artifact set is complete and the evidence supports the release claim.	Release Certification Authority.

9.4 The Sign-Off Process

The RCA's sign-off is the constitutional act that converts an implementation into a releasable artifact. The sign-off process is deliberate and verifiable; it is not a rubber stamp. The RCA SHALL execute the following steps in order, and SHALL NOT sign off unless every step completes successfully. The asymmetry of authority SHALL be respected throughout: the RCA can stop a release, but the RCA cannot override a runtime failure. If the runtime evidence shows the feature does not work, the feature does not work, and no amount of business pressure, schedule pressure, or executive interest can change that fact.

Constitution Reference: Volume II, Chapter 22 §22.4 (Sign-Off Authority and Asymmetry)

On sign-off, the RCA records the certification in the release record, attaching the complete artifact set. On rejection, the RCA SHALL return specific, actionable deficiencies to the implementing team; a rejection without specifics is itself a process violation, because it does not permit the team to remediate. The RCA's rejection SHALL cite the specific artifact(s) inadequate, the specific pillar(s) failed, and the specific Constitution clause(s) violated.

9.5 Re-Certification Triggers

Certification is not a permanent state; it is a state that holds for a specific version of the implementation against a specific version of the Constitution. Any change that could affect the validity of the prior certification SHALL trigger re-certification. The triggers below are exhaustive; an implementation that has not triggered re-certification in the maximum interval (one year) SHALL be re-certified on principle, because the world will have drifted even if the implementation has not.

Trigger	Action	Owner
Implementation modified (any merged PR to the release branch since last certification).	Re-certify the affected pillar(s) at minimum; full re-certification if the change is risk-class R1.	Implementing team requests; RCA reviews.
Dependency major-version upgrade (e.g., Django 4 -> 5).	Full re-certification, all five pillars.	Implementing team requests; RCA reviews.
Constitution clause amended (per Vol II Ch 36).	Re-certify every implementation that cited the amended clause.	Engineering Leadership identifies affected implementations; RCA reviews each.
Playbook checklist amended.	Re-certify every implementation currently in flight that uses the amended checklist.	Engineering Leadership identifies; RCA reviews.
Invariant added or modified.	Re-certify every implementation whose five-pillar evidence includes invariant monitor outputs.	Engineering Leadership identifies; RCA reviews.
Incident postmortem identifies a	Re-certify the feature(s) identified	Owning team requests; RCA

feature gap.	in the postmortem action items.	reviews.
Maximum interval (1 year) elapsed since last certification.	Full re-certification; the implementation is treated as if newly implemented.	RCA schedules; owning team executes.

9.6 The Certification Repository

The Certification Repository is the immutable store of every certified artifact set ever produced. It is version-controlled, append-only for certified artifacts (no certified artifact MAY be modified or deleted, even by the CSA), and accessible to three classes of user: the RCA (read and write during certification), Engineering Leadership (read), and external auditors (read, on request). The repository's immutability guarantee is the operational form of the Constitution's evidentiary requirement: an auditor SHALL be able to inspect, years later, the exact evidence that supported a specific release decision.

The repository is organized as: `certifications/<release-id>/<artifact-id>`. Every artifact is content-addressed (SHA-256); every certification is signed by the RCA's key. A re-certification creates a new release-id directory; it does not overwrite the prior one. The repository's audit log itself is hash-chained (per INV-A-03), so that any attempt to modify the repository is detectable.

9.7 Certification Anti-Patterns

The following anti-patterns SHALL be detected by the RCA during sign-off review and SHALL result in immediate rejection. Each anti-pattern is a specific failure mode of the certification regime; each has a specific detection signal the RCA SHALL apply.

Anti-Pattern	Why Forbidden	Detection
Self-certification	The implementing team certifies its own work. Violates separation of duties; the Constitution requires the RCA to be distinct from the implementing team.	Check the certifier's team against the implementing team; reject on any overlap.
Partial-certification-as-complete	Four of five pillars pass; the team marks certification complete and files a follow-up ticket for the fifth.	Check each pillar artifact; any missing or incomplete pillar is an automatic rejection regardless of the others.
Evidence-less sign-off	The RCA signs off without the artifact set; 'I trust the team'.	The certification repository SHALL contain the artifact set for every signed-off certification. A sign-off without artifacts in the repository is itself a process violation.

Stale certification treated as current	A release is shipped against a certification performed 18 months ago; no re-certification triggered.	Check the certification date against the maximum interval (§9.5); any certification older than 1 year is invalid until re-certified.
Pressure-driven sign-off	The RCA is pressured by schedule or business stakeholders to sign off despite a runtime failure.	The asymmetry is absolute: the RCA SHALL refuse and SHALL escalate to the CSA. Any attempt to pressure the RCA SHALL itself be recorded as a process violation.

Procedure: Requesting Certification

Step 1. Confirm pre-certification gates are passed The PR is merged to the release branch; the Operational level (Level 3) is certified; the Security level (Level 4) is certified. If any lower level is not certified, certification SHALL NOT be requested.

Step 2. Assemble the artifact set The implementing team and QA assemble the ten mandatory artifacts (§9.3). Each artifact SHALL be content-addressed and stored in the certification staging area.

Step 3. Verify artifact adequacy (self-check) Before requesting RCA review, the team performs a self-check: does each artifact actually support the claim it represents? Inadequate artifacts SHALL be re-produced before the request is made.

Step 4. Open the certification request The team opens a certification request ticket citing: the release-id, the Constitution clause(s) implemented, the artifact set location, and the requested certification level (5 or 6).

Step 5. Notify the RCA The RCA is notified via the certification request queue. The RCA acknowledges the request and schedules the review.

Procedure: Conducting Certification Review

Step 1. Open the artifact set The RCA opens the certification staging area and verifies all ten artifacts are present and content-addressed. Any missing artifact is an immediate rejection.

Step 2. Verify each pillar's evidence For each of the five pillars (Chapter 10), the RCA verifies the evidence actually supports the claim. The RCA SHALL NOT accept a screenshot at face value if it does not show what it claims to show; the RCA SHALL NOT accept a database dump if the dump does not include the relevant before/after state.

Step 3. Verify invariants hold The RCA consults the Invariant Dashboard (Chapter 8) and confirms no invariant is violated by the feature. Any violation blocks certification absolutely.

Step 4. Verify the Constitution clause is satisfied The RCA re-reads the cited Constitution clause and confirms the implementation actually satisfies it. This is not a formality; it is the substantive act of certification.

Step 5. Check for anti-patterns The RCA applies every anti-pattern detection (§9.7). Any anti-pattern detected is an immediate rejection with cited reason.

Step 6. Sign off or reject If all checks pass, the RCA signs off, records the certification in the release record, and moves the artifact set to the immutable Certification Repository. If any check fails, the RCA returns specific deficiencies and rejects.

Procedure: Handling Certification Rejection

Step 1. Acknowledge the rejection The implementing team acknowledges the rejection in the certification request ticket. The acknowledgment is not a debate; the RCA's rejection is binding.

Step 2. Read the cited deficiencies The team reads the specific deficiencies cited by the RCA. If any deficiency is unclear, the team requests clarification; the RCA SHALL provide it.

Step 3. Remediate The team produces a remediation plan with specific actions and owners. The remediation SHALL address every cited deficiency; partial remediation is not acceptable.

Step 4. Re-produce the affected artifacts The team re-produces the artifacts the RCA cited as inadequate. The re-produced artifacts are content-addressed as new versions; the prior inadequate artifacts are retained in the staging area for auditability.

Step 5. Re-request certification The team re-opens the certification request with the new artifact set. The RCA re-reviews, with attention to the previously-cited deficiencies.

Checklist: Certification (GATE to Release)

- ☐ **CT-01** Constitution clause(s) cited by stable ID (artifact #1).
- ☐ **CT-02** PR merged to release branch; feature flag off.
- ☐ **CT-03** Operational level (Level 3) certified.
- ☐ **CT-04** Security level (Level 4) certified; no SEV-1/SEV-2 vulns.
- ☐ **CT-05** Runtime execution log captured (artifact #2).
- ☐ **CT-06** Invariant monitor outputs captured (artifact #3).
- ☐ **CT-07** Visual evidence captured across required browsers (artifact #4).
- ☐ **CT-08** Database before/after verification captured (artifact #5).
- ☐ **CT-09** Storage verification: listing, SHA-256, retrieval (artifact #6).
- ☐ **CT-10** API verification: request/response, contract, idempotency (artifact #7).
- ☐ **CT-11** Idempotency test passes (artifact #8).
- ☐ **CT-12** Rollback test passes (artifact #9).
- ☐ **CT-13** No invariant violations during certification.
- ☐ **CT-14** Anti-pattern scan complete; no anti-patterns detected.
- ☐ **CT-15** RCA sign-off recorded in release record (artifact #10).

9.8 Roles and Ownership

Role	Authority	Boundaries
Implementing Engineer	Produces the implementation and the self-check evidence.	Cannot certify own work; cannot sign off release.
QA Engineer	Produces the five-pillar evidence and the artifact set (artifacts #2-#9).	Cannot sign off release; that is RCA's authority.
Security Engineer	Owens the Security certification level (Level 4).	Cannot override the RCA; cannot waive a runtime failure.
Release Certification Authority	Reviews the artifact set; signs off or rejects; owns the Certification Repository.	Cannot override a runtime failure; cannot self-certify; cannot be pressured.
Release Manager	Schedules deployment after RCA sign-off.	Cannot deploy without RCA sign-off; cannot skip the gate.
Chief Systems Architect	Co-signs Production-level certification (Level 7) with the RCA.	Cannot override the RCA on a runtime matter; can halt any release.

9.9 Evidence and Records

The certification regime's evidence is concentrated in the Certification Repository (§9.6), which is the immutable, signed, content-addressed store of every certified artifact set. The repository is the operational form of the Constitution's evidentiary requirement: an auditor SHALL be able to inspect, years later, the exact evidence that supported a specific release decision. Evidence outside the repository (e.g., in the task tracker, in the version control system) is referenced from the repository but not duplicated there.

Evidence	Storage	Retention	Consumer
Certification artifact set (10 artifacts)	Certification Repository (immutable, signed).	Lifetime of the platform.	RCA; external auditor; CSA.
Certification request ticket	Task tracker.	Lifetime + 5 years.	Implementing team; RCA.
Rejection records with cited deficiencies	Task tracker (linked from Certification Repository).	Lifetime + 5 years.	Implementing team; RCA; Engineering Leadership.
Re-certification trigger log	Engineering Leadership repository.	Lifetime of the platform.	RCA; external auditor.
RCA sign-off key signatures	Certification Repository (per certification).	Lifetime of the platform.	External auditor.

Chapter 10 — The Five Certification Pillars

10.1 Purpose

This chapter operationalizes Volume I Chapter 8 §8.2 (Certification Standards) and Volume II Chapter 20 (Runtime Verification Constitution). Certification is not a single act; it is a five-pillar regime in which each pillar verifies a distinct dimension of the implementation's correctness. The five pillars are: Runtime Verification, Visual Verification, Database Verification, Storage Verification, and API Verification. A feature passing four of five pillars is NOT 'mostly complete'; it is incomplete, and the missing pillar SHALL block certification absolutely. The pillars complement each other: runtime verification proves the feature executes; visual verification proves the user sees what they should; database verification proves the persisted state is correct; storage verification proves the artifacts are preserved; API verification proves the contracts hold. Together they constitute the constitutional standard of evidence for release.

Constitution Reference: Volume I, Chapter 8 §8.2 (The Five Pillars); Volume II, Chapter 20 (Runtime Verification Constitution)

This chapter specifies, for each pillar, what it verifies, how the evidence is produced, what format the evidence takes, where it is stored, and what retention applies. Each pillar has an associated procedure that the QA engineer SHALL follow to produce the evidence; the procedures are operational and concrete, not abstract. The chapter closes with the pillar interactions (§10.7), the common certification failures (§10.8), and the master five-pillar checklist.

10.2 Pillar 1: Runtime Verification

Runtime verification proves that the implementation, executing in a production-equivalent environment, actually performs the behaviour the Constitution clause requires. It is the operational form of Foundational Principle I: no implementation is correct because it compiles or passes static analysis — every user-visible feature must be verified through runtime execution and objective evidence. The pillar's evidence is the logs, traces, and invariant monitor outputs produced during the certification execution.

What it verifies: that the feature executes without errors; that the execution path matches the design; that the invariant monitors remain green throughout the execution; that the observability stack captures the feature's signals. How: the QA engineer runs the certification scenario in the production-equivalent environment, with tracing enabled, while the Invariant Dashboard is observed. Evidence format: a structured execution log (JSON lines), a distributed trace (OpenTelemetry export), and a snapshot of the Invariant Dashboard during execution. Storage: Certification Repository, content-addressed. Retention: lifetime of the platform.

Procedure: Producing Runtime Evidence

Step 1. Prepare the production-equivalent environment Ensure the environment is on the release branch, with the feature flag off, with the latest certified data snapshot loaded. Confirm tracing and logging are enabled at the certification level.

Step 2. Define the certification scenario The scenario SHALL exercise every code path introduced by the feature, including the error paths. The scenario is derived from the Constitution clause's acceptance criteria.

Step 3. Execute the scenario with tracing enabled Run the scenario; capture the OpenTelemetry trace and the structured log output. The trace SHALL span the entire user-visible operation.

Step 4. Observe the Invariant Dashboard during execution Open the Invariant Dashboard (Chapter 8). Confirm no panel goes red during the execution. Capture a snapshot of the dashboard at the scenario's peak.

Step 5. Capture the evidence Export the trace, the log, and the dashboard snapshot to the Certification Repository. Each artifact is content-addressed (SHA-256).

Step 6. Verify the evidence supports the claim The QA engineer reads the trace and confirms the feature executed the expected path. A trace that does not show the expected path is a pillar failure, even if no errors appear.

10.3 Pillar 2: Visual Verification (Screenshots)

Visual verification proves that the user-visible behaviour of the feature matches the design and the Constitution clause, across every user-visible state and every supported browser. It is the operational form of the principle that 'the user interface is the contract': a feature that works at the API layer but renders incorrectly is a feature that does not work. The pillar's evidence is screenshots and, where useful, screen recordings of every user-visible state.

What it verifies: every state of the feature (initial, loading, success, error, empty, permission-denied, etc.) renders correctly in every supported browser (Chrome, Firefox, Safari, Edge; mobile Safari, mobile Chrome). How: the QA engineer (or an automated visual regression tool) drives the feature through each state in each browser and captures a screenshot. Evidence format: PNG screenshots named <feature>-<state>-<browser>.png, with a manifest JSON. Storage: Certification Repository. Retention: lifetime of the platform.

Procedure: Producing Visual Evidence

Step 1. Enumerate the user-visible states List every state the feature can be in: initial, loading, success, error (network, validation, permission), empty, populated, long-content (truncation), and any feature-specific states.

Step 2. Enumerate the supported browsers Per Volume I Chapter 9 §9.4 (Browser Support Matrix): Chrome, Firefox, Safari, Edge on desktop; Safari and Chrome on mobile. Confirm the matrix is current; any browser added since the last certification SHALL be included.

Step 3. Capture a screenshot of every state in every browser For each (state, browser) pair, capture a full-page screenshot. The screenshot SHALL include the URL bar (to prove the environment) and the timestamp (to prove the recency).

Step 4. Capture recordings for time-dependent behaviour Where the feature has time-dependent behaviour (animations, progressive disclosure, async loading), capture a screen recording in addition to the static screenshot.

Step 5. Build the manifest Author a manifest JSON listing every screenshot/recording with its state, browser, environment URL, timestamp, and SHA-256 hash.

Step 6. Store in the Certification Repository Upload the screenshots, recordings, and manifest. The manifest is the index the RCA uses during review.

10.4 Pillar 3: Database Verification

Database verification proves that the persisted state of the platform after the feature executes is exactly the state the Constitution clause requires. It is the operational form of the principle that 'the database is the truth': a feature whose API returns success but whose database writes are wrong or incomplete is a feature that will silently corrupt the scholarly record. The pillar's evidence is before/after queries against a read replica.

What it verifies: every table the feature writes to has the expected rows after execution; no table the feature should not touch has been modified; the relationships between rows are correct (foreign keys, join tables); the audit log captures every transition. How: the QA engineer captures a 'before' snapshot of the affected tables, executes the scenario, captures an 'after' snapshot, and produces a diff. Evidence format: SQL queries + their result sets (CSV or JSON), plus a diff document. Storage: Certification Repository. Retention: lifetime of the platform.

Procedure: Producing Database Evidence

Step 1. Identify the affected tables From the implementation, enumerate every table the feature writes to, reads from in a way that could affect caching, or transitions (lifecycle, audit).

Step 2. Capture the 'before' snapshot Run the verification query against the read replica before executing the scenario. The query SHALL return the relevant rows for the test fixture.

Step 3. Execute the scenario Run the certification scenario (the same scenario as Pillar 1, executed against the production-equivalent environment).

Step 4. Capture the 'after' snapshot Re-run the verification query. The result set SHALL match the expected after-state defined in the acceptance criteria.

Step 5. Produce the diff Diff the before and after result sets. The diff SHALL show only the expected changes; any unexpected change is a pillar failure.

Step 6. Verify the audit log Query the audit log for every transition the feature caused. Every transition SHALL be present; every transition SHALL have the correct actor, timestamp, and payload.

10.5 Pillar 4: Storage Verification

Storage verification proves that every artifact the feature produces (files, derivatives, preserved copies) is present, content-addressed, and retrievable. It is the operational form of the principle that 'an artifact that cannot be retrieved does not exist': a feature that writes a file but cannot retrieve it later has not completed its work. The pillar's evidence is an artifact listing, SHA-256 hashes, and a retrieval test for each artifact.

What it verifies: every artifact the feature produces exists in the artifact store; every artifact has a SHA-256 hash recorded; every artifact is retrievable via its content-addressed handle; the preservation inventory (if applicable) records the artifact. How: the QA engineer lists the artifacts the feature should have produced, verifies each is present, computes the hash, and performs a retrieval test. Evidence format: artifact listing (CSV), SHA-256 manifest (JSON), retrieval log (with timings). Storage: Certification Repository. Retention: lifetime of the platform.

Procedure: Producing Storage Evidence

Step 1. Enumerate the expected artifacts From the implementation, list every artifact the feature should have produced: primary file, derivatives (thumbnails, full-text indices), preservation copies.

Step 2. List the actual artifacts in the store Query the artifact store for objects created during the scenario. The actual list SHALL match the expected list; any missing artifact is a pillar failure.

Step 3. Compute the SHA-256 hash of each artifact For every artifact, compute the SHA-256 hash. The hash SHALL match the hash recorded by the feature at write time; any mismatch indicates corruption.

Step 4. Retrieve each artifact via its content-addressed handle For every artifact, perform a retrieval test. The retrieval SHALL complete within the SLO (typically 30 seconds); the retrieved content SHALL hash-match the stored content.

Step 5. Verify the preservation inventory If the artifact is preserved (per Vol II Ch 30), confirm the preservation inventory records the artifact, its replica locations, and its fixity schedule.

Step 6. Build the storage evidence pack Assemble the listing, the SHA-256 manifest, and the retrieval log into the storage evidence pack. Store in the Certification Repository.

10.6 Pillar 5: API Verification

API verification proves that every API endpoint the feature exposes (or consumes) honours its contract, returns the documented responses, and is idempotent where required. It is the operational form of the principle that 'the API is the integration contract': a feature whose UI works but whose API is wrong will break every consumer of the platform. The pillar's evidence is request/response recordings, contract test results, and idempotency test results.

What it verifies: every endpoint the feature exposes returns the documented response for each documented input; contract tests pass against the live API; idempotent endpoints are actually idempotent (retrying the same request produces the same end-state). How: the QA engineer drives the

API through every documented scenario using a contract-testing tool, records the request/response pairs, and runs explicit idempotency tests (send the same request twice with the same idempotency key; verify the end-state is unchanged). Evidence format: HAR or JSON request/response recordings, contract test report, idempotency test report. Storage: Certification Repository. Retention: lifetime of the platform.

Procedure: Producing API Evidence

Step 1. Enumerate the affected endpoints From the implementation, list every API endpoint the feature exposes, modifies, or consumes. Each endpoint SHALL have a documented contract (OpenAPI or equivalent).

Step 2. Capture request/response recordings Drive each endpoint through every documented scenario (success, validation error, permission error, not-found, etc.). Record the request and response for each. The recording SHALL include headers, body, and timing.

Step 3. Run the contract tests Execute the contract test suite against the live API. Every test SHALL pass; any failure is a pillar failure.

Step 4. Run the idempotency tests For every idempotent endpoint, send the same request twice with the same idempotency key. Verify the second response is identical to the first; verify the end-state (database, storage) is unchanged after the second request.

Step 5. Run the negative tests Drive each endpoint with malformed, oversized, and adversarial inputs. The API SHALL handle each gracefully (return the documented error; never crash; never leak internal state).

Step 6. Build the API evidence pack Assemble the recordings, the contract test report, and the idempotency test report. Store in the Certification Repository.

10.7 Pillar Interactions

The five pillars are not redundant; they are complementary. Each pillar verifies a dimension the others do not. Runtime verification proves execution but not visible behaviour. Visual verification proves visible behaviour but not persisted state. Database verification proves persisted state but not artifact integrity. Storage verification proves artifact integrity but not contract conformance. API verification proves contract conformance but not user-visible behaviour. Together they cover the full evidence space; individually each leaves a gap the others fill.

The constitutional standard is that all five pillars SHALL pass. A feature passing four of five is NOT 'mostly complete'; it is incomplete. The missing pillar represents a class of failure the other pillars cannot detect, and shipping without that coverage is shipping a known blind spot. The RCA SHALL NOT trade off one pillar against another; the RCA SHALL NOT accept 'we will add the missing pillar post-release' as a certification argument, because post-release evidence is not certification evidence.

Where a pillar is genuinely not applicable (e.g., a feature with no user-visible state has no Visual pillar to verify), the RCA MAY mark the pillar N/A, but the N/A SHALL be justified in writing and the justification

SHALL cite the specific feature property that makes the pillar inapplicable. An unjustified N/A is equivalent to a missing pillar and blocks certification.

10.8 Common Certification Failures

The table below enumerates the common failures the RCA observes in practice, the pillar each failure corresponds to, the detection signal, and the remediation. The list is not exhaustive; it is a starting point for the RCA's review and a teaching tool for the implementing team.

Failure	Pillar	Detection	Remediation
Trace shows the happy path only; error paths untested.	Runtime	Trace contains no error-path spans.	Extend the certification scenario to cover error paths; re-run.
Screenshots captured only in Chrome; other browsers skipped.	Visual	Manifest lists Chrome only; other browsers absent.	Re-capture in every supported browser; re-upload.
Database before-snapshot taken after the scenario started.	Database	Before-snapshot timestamp is after the scenario start timestamp.	Re-execute the procedure; the before-snapshot MUST precede the scenario.
SHA-256 hash of stored artifact mismatches recorded hash.	Storage	Hash comparison fails.	Treat as data corruption (SEV-1); investigate root cause; re-store the artifact; re-verify.
API contract test passes but idempotency test fails.	API	Idempotency test report shows end-state changed on retry.	Fix the idempotency bug; re-run the contract and idempotency tests.
Invariant monitor goes red during certification execution.	Runtime (cross-cutting)	Invariant Dashboard snapshot shows a red panel.	Treat as an invariant violation (Chapter 8); block certification; remediate the invariant breach first.
Audit log missing a transition the feature caused.	Database (cross-cutting)	Audit log query returns fewer transitions than expected.	Treat as INV-A-01 violation; block certification; remediate the audit emission.
Evidence files exist but are not content-addressed.	All pillars	Artifacts in staging area lack SHA-256 hashes.	Hash every artifact; re-upload with hashes; the Certification Repository rejects un-hashed

artifacts.

Checklist: Five-Pillar Evidence Complete

- ☐ **FP-01** Runtime: trace spans the entire user-visible operation.
- ☐ **FP-02** Runtime: no errors in the structured log during the scenario.
- ☐ **FP-03** Runtime: Invariant Dashboard remained green throughout.
- ☐ **FP-04** Visual: every user-visible state captured in every supported browser.
- ☐ **FP-05** Visual: screenshots include URL bar and timestamp.
- ☐ **FP-06** Visual: recordings captured for time-dependent behaviour.
- ☐ **FP-07** Database: 'before' snapshot precedes scenario start.
- ☐ **FP-08** Database: 'after' snapshot matches expected state exactly.
- ☐ **FP-09** Database: audit log captures every transition.
- ☐ **FP-10** Storage: every expected artifact is present in the store.
- ☐ **FP-11** Storage: every artifact SHA-256 matches recorded hash.
- ☐ **FP-12** Storage: every artifact retrievable within SLO.
- ☐ **FP-13** API: request/response recordings cover every documented scenario.
- ☐ **FP-14** API: contract tests pass against the live API.
- ☐ **FP-15** API: idempotency tests pass; retries leave end-state unchanged.
- ☐ **FP-16** All evidence content-addressed and stored in Certification Repository.
- ☐ **FP-17** Any pillar marked N/A is justified in writing with feature property citation.

10.9 Roles and Ownership

Role	Authority	Boundaries
QA Engineer	Produces the five-pillar evidence; runs the scenarios; uploads artifacts.	Cannot sign off release; cannot waive a pillar.
Implementing Engineer	Provides the implementation; supports QA during evidence production.	Cannot produce own certification evidence (separation of duties).
Release Certification Authority	Reviews the five-pillar evidence; signs off or rejects.	Cannot override a pillar failure; cannot accept 'will add later'.
Engineering Leadership	Owens the Browser Support Matrix; owns the contract definitions.	Cannot amend a contract during certification; amendments trigger

re-certification.

10.10 Evidence and Records

The five-pillar evidence is the substantive content of the Certification Repository. Each artifact is content-addressed, signed by the producer, and retained for the lifetime of the platform. The evidence is the auditor's primary material; the auditor SHALL be able to reconstruct, from the evidence alone, what was certified and on what basis.

Pillar	Evidence Format	Storage	Retention
Runtime	OpenTelemetry trace + structured log + dashboard snapshot.	Certification Repository.	Lifetime of the platform.
Visual	PNG screenshots + screen recordings + manifest JSON.	Certification Repository.	Lifetime of the platform.
Database	Before/after SQL result sets (CSV/JSON) + diff + audit log query.	Certification Repository.	Lifetime of the platform.
Storage	Artifact listing (CSV) + SHA-256 manifest (JSON) + retrieval log.	Certification Repository.	Lifetime of the platform.
API	HAR/JSON request-response recordings + contract test report + idempotency test report.	Certification Repository.	Lifetime of the platform.

Chapter 11 — Release and Deployment

11.1 Purpose

This chapter operationalizes Volume I Chapter 5.07 (Release and Deployment). Release is the stage at which a certified implementation is packaged, signed, and staged for deployment; deployment is the stage at which the release is promoted to production through a staged rollout. Both stages SHALL be executed under the authority of the Release Manager, with the on-call engineer present, and within the discipline of the staged rollout strategy. The purpose of this chapter is to specify, in operational terms, the release cadence, the deployment pipeline, the staged rollout strategy, the production readiness review, the feature flag regime, the deployment communication protocol, and the emergency release procedure.

Constitution Reference: Volume I, Chapter 5.07 (Release and Deployment); Volume II, Chapter 22 (Certification Constitution) — Release and Production levels

Release and deployment are the operational hand-off from certification to production. The constitutional constraints do not relax at this stage; if anything, they tighten, because a release in production affects users and the scholarly record directly. The Release Manager SHALL NOT deploy without RCA sign-off; the on-call SHALL NOT enable a feature flag without post-deploy verification; and no emergency MAY bypass the staged rollout strategy (though it MAY compress it, per §11.8).

11.2 Release Cadence

Opus Publica RC1 operates on a weekly release train. The train departs every Wednesday at 14:00 UTC; the cut-off for inclusion is the preceding Tuesday at 23:59 UTC. Implementations certified after the cut-off wait for the next train. The weekly cadence balances two pressures: feature velocity (the platform needs to ship) and operational stability (frequent deploys are themselves a source of risk). The cadence is calibrated to RC1's scale; it SHALL be reviewed at RC2.

Critical fixes that cannot wait for the weekly train follow the hotfix path (§11.8). The hotfix path is expedited but not skipped: every gate that applies to a standard release applies to a hotfix, with compressed timing. The CSA's approval is required to skip certification for a hotfix, and certification SHALL NOT be skipped for any hotfix that touches identity, metadata, audit, or preservation subsystems.

Release naming follows: <semver>-RC<rc-number>.<YYYYMMDD>. For example, 1.0.0-RC1.20260809. The semver portion reflects the semantic change (major for breaking changes; minor for new features; patch for fixes). The RC tag reflects the release candidate number within the RC1 cycle. The date reflects the build date. The release name is the primary key of the release record and the Certification Repository's top-level directory for the release.

11.3 The Deployment Pipeline

The deployment pipeline is the automated sequence of stages a release passes through from merge to full production. Each stage has defined entry criteria, exit criteria, and gates. The pipeline SHALL NOT be bypassed; the pipeline SHALL NOT be reordered. The pipeline is the operational form of the Constitution's release discipline.

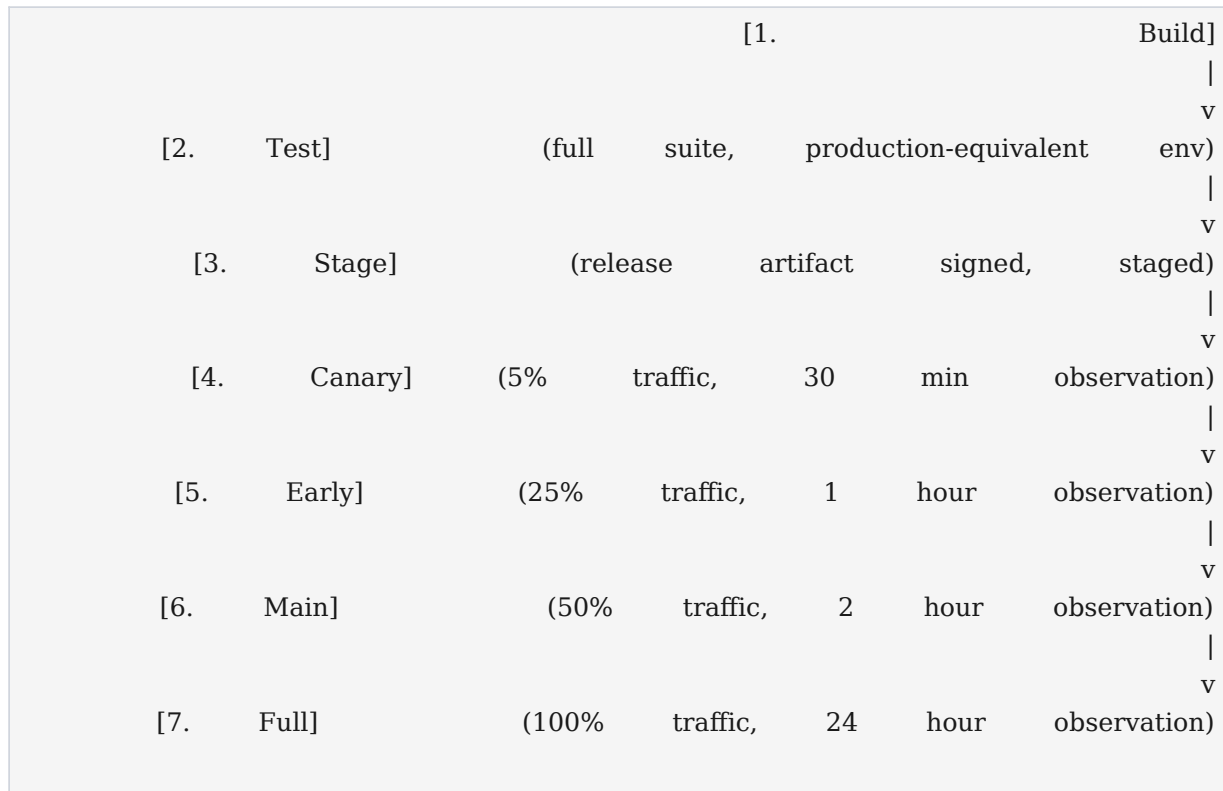


Figure 11.1: The Seven-Stage Deployment Pipeline

Stage	Entry Criteria	Exit Criteria	Gate Owner
1. Build	PR merged to release branch.	Artifact built, content-addressed, signed.	CI pipeline (automated).
2. Test	Build artifact exists; production-equivalent env ready.	Full test suite passes; five-pillar evidence captured.	QA Engineer.
3. Stage	Test stage green; RCA sign-off recorded.	Release artifact staged; hash matches certified artifact.	Release Manager.
4. Canary	Stage green; on-call paged; deploy window open.	5% rollout stable for 30 min; SLO green; no SEV-1/SEV-2 alerts.	On-Call Engineer.
5. Early	Canary green.	25% rollout stable for 1 hour; SLO green.	On-Call Engineer.

6. Main	Early green.	50% rollout stable for 2 hours; SLO green.	On-Call Engineer.
7. Full	Main green.	100% rollout stable for 24 hours; burn-in complete.	On-Call Engineer + Owning Team.

11.4 Staged Rollout Strategy

The staged rollout strategy specifies the percentage of traffic exposed at each stage, the observation period, the success criteria, and the rollback trigger. The strategy is calibrated to detect failures early (at low traffic) and to give confidence before scaling (at higher traffic). The on-call engineer SHALL NOT advance to the next stage until the current stage's success criteria are met; the on-call engineer SHALL roll back immediately upon any rollback trigger.

Stage	Traffic	Observation	Success Criteria	Rollback Trigger
Canary	5%	30 minutes	No SEV-1/SEV-2 alerts; error rate < 0.1%; SLO green; invariant monitors green.	Any SEV-1 alert; error rate > 1%; any invariant violation; any rollback test failure.
Early	25%	1 hour	Same as Canary; plus: p95 latency within SLO; no anomalous traffic patterns.	Same as Canary; plus: p95 latency > 2× baseline.
Main	50%	2 hours	Same as Early; plus: user-facing metrics stable; no support spike.	Same as Early; plus: support ticket rate > 2× baseline.
Full	100%	24 hours	Burn-in complete; no SEV-1/SEV-2 in 24 hours; SLO budget intact.	Any SEV-1; any SEV-2 not mitigated within 4 hours.

11.5 Production Readiness Review

Before a feature may enter a release, the Production Readiness Review SHALL be conducted. The review is a checklist the implementing team completes and the Release Manager verifies. The review ensures the feature is not just certified (which proves correctness) but also operable (which proves runnability). A

feature that is correct but not operable is a feature that will produce an incident on the first Saturday morning it is paged.

The Production Readiness Review covers: on-call assignment (a team is carrying the pager for the feature), runbook (written and linked from the on-call runbook index), SLOs (defined and dashboards live), dashboards (the feature's metrics are on a dashboard the on-call can see), alerts (configured and routed to the correct on-call), rollback tested (the rollback path was exercised in the production-equivalent environment during certification). Each item SHALL be satisfied; an unsatisfied item blocks the feature from entering the release.

11.6 Feature Flags

Feature flags decouple deployment from release. A feature MAY be deployed (in the code path) without being released (visible to users). The flag lifecycle is: created → enabled-in-staging → canary → enabled → deprecated → removed. Each transition SHALL be recorded. A flag SHALL have a named owner (the team accountable for the feature). A flag SHALL have a maximum age (90 days from enabled to deprecated); flags older than 90 days SHALL be reviewed for removal. Flag naming follows: <team>-<feature>-<purpose>, e.g., identity-orcid-autolink-rc1.

Flag discipline is the operational defence against flag rot: the slow accumulation of flags that no one remembers the purpose of, that no one dares remove, and that obscure the production code path. The weekly engineering meeting SHALL review the flag inventory; flags nearing maximum age SHALL be scheduled for removal or formally extended (with a recorded justification).

11.7 Deployment Communication

Deployment communication is the operational discipline of keeping stakeholders informed before, during, and after a deployment. The communication protocol is mandatory; a silent deployment is a deployment that, when it goes wrong, surprises everyone. The protocol has four phases: pre-deploy announcement, during-deploy status, post-deploy summary, and incident communication (if rollback or incident).

Phase	Audience	Channel	Content
Pre-deploy	Engineering team; stakeholders.	#eng-releases; release ticket.	Release name; contents; deploy window; on-call; rollback plan.
During-deploy	Engineering team.	#eng-releases.	Per-stage status updates: 'Canary started', 'Canary green', 'Early started', etc.
Post-deploy	Engineering team; stakeholders.	#eng-releases; release ticket.	Release name; deploy completed; post-deploy verification result; any

			anomalies.
Incident	Engineering team; CSA; stakeholders; users (if user-visible).	#eng-incidents; status page (if user-visible).	What happened; what is being done; expected resolution; post-incident summary to follow.

11.8 Emergency Releases

Emergency releases (hotfixes) follow an expedited path. Expedited means the timings are compressed, not that the gates are skipped. Every gate that applies to a standard release applies to a hotfix: certification (including five-pillar evidence), staged rollout (compressed to canary + full), rollback testing, post-deploy verification. The CSA's approval is required to skip certification for a hotfix, and certification SHALL NOT be skipped for any hotfix that touches identity, metadata, audit, or preservation subsystems — those subsystems' invariants are too consequential to release unverified.

A hotfix SHALL be requested via the hotfix request ticket, citing the incident that justifies the hotfix and the specific gate(s) requested to be expedited. The CSA SHALL approve or deny within 1 hour. On approval, the Release Manager SHALL execute the hotfix pipeline, compressed but complete. The post-deploy review for a hotfix SHALL include a retrospective on whether the hotfix was justified and whether the expedited gates were adequate.

Procedure: Executing a Standard Deployment

Step 1. Confirm pre-deploy gates RCA sign-off recorded; release artifact staged; on-call present and briefed; deploy window open; communication pre-announcement sent.

Step 2. Execute the Build-Test-Stage sequence The CI pipeline builds the artifact, runs the test suite, and stages the release. Verify the staged artifact's hash matches the certified artifact's hash; any mismatch is an immediate halt.

Step 3. Begin the Canary stage Roll out to 5% of traffic. Observe for 30 minutes. The on-call engineer watches the SLO dashboard, the alert feed, and the Invariant Dashboard. If any rollback trigger fires, execute the rollback procedure (Chapter 12 §12.3) immediately.

Step 4. Advance through Early and Main On canary green, advance to Early (25%, 1 hour). On Early green, advance to Main (50%, 2 hours). Each advance SHALL be recorded in the release ticket.

Step 5. Advance to Full On Main green, advance to Full (100%). Observe for 24 hours (the burn-in). The on-call engineer remains responsible; the owning team takes over at the end of the burn-in.

Step 6. Conduct post-deploy verification Run the post-deploy verification suite: runtime checks, invariant monitors, SLO dashboard. All SHALL be green.

Step 7. Send the post-deploy summary Post the summary to #eng-releases and the release ticket. Close the release ticket on successful burn-in.

Procedure: Executing an Emergency Hotfix Deployment

Step 1. Open the hotfix request The on-call engineer (or incident commander) opens a hotfix request ticket citing the incident, the specific fix, and the gates requested to be expedited.

Step 2. Obtain CSA approval The CSA reviews and approves or denies within 1 hour. If the hotfix touches identity, metadata, audit, or preservation, certification SHALL NOT be skipped — only the timing is compressed.

Step 3. Execute expedited certification Where certification is required but compressed: the five-pillar evidence SHALL still be produced; the timeline is compressed to the minimum consistent with rigour (typically 2-4 hours). The RCA SHALL review the artifact set before sign-off.

Step 4. Execute the Build-Test-Stage sequence Same as standard; CI pipeline; hash verification.

Step 5. Execute compressed staged rollout Canary (5%, 15 minutes) → Full (100%, 24-hour observation). Early and Main stages MAY be skipped on CSA's explicit approval; the canary SHALL NOT be skipped.

Step 6. Communicate throughout Use the incident communication protocol (§11.7). The hotfix is an extension of the incident response.

Step 7. Conduct the post-deploy retrospective Within 5 business days, the team conducts a retrospective: was the hotfix justified? Were the expedited gates adequate? Were any shortcuts that should be reversed in the next standard release?

Checklist: Release Readiness (GATE)

- ☐ **RR-01** RCA sign-off recorded (Level 6 certification).
- ☐ **RR-02** Release artifact built, content-addressed, signed.
- ☐ **RR-03** Staged artifact hash matches certified artifact hash.
- ☐ **RR-04** Production Readiness Review complete; every item satisfied.
- ☐ **RR-05** On-call assigned; runbook written and linked.
- ☐ **RR-06** SLOs defined; dashboards live.
- ☐ **RR-07** Alerts configured; routed to correct on-call.
- ☐ **RR-08** Rollback tested in production-equivalent env (CT-09).
- ☐ **RR-09** Feature flag (if any) in correct lifecycle state.
- ☐ **RR-10** Deploy window open; on-call briefed.
- ☐ **RR-11** Pre-deploy announcement sent to #eng-releases.
- ☐ **RR-12** Staged rollout plan documented (percentages, observation windows).
- ☐ **RR-13** Communication protocol agreed (phases, channels, content).
- ☐ **RR-14** Rollback procedure documented and accessible to on-call.

11.9 Roles and Ownership

Role	Authority	Boundaries
Release Manager	Owens the release pipeline; schedules deploys; verifies hashes.	Cannot deploy without RCA sign-off; cannot skip gates.
Release Certification Authority	Sign-off is the gate to deployment.	Cannot override a runtime failure; cannot be overridden by the Release Manager.
On-Call Engineer	Executes staged rollout; watches the dashboards; rolls back on trigger.	Cannot skip a stage; cannot advance on a red signal.
Owning Team	Takes ownership at the end of the burn-in (24h).	Cannot abdicate the pager; cannot ignore alerts.
Chief Systems Architect	Approves hotfixes; approves gate expeditions.	Cannot override the RCA on runtime matters; cannot waive certification for identity/metadata/audit/preservation.

11.10 Evidence and Records

Release and deployment evidence is the operational record of what was deployed, when, by whom, with what result. The evidence SHALL be sufficient to reconstruct, years later, the exact state of a specific deployment and the exact decisions made during it. The release record is the primary artifact; the deployment logs are the secondary.

Evidence	Storage	Retention	Consumer
Release record (name, contents, hash, signature, certification ref)	Release Repository.	Lifetime of the platform.	Release Manager; RCA; external auditor.
Deployment logs (per-stage timings, traffic percentages, observations)	Observability platform (hot); object store (cold).	Hot: 90 days; Cold: 5 years.	On-call; Engineering Leadership.
Post-deploy verification results	Release Repository (linked from release record).	Lifetime of the platform.	RCA; external auditor.
Communication transcripts (#eng-releases, release ticket)	Chat archive (exported to object store); task tracker.	Hot: 90 days; Cold: 2 years.	On-call; Engineering Leadership.

Hotfix request and CSA approval	Task tracker; Engineering Leadership repository.	Lifetime of the platform.	RCA; CSA; external auditor.
Feature flag inventory and lifecycle transitions	Feature flag service (exported weekly to object store).	Lifetime of the platform.	Engineering Leadership; owning teams.

Chapter 12 — Rollback and Incident Response

12.1 Purpose

This chapter operationalizes Volume I Chapter 5.08 (Rollback and Incident Response) and Volume II Chapter 21 (Failure Constitution). Despite the certification regime of Chapters 9-11, incidents will occur: software has bugs, dependencies fail, networks partition, operators make mistakes. The platform's measure is not whether incidents occur but whether the response to them is disciplined, blameless, and learning. This chapter specifies, in operational terms, when to roll back, how to roll back, how incidents are classified, how the incident lifecycle flows, how incident command operates, how postmortems are conducted, and what anti-patterns SHALL be refused. Rollback is the operational form of the Constitution's failure discipline; the postmortem is the operational form of the Constitution's learning discipline.

Constitution Reference: Volume I, Chapter 5.08 (Rollback and Incident Response); Volume II, Chapter 21 (Failure Constitution)

The Constitution's Failure Constitution (Vol II Ch 21) establishes the asymmetry that governs this chapter: a rollback is always cheaper than an incident, and an incident that is not learned from is an incident that recurs. Every procedure in this chapter honours the asymmetry. The chapter does not restate the rollback testing requirement (which is in §12.4 and cross-referenced from Chapter 9 CT-09) nor the general incident severity classification (which is in §12.5); it assumes those and operates them.

12.2 When to Rollback

Rollback is the operational act of reversing a deployment to a prior known-good state. The decision to roll back SHALL be made on the following criteria, any one of which is sufficient. The on-call engineer SHALL NOT wait for multiple criteria to be met; the first criterion met SHALL trigger the rollback decision. Hesitation is the enemy of mitigation; a rollback is reversible, but a slow incident is not.

Criterion	Threshold	Action
SEV-1 incident declared	Any SEV-1 incident (per §12.5) attributable to the release.	Roll back immediately; communicate; investigate post-rollback.
SLO burn rate critical	Error budget burn rate exceeds 10× the canonical rate (Vol I Ch 5.08 §5.08.4).	Roll back; restore SLO; investigate.
Invariant violation	Any SEV-1 invariant violation (Chapter 8 §8.4) attributable to the release.	Roll back immediately; the invariant SHALL hold.
Feature unavailability > 5 minutes	A feature deployed in the release is unavailable (errors or timeouts) for	Roll back; investigate the

	more than 5 consecutive minutes.	unavailability.
On-call judgement	The on-call engineer judges, based on observed anomalies, that the release is unsafe.	Roll back. The on-call's judgement SHALL be respected; a post-rollback review determines whether the judgement was correct.

12.3 Rollback Procedure

The rollback procedure is the operational sequence the on-call engineer follows to reverse a deployment. The procedure SHALL be rehearsed during certification (CT-09; see §12.4); an unrehearsed rollback during an incident is a known failure mode. The procedure is six steps: detect, decide, execute, verify, communicate, postmortem.

Procedure: Executing a Rollback

Step 1. Detect An alert, an SLO dip, an invariant violation, or an on-call observation triggers the rollback consideration. The trigger event SHALL be recorded with timestamp and observed evidence.

Step 2. Decide The on-call engineer (or incident commander, if declared) applies the criteria in §12.2. If any criterion is met, the decision is to roll back. The decision SHALL be recorded with the criterion cited. Hesitation beyond the criterion being met is forbidden.

Step 3. Execute the rollback The on-call engineer triggers the rollback pipeline: revert the feature flag (if applicable); redeploy the prior release artifact (content-addressed; the prior release's hash is the target). The rollback SHALL be executed via the same staged mechanism as the forward deployment, compressed to canary + full.

Step 4. Verify After rollback, verify the platform is restored: SLOs green, invariant monitors green, error rate returned to baseline. The verification SHALL be explicit; the on-call SHALL NOT assume restoration.

Step 5. Communicate Notify #eng-incidents, #eng-releases, and (if user-visible) the status page. The communication SHALL state: what was rolled back, why, the current state, the next step (investigation).

Step 6. Postmortem Within 5 business days, author a postmortem (per §12.8) that addresses: what triggered the rollback; whether the rollback decision was correct; whether the rollback executed cleanly; what structural change will prevent recurrence.

12.4 Rollback Testing

Every release SHALL have a tested rollback path. The test SHALL be performed during certification (CT-09; Chapter 9 §9.3 artifact #9) in the production-equivalent environment. The test SHALL exercise the actual rollback procedure (the same pipeline the on-call would use), not a simulated or partial rollback. A release whose rollback path has not been tested SHALL NOT be certified; a release whose rollback path was tested but failed SHALL NOT be certified until the rollback path is fixed and re-tested.

Rollback testing is the operational defence against the failure mode in which a rollback is attempted during an incident and fails — leaving the platform in a worse state than before. The Constitution (Vol II Ch 21) treats rollback reliability as a first-class property of every release, not a nice-to-have. The Release Manager SHALL verify, before deployment, that the rollback path was tested within the certification window and that the test report is in the artifact set.

12.5 Incident Severity Classification

Incident severity is the operational classification that drives response time, escalation, and communication. Severity is declared by the on-call engineer (or incident commander); severity MAY be upgraded as more is learned, and MAY be downgraded only by the incident commander with recorded justification. The classification below is the platform-wide standard; subsystems (e.g., invariant violations in Chapter 8 §8.4) MAY have specialized overlays but SHALL NOT contradict this classification.

Severity	Definition	Response Time	Escalation	Communication
SEV-1	Critical: platform-wide outage; data integrity at risk; scholar record at risk (e.g., INV-M-01 breach; INV-A-03 breach; INV-V-01 breach).	Ack 5 min; mitigation 30 min.	Incident commander assigned; CSA notified 1 hour; Engineering Leadership notified immediately.	Status page; stakeholder update every 30 min during incident.
SEV-2	High: major feature unavailable or degraded; subsystem integrity at risk (e.g., INV-C-01 orphan; INV-L-01 missing DOI).	Ack 15 min; mitigation 4 hours.	Owning team engaged; Engineering Leadership notified; CSA in daily digest.	#eng-incidents; stakeholder update at mitigation.
SEV-3	Moderate: minor feature degraded; workaround exists; no data integrity risk.	Ack 1 business day; mitigation 5 business days.	Owning team engaged.	#eng-incidents ticket.
SEV-4	Low: cosmetic; non-user-facing; no operational impact.	Ack 1 week; mitigation per sprint.	Ticket only.	Ticket only.
SEV-5	Informational: observed anomaly with no current	Ack 1 month; mitigation if trend worsens.	Ticket only.	Ticket only.

	impact; tracked for trend.			
--	----------------------------	--	--	--

12.6 Incident Lifecycle

Every incident traverses the same lifecycle: detect → triage → mitigate → resolve → postmortem. The lifecycle's stages are sequential; each stage's exit is the next stage's entry. Time targets per severity are explicit; an incident that exceeds its target SHALL be escalated.

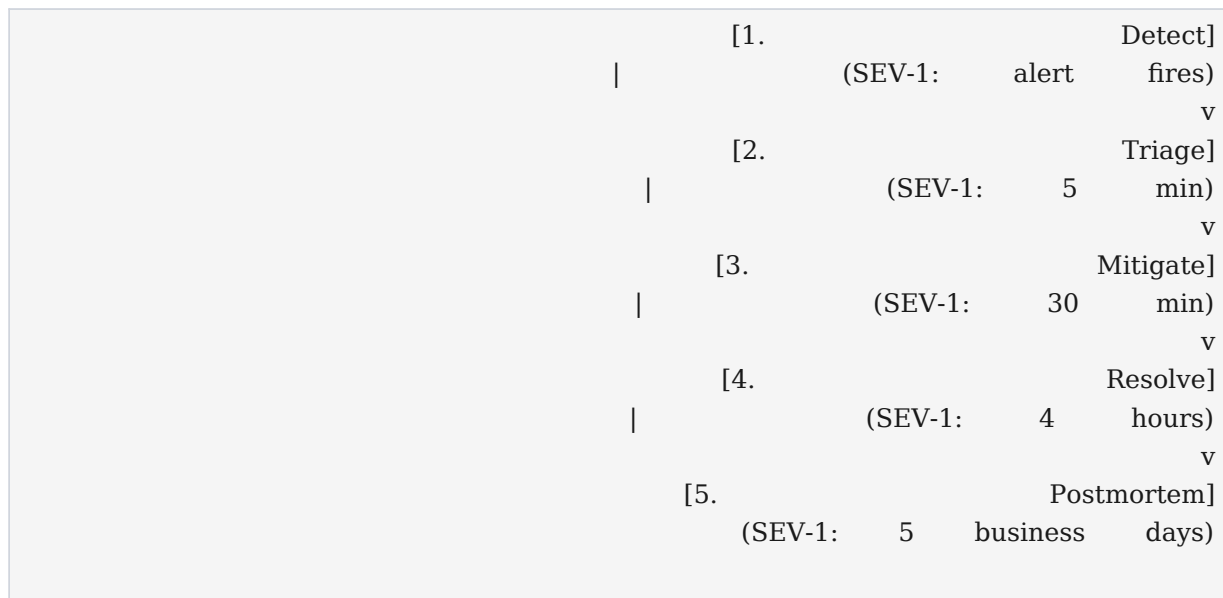


Figure 12.1: The Incident Lifecycle

Stage	SEV-1 Target	SEV-2 Target	SEV-3 Target	Owner
Detect	Immediate (alert)	15 min	1 business day	Monitoring / on-call.
Triage	5 min	30 min	1 business day	On-call / commander.
Mitigate	30 min	4 hours	5 business days	On-call / owning team.
Resolve	4 hours	1 business day	1 sprint	Owning team.
Postmortem	5 business days	5 business days	Optional (commander discretion)	Owning team.

12.7 Incident Command

For SEV-1 incidents (and SEV-2 incidents at the commander's discretion), the incident command regime is invoked. An Incident Commander (IC) is assigned; the IC is the single decision-maker during the incident. The IC's authority is operational: they MAY direct any engineer to take any action within their authority to mitigate the incident; they MAY declare a rollback; they MAY invoke the hotfix path. The IC's authority is bounded: they SHALL NOT override the Constitution; they SHALL NOT weaken an invariant; they SHALL NOT bypass certification for a hotfix without CSA approval.

The war room is the operational coordination point: a dedicated chat channel (#eng-incidents-<incident-id>) and, where geography permits, a physical or video room. The IC establishes the communication cadence: for SEV-1, every 30 minutes; for SEV-2, hourly; the cadence is the heartbeat of the response. Stakeholder updates are sent at each cadence tick, even if the update is 'no change; investigation continues'. Silence during an incident is forbidden; silence breeds rumour and panic.

12.8 Postmortem Process

Every SEV-1 and SEV-2 incident SHALL have a blameless postmortem authored within 5 business days of resolution. The postmortem is the operational form of the Constitution's learning discipline: an incident that is not learned from is an incident that recurs. Blameless means the postmortem focuses on the system, not the individuals; it asks 'what about the system made this failure possible' rather than 'who made the mistake'. Individuals are not named as causes; processes, tools, and assumptions are.

The postmortem template has the following sections: Summary (what happened, in two paragraphs); Timeline (minute-by-minute from detect to resolve); Impact (users affected, data affected, SLO budget consumed); Root Cause Analysis (5 whys, terminating in a structural cause); Action Items (specific, owned, dated); Lessons Learned (what surprised us, what worked well, what did not). The action items SHALL have named owners and due dates; action items without owners are wishes, not actions.

Follow-through tracking is mandatory: the action items are tracked in the task tracker, reviewed in the weekly engineering meeting, and marked complete only when the structural change is verified. The postmortem review cadence is monthly: the Engineering Leadership Team reviews all postmortems from the prior month, looking for patterns (multiple incidents with similar root causes) and verifying action-item completion. A postmortem whose action items are not complete by their due dates SHALL be escalated to the CSA.

Runbook: SEV-1 Incident Response

Detect

An alert fires (or an invariant monitor goes red, or an on-call observation triggers). The on-call acks within 5 minutes. If no ack, the page escalates per the PagerDuty chain.

Triage

The on-call confirms the incident is real (not a false positive). Declares SEV-1 in #eng-incidents. Assigns an Incident Commander (often themselves). Opens the incident channel (#eng-incidents-<id>). Notifies Engineering Leadership.

Mitigate

The IC directs mitigation: rollback (per §12.3), feature flag off, traffic shift, or hotfix (per Chapter 11 §11.8). The goal of mitigation is to restore service, not to fix root cause. Root cause investigation happens after mitigation.

Communicate

The IC sends the first stakeholder update within 30 minutes of detection. Subsequent updates at the 30-minute cadence. Status page updated for user-visible incidents. CSA notified within 1 hour of detection.

Resolve

Mitigation confirmed effective. Service restored. SLOs green. Invariant monitors green. The IC declares the incident resolved and closes the incident channel; the incident ticket remains open for the postmortem.

Postmortem

Within 5 business days, the postmortem is authored (per §12.8). The CSA reviews personally. Action items are filed with owners and due dates.

Runbook: SEV-2 Incident Response**Detect**

An alert fires or an invariant monitor goes red. The on-call acks within 15 minutes.

Triage

The on-call confirms the incident is real. Declares SEV-2 in #eng-incidents. Engages the owning team. An Incident Commander is assigned at the on-call's discretion (typically for SEV-2 incidents that may escalate to SEV-1).

Mitigate

Mitigation within 4 hours. If mitigation is not achievable within 4 hours, the incident escalates to SEV-1 (an incident that exceeds its mitigation window is, by definition, a higher severity).

Communicate

Update #eng-incidents at mitigation. Stakeholder update at mitigation (no 30-minute cadence required for SEV-2). CSA notified in the daily digest.

Resolve

Mitigation confirmed. Service restored. The on-call declares resolved; the ticket remains open for the postmortem.

Postmortem

Within 5 business days, a short-form postmortem (300-600 words). The RCA reviews in the weekly certification review.

12.9 Postmortem Anti-Patterns

The following postmortem anti-patterns SHALL be detected during the monthly postmortem review and SHALL be remediated. Each anti-pattern is a specific failure mode of the learning discipline; each compromises the platform's ability to learn from incidents.

Anti-Pattern	Why Forbidden	Detection / Remediation
Blame	The postmortem names an individual as the cause; 'engineer X pushed the bad commit'.	Rewrite focusing on the system: what about the review process, the test coverage, or the deploy pipeline made the bad commit reach production?
Vague action items	Action items like 'improve test coverage' or 'be more careful'.	Rewrite as specific, owned, dated: 'Add integration test for X by <date>, owned by <engineer>'.
No follow-through	Action items are filed but never completed; the postmortem becomes a historical document.	Track action items in the task tracker; review weekly; escalate overdue items to the CSA.
Postmortem never published	The postmortem is authored but sits in a local file; the team does not see it.	Publish to the postmortem repository (version-controlled, searchable) within 5 business days; announce in #eng-incidents.
Postmortem rationalizes the incident	The postmortem explains why the incident was unavoidable, rather than why it was possible.	Rewrite: every incident is structurally possible until the structural cause is removed. The postmortem's job is to identify the structural cause, not to defend the team.

Procedure: Conducting a Postmortem

Step 1. Schedule the postmortem within 24 hours of resolution The postmortem author (typically the IC or the owning team's tech lead) schedules the postmortem meeting within 24 hours of incident resolution, to occur within 5 business days.

Step 2. Gather the timeline Assemble the minute-by-minute timeline from chat logs, PagerDuty, and observability. The timeline SHALL start at detection and end at resolution; every significant action and observation SHALL be on it.

Step 3. Conduct the 5 whys Starting from the impact, ask 'why' until you reach a structural cause (a process, a tool, an assumption — not a person). Five iterations is typical but not mandatory; the goal is the structural cause, not the count.

Step 4. Identify action items Each structural cause SHALL have at least one action item. Action items SHALL be specific, owned, and dated. 'Add monitoring for X' is acceptable if X is specific and the owner is named.

Step 5. Draft the postmortem Using the template (§12.8): Summary, Timeline, Impact, Root Cause, Action Items, Lessons Learned. Blameless throughout.

Step 6. Review with stakeholders Review the draft with the incident responders, the owning team, and (for SEV-1) the CSA. Incorporate feedback. The review is collaborative, not adversarial.

Step 7. Publish Publish to the postmortem repository. Announce in #eng-incidents. File action items in the task tracker with owners and due dates.

Step 8. Track follow-through The author (or the owning team's tech lead) tracks action items to completion. Action items are reviewed weekly in the engineering meeting until complete.

Checklist: Rollback Readiness

- ☐ **RB-01** Rollback path tested in certification (CT-09).
- ☐ **RB-02** Rollback test report in artifact set.
- ☐ **RB-03** Prior release artifact hash recorded and accessible.
- ☐ **RB-04** Rollback pipeline operational; on-call can trigger.
- ☐ **RB-05** Feature flag can be disabled independently of deploy.
- ☐ **RB-06** Database rollback path documented (forward migrations reversible or compensating migrations identified).
- ☐ **RB-07** On-call briefed on rollback procedure.
- ☐ **RB-08** Rollback decision criteria (§12.2) in on-call runbook.
- ☐ **RB-09** Communication template for rollback prepared.
- ☐ **RB-10** Rollback verification checklist prepared (SLOs, invariants, error rate).

Checklist: Incident Response

- ☐ **IR-01** Alert acked within severity target (5/15 min).
- ☐ **IR-02** Incident declared in #eng-incidents with severity.
- ☐ **IR-03** Incident channel opened (#eng-incidents-<id>).
- ☐ **IR-04** Incident Commander assigned (SEV-1 mandatory; SEV-2 discretionary).
- ☐ **IR-05** Engineering Leadership notified (SEV-1 immediate; SEV-2 in daily digest).
- ☐ **IR-06** CSA notified within 1 hour (SEV-1).

- ❑ **IR-07** Mitigation executed within severity target.
- ❑ **IR-08** Stakeholder updates sent at cadence (SEV-1: 30 min; SEV-2: at mitigation).
- ❑ **IR-09** Status page updated (if user-visible).
- ❑ **IR-10** Resolution verified: SLOs green, invariants green.
- ❑ **IR-11** Incident declared resolved; ticket kept open for postmortem.
- ❑ **IR-12** Postmortem authored within 5 business days; published; action items filed.

12.10 Roles and Ownership

Role	Authority	Boundaries
On-Call Engineer	Acks alerts; triages; mitigates; declares incidents.	Cannot override the Constitution; cannot weaken an invariant; cannot bypass certification without CSA approval.
Incident Commander	Single decision-maker during a SEV-1 incident; directs mitigation.	Cannot override the CSA; cannot bypass certification without CSA approval.
Owning Team	Owens the postmortem; owns the action items; carries the pager post burn-in.	Cannot abdicate the postmortem; cannot close action items without verification.
Release Certification Authority	Reviews SEV-2 postmortems in the weekly certification review; may block future certifications on outstanding action items.	Cannot override the CSA; cannot unilaterally close a postmortem.
Chief Systems Architect	Reviews SEV-1 postmortems personally; approves hotfix gate expeditions.	Cannot override a runtime certification failure; cannot waive invariant requirements.

12.11 Evidence and Records

Incident evidence is the operational record of what happened, what was done, and what was learned. The evidence SHALL be sufficient to reconstruct, years later, the timeline, the decisions, and the follow-through of any incident. The postmortem is the primary artifact; the chat logs, PagerDuty records, and observability snapshots are the supporting material.

Evidence	Storage	Retention	Consumer
Postmortem document	Postmortem repository (version-controlled).	Lifetime of the platform.	All engineers; CSA; external auditor.

Incident timeline (chat logs, PagerDuty, observability)	Chat archive; PagerDuty export; observability platform.	Hot: 90 days; Cold: 5 years.	Postmortem author; RCA; CSA.
Action item tracker	Task tracker (linked from postmortem).	Lifetime + 5 years.	Owning team; Engineering Leadership; CSA.
Stakeholder communication transcripts	Chat archive; status page export.	Hot: 90 days; Cold: 2 years.	Engineering Leadership; CSA.
Rollback test report	Certification Repository (per release).	Lifetime of the platform.	RCA; Release Manager; external auditor.
Hotfix approval (CSA) and gate expedition record	Engineering Leadership repository.	Lifetime of the platform.	CSA; RCA; external auditor.

Chapter 13 — On-Call and Operational Ownership

13.1 Purpose

This chapter operationalizes Volume I Chapter 5.09 (On-Call and Operational Ownership) and the Constitution's Principle 10 (You Build It, You Run It). Operational ownership is the doctrine that the team which designs, implements, and certifies a feature is the team that carries the pager for it in production. The chapter converts the principle into a rotation structure, an eligibility rule, an authority matrix, a handoff protocol, a recovery policy, and a no-on-call-no-release gate that the Release Manager SHALL enforce before any service is promoted to production.

An on-call rotation is not a convenience; it is a constitutional requirement. A feature that ships without an on-call owner is a feature whose incidents will be answered by nobody, whose alerts will page into the void, and whose failures will accumulate unobserved. The Playbook forbids this condition absolutely: no on-call, no release.

Constitution Reference: *Volume I, Chapter 5.09 (On-Call and Operational Ownership); Volume I, Chapter 2 §2.10 (Principle 10: You Build It, You Run It) — PRINCIPLE-10 / OP-OWN-001*

13.2 On-Call Rotation

Every production service SHALL be covered by a continuous on-call rotation. The rotation SHALL consist of at least two roles — a primary on-call and a secondary on-call — so that no single individual is the sole responder and so that the primary can escalate without leaving the service uncovered.

- **Primary on-call** — the first responder to every page. Acknowledges within 5 minutes; drives triage and mitigation; convenes the war room when an incident is declared (Chapter 16).
- **Secondary on-call** — the backup responder. Acknowledges within 10 minutes if the primary has not; assumes the primary role if the primary is unreachable or already engaged in a different incident.

Rotation length SHALL be one calendar week, beginning Monday 10:00 local time and ending the following Monday 10:00 local time. The 10:00 handoff time is chosen so that the incoming primary takes the pager at the start of the working day rather than at the end of the outgoing primary's day, when fatigue and context-loss are highest. A one-week rotation balances the cost of context-switching (favouring longer rotations) against the cost of fatigue and disrupted sleep (favouring shorter rotations).

Handoff day and time are normative: the rotation SHALL NOT drift, and a primary who cannot take the rotation at the designated time SHALL arrange a documented swap with another eligible engineer at least 72 hours in advance. Ad hoc swaps within the 72-hour window require the Tech Lead's concurrence and a recorded note in the rotation log.

Follow-the-sun considerations: at RC1, Opus Publica operates a single-region deployment with a single time-zone-distributed team. The primary/secondary model assumes that both roles are reachable within the SLA-defined acknowledgement window regardless of local hour. When the team grows to span

multiple regions, the rotation SHALL be split into region-local sub-rotations with a documented transfer of the pager at the regional handoff boundary. Follow-the-sun does not eliminate the secondary; it changes the secondary's region.

13.3 On-Call Eligibility

Eligibility for on-call is earned by shipping. Every engineer who has merged at least one feature to the release branch of the service in question is eligible for the rotation. The eligibility rule is the operational form of Principle 10: an engineer who has not shipped a feature has no operational context for the service and SHALL NOT carry its pager.

The converse is also true: an engineer who has shipped a feature to the service SHALL join the rotation. Shipping creates a debt of operational ownership that is repaid by carrying the pager. The Tech Lead SHALL NOT exempt an engineer from the rotation on the grounds of seniority, role, or preference; the only exemptions are documented leave, on-call for a different service in the same window (a conflict resolved by the Tech Lead), and the training gap described below.

Before joining the rotation, an engineer SHALL complete on-call training. Training is not a formality; it covers the on-call kit, the runbook library, the alert-routing configuration, the rollback tooling, the communication channels, the break-glass procedure (Chapter 14 §14.5), the war-room convention (Chapter 16 §16.4), and the incident-command handoff. Training is conducted by the outgoing rotation lead and SHALL include at least one shadow shift in which the trainee follows the primary through a full rotation without holding the pager.

Eligibility Criterion	Requirement	Verified By
Shipped feature	At least one merged PR to the service's release branch.	Tech Lead (rotation roster).
On-call training	Completed the on-call training syllabus.	Rotation lead (training log).
Shadow shift	One full rotation shadowing the primary.	Rotation lead (shadow log).
Runbook familiarity	Read every runbook for the service.	Rotation lead (read-receipts).
Tooling access	On-call kit access provisioned and tested.	Platform team (access log).

13.4 On-Call Authority

The on-call engineer holds defined authority to act in the operational interest of the service. This authority is granted so that the on-call can protect users and the scholarly record without waiting for approval that would arrive too late. The authority is bounded: it authorizes the actions needed to mitigate incidents; it does not authorize the actions that change the system's constitutional posture.

What the On-Call MAY Do

- **Rollback** — execute a rollback to the last certified release without further approval when an incident is in progress.
- **Shed load** — shed non-essential load (disable feature flags, throttle ingest, defer batch jobs) to protect critical-path user requests.
- **Page experts** — page subject-matter experts, the Release Manager, the RCA, or the Chief Systems Architect when the incident exceeds the on-call's expertise or authority.
- **Declare incident** — declare a SEV-1 or SEV-2 incident and convene a war room (Chapter 16).
- **Toggle feature flag** — enable or disable a feature flag for mitigation, with the change recorded in the flag audit log.
- **Break-glass** — invoke break-glass production access under the conditions of Chapter 14 §14.5.

What the On-Call MAY NOT Do

- **Change the Constitution** — the Constitution is amended only through Evolution Governance (Vol II Ch 36). The on-call cannot suspend an invariant, override a rule, or modify a contract.
- **Skip certification** — the Production Acceptance Gate (Chapter 19) is absolute. The on-call cannot release an uncertified build to mitigate an incident; the on-call rolls back to the last certified build.
- **Delete artifacts** — published artifacts are immutable except via the legal-removal process (Chapter 14 §14.7). The on-call cannot delete a published article, even one containing errors.
- **Modify the audit log** — the audit log is append-only and immutable (Vol II Ch 34).
- **Direct DB mutation** — production database writes go through the application layer; direct DB mutation is break-glass only (Chapter 14 §14.5).

Authority Boundary

When in doubt, the on-call rolls back and escalates. The on-call does not make architectural decisions under pressure; the on-call restores service to the last known-good state and hands the architectural decision to the team in daylight. This is the operational form of the Principle of Reversibility (Vol I Ch 2 §2.7).

13.5 The On-Call Kit

Every on-call engineer SHALL have provisioned access to the on-call kit before taking the pager. The kit is the set of tools, dashboards, and credentials that the on-call uses to assess, mitigate, and communicate. Kit access is verified at handoff; an incoming primary who cannot reach any kit component SHALL NOT accept the pager.

Kit Component	What It Provides	Access Verified At
SLO dashboard	Live SLI values, error-budget state, burn-rate alerts.	Handoff (manual).

Runbook library	Step-by-step responses for every alert and failure mode.	Handoff (manual).
Alert routing console	PagerDuty/Opsgenie routing; acknowledgement; escalation.	Handoff (manual).
Rollback tooling	One-command rollback to any certified release.	Handoff (manual + automated probe).
Communication channels	War-room bridge, incident chat, status-page console.	Handoff (manual).
Break-glass credentials	Time-boxed, audited production access.	Provisioned by Platform; verified weekly.
Deployment console	Read access to deployment state; write for rollback.	Handoff (manual).
Audit log viewer	Read-only access to the immutable audit log.	Handoff (manual).

The on-call kit is the operational surface area of the role. Its components SHALL be documented in a single on-call kit manifest that the incoming primary reviews at every handoff. The manifest is versioned; changes to the kit (a new dashboard, a renamed runbook, a re-provisioned credential) SHALL be reflected in the manifest within 24 hours.

13.6 Handoff Protocol

Handoff is the most error-prone moment in the rotation. The incoming primary inherits not just the pager but the state of every open alert, every in-flight deployment, and every unresolved anomaly. A sloppy handoff is the proximate cause of a large class of incidents in which the new primary discovers, mid-page, that the previous primary knew something they did not write down. The handoff protocol exists to make that class of incident impossible.

Procedure: On-Call Handoff

Step 1. Review open incidents The outgoing primary briefs the incoming primary on every open incident: current state, mitigation in effect, next planned action, and the war-room channel link if any.

Step 2. Review pending alerts The outgoing primary walks the incoming primary through every alert that fired in the last 24 hours but was not promoted to an incident. Each alert is either closed (with rationale) or carried forward as a watch item.

Step 3. Review in-flight deployments The outgoing primary identifies every deployment in progress: which stage of the rollout, the observation window remaining, the rollback trigger conditions, and the owning Release Manager.

Step 4. Review scheduled jobs The outgoing primary identifies every scheduled job (backups, reconciliation, DOI deposits, preservation ticks) that will fire during the incoming primary's shift, with the expected and the failure-mode response.

Step 5. Transfer the pager The incoming primary acknowledges a test page on their device to confirm alert routing. The outgoing primary does not close their rotation entry in the log until the test page is acknowledged.

Step 6. Record the handoff The handoff SHALL be recorded in the rotation log with timestamp, incoming primary, outgoing primary, open items carried forward, and any anomalies noted. The log entry is the audit artifact.

13.7 On-Call Compensation and Wellbeing

On-call is a load. The Playbook acknowledges this and establishes policy to prevent the load from becoming corrosive. Where local employment law or collective agreement requires compensation for on-call duty and for pages answered outside working hours, that compensation SHALL be paid. Where no such requirement exists, the organization SHALL nonetheless recognize the load through time-off-in-lieu or equivalent, because unrecognized on-call load produces burnout and burnout produces incidents.

- **Max consecutive shifts** — no engineer SHALL carry the primary pager for more than two consecutive weeks. A two-week primary stint is permitted only when the rotation cannot be staffed otherwise and SHALL be followed by at least one full week off the rotation.
- **Post-incident recovery** — an engineer who has been paged between 22:00 and 06:00 local for more than 30 minutes of work SHALL be excused from the next morning's standup and MAY take a half-day recovery with no loss of pay or PTO balance.
- **No on-call during PTO** — an engineer on PTO SHALL NOT be on the rotation. The Tech Lead SHALL re-roster the rotation before the PTO begins. If the engineer is the only eligible primary, the service's release SHALL be deferred until the engineer returns or a new primary is trained.
- **No concurrent rotations** — an engineer on call for one service SHALL NOT simultaneously be on call for another service. Cross-service concurrency produces divided attention and SHALL be resolved by the Tech Lead before the rotation begins.

These wellbeing rules are not optional courtesies; they are the operational form of the Constitution's duty to the engineer as a person. A team that burns out its on-call loses the rotation, and a service that loses its rotation loses its release authority.

13.8 No-On-Call-No-Release Rule

RULE OP-OWN-002 A service without a staffed, trained, and tested on-call rotation SHALL NOT be released to production. A feature without an identified on-call owner in the owning team's rotation SHALL NOT be released. The Release Manager SHALL verify the rotation at the Release Gate (Chapter 11) and SHALL block release if the rotation is missing, understaffed, or untrained.

The No-On-Call-No-Release rule is the operational teeth of Principle 10. It is enforced at two points: at service first-release (the service cannot go to production until a rotation exists for it), and at feature release within an existing service (the feature cannot go to production until the owning team's rotation has been updated to include it in the runbook and the on-call kit).

The rule is also the gate against orphan features. A feature whose implementing engineer has left the team, and whose owning team has not absorbed it into a rotation, is an orphan. Orphans are forbidden; the team SHALL either adopt the feature into a rotation or remove it from the service before the next release.

Procedure: Responding to a Page

Step 1. Acknowledge the page Acknowledge within the SLA window (5 minutes for primary, 10 for secondary). Acknowledgement is not resolution; it tells the routing system that a human is engaged.

Step 2. Read the alert payload The alert SHALL carry: SLI name, current value, threshold crossed, burn rate, affected service, runbook link, and recent deployments. If any field is missing, file a runbook-fix ticket after the incident.

Step 3. Open the linked runbook Every alert SHALL link to a runbook. The on-call follows the runbook step-by-step. If the runbook does not cover the observed symptom, the on-call escalates to the SME and the runbook is amended post-incident.

Step 4. Assess severity Decide whether the page is a SEV-1, SEV-2, SEV-3, or noise. If SEV-1 or SEV-2, declare an incident (Chapter 16). If noise, silence the alert (not the SLI) and file an alert-quality ticket.

Step 5. Mitigate or escalate Mitigate within authority (rollback, shed load, toggle flag). If mitigation exceeds authority, escalate to the SME, Release Manager, or Chief Systems Architect per the authority matrix (Chapter 14).

Step 6. Communicate Post an initial update to the incident channel within 15 minutes of acknowledgement, even if the update is 'investigating'. Silence breeds panic; visible progress breeds calm.

Step 7. Resolve or hand off If the page is resolved, record the resolution with timestamp and root-cause hypothesis. If the page is unresolved at shift end, hand it off per the handoff protocol (§13.6).

Runbook: First 5 Minutes of a Page

Acknowledge

Acknowledge the page in the alert-routing console. The acknowledgement stops the escalation timer. If you are the secondary and the primary has not acknowledged within 5 minutes, you become the primary and acknowledge.

Assess

Read the alert payload: SLI, current value, threshold, burn rate, service, runbook link, recent deployments. Open the runbook. Glance at the SLO dashboard to confirm the alert is real and not a measurement artifact. Glance at the deployment console to see if a recent deployment is the proximate cause.

Act

If the runbook prescribes a mitigation (typically: rollback to the last certified release, shed non-essential load, or toggle a feature flag), execute it now. Mitigation precedes investigation. The scholarly record and the user come first; root cause comes after service is restored.

Alert

If the mitigation is not sufficient, or the runbook does not cover the symptom, escalate. Page the SME. If the impact is user-visible and severe, declare an incident (Chapter 16) and convene a war room. Post the first update to the incident channel.

Ascertain

Once service is restored, ascertain what happened. Capture logs, traces, and the timeline before they rotate. If the incident is SEV-1 or SEV-2, a postmortem will be required (Chapter 16 §16.7); the artifacts you preserve now are the postmortem's evidence.

Checklist: On-Call Readiness

- ☐ **OC-01** Rotation roster published for the next 8 weeks; no gaps.
- ☐ **OC-02** Every primary and secondary has shipped at least one feature to the service.
- ☐ **OC-03** Every primary and secondary has completed on-call training and one shadow shift.
- ☐ **OC-04** On-call kit manifest current; every component reachable.
- ☐ **OC-05** Runbook for every alert exists and links from the alert.
- ☐ **OC-06** Rollback tooling tested in the last 30 days.
- ☐ **OC-07** Break-glass credentials provisioned and not expired.
- ☐ **OC-08** SLO dashboard live; burn-rate alerts firing.
- ☐ **OC-09** Communication channels (war-room bridge, incident chat, status page) reachable.
- ☐ **OC-10** Handoff log template available; last handoff recorded.
- ☐ **OC-11** No engineer on PTO is on the rotation.
- ☐ **OC-12** No engineer holds concurrent primary on two services.

13.9 Roles and Ownership

Role	Owns	Authority	Boundaries
------	------	-----------	------------

Primary On-Call	First response; mitigation; rollback.	Rollback; shed load; toggle flag; declare incident; break-glass.	Cannot change Constitution; cannot skip certification; cannot delete artifacts.
Secondary On-Call	Backup response; primary escalation target.	Assumes primary if primary unreachable; same authority as primary.	Same boundaries as primary.
Rotation Lead	Roster; training; shadow shifts.	Schedules the rotation; certifies training completion.	Cannot exempt engineers from eligibility; cannot override CSA.
Tech Lead	Service ownership; runbook library.	Approves swaps; resolves cross-service conflicts; assigns orphans.	Cannot suspend the No-On-Call-No-Release rule.
Release Manager	Release Gate enforcement.	Blocks release if rotation missing or understaffed.	Cannot override the No-On-Call-No-Release rule.
Chief Systems Architect	Constitutional posture.	Approves exceptions to the rotation model in writing.	Cannot suspend invariants; cannot override RCA on runtime.

13.10 Evidence and Records

On-call operations produce evidence that SHALL be retained for audit, postmortem, and regulatory review. The evidence is the proof that the rotation was staffed, the handoff was performed, the alert was answered, and the mitigation was executed. Without this evidence, the operational state of the service is unverifiable.

Record	Produced By	Storage	Retention
Rotation roster	Rotation Lead	Rotation log (versioned).	Lifetime of the service + 5 years.
Handoff log entries	Outgoing primary	Rotation log.	Lifetime + 5 years.
Training completion	Rotation Lead	Training log.	Lifetime of the engineer's tenure + 5 years.
On-call kit manifest	Platform team	Runbook repository.	Lifetime + 5 years.
Page acknowledgements	Alert routing system	Observability platform.	90 days hot; 2 years cold.
Mitigation actions	On-call engineer	Audit log (immutable).	Lifetime of the platform.
Break-glass invocations	On-call engineer	Audit log + break-glass	Lifetime of the platform.

		review queue.	
Wellbeing time-off records	HR / Tech Lead	HR system.	Per local employment law (minimum 2 years).

Chapter 14 — Operational Governance Execution

14.1 Purpose

This chapter operationalizes Volume II Chapter 34 (Operational Governance). Where Volume II Chapter 34 establishes the authority matrix in normative form, this chapter is the execution manual: who performs each action, who approves it, what audit trail is recorded, and how the action is executed step by step. The chapter is the answer to the everyday question — 'who is allowed to do this, and how do they do it?' — for every operationally significant action on the Opus Publica platform.

Operational governance exists because the platform's invariants (Vol II Ch 26) and the scholarly record's immutability requirements make many actions irreversible or reversible only at high cost. The authority matrix is not a bureaucratic layer; it is the mechanism by which the Constitution's protective rules are enforced at the moment of action. An action taken outside the matrix is an action that bypasses the Constitution, and the audit log SHALL make such bypass visible.

Constitution Reference: Volume II, Chapter 34 (Operational Governance) — OP-GOV-001 ... OP-GOV-014

14.2 The Authority Matrix in Practice

The authority matrix enumerates the fourteen operationally significant actions on the platform, assigns each a performer and an approver, and specifies the audit record that SHALL be produced. The matrix is normative: no action MAY be performed except as the matrix specifies. Three of the fourteen actions are forbidden absolutely; one is forbidden except via the legal-removal process (§14.7).

#	Action	Performer	Approver	Audit Recorded	How-To
1	Press Publish	Release Manager	RCA	Release record + audit log entry.	Ch 11 release procedure.
2	Issue Retraction	Editorial (engineering executes)	Editorial Board + CSA	Retraction record; DOI metadata update; audit log.	Ch 14 §14.7 (legal-removal variant).
3	Issue Correction	Editorial (engineering executes)	Editorial Board	Correction record; DOI metadata update; audit log.	Correction notice workflow.
4	Rotate External Credential	Platform on-call	Tech Lead	Credential rotation log; audit log.	Ch 14 §14.3 procedure.
5	Rotate TLS	Platform on-call	Tech Lead	Certificate	Ch 14 §14.3

	Certificate			rotation log; audit log.	procedure.
6	Approve DB Migration	DB Migration Reviewer	RCA + CSA	Migration proposal + approval record.	Ch 14 §14.4 procedure.
7	Approve Production Deployment	Release Manager	RCA	Deployment record; RCA sign-off.	Ch 11 deployment procedure.
8	Approve Rollback	On-call	On-call (within authority)	Audit log; incident record if applicable.	Ch 12 rollback procedure.
9	Initiate Disaster Recovery	On-call	CSA	DR activation record; audit log.	Ch 14 §14.6 runbook.
10	Declare Incident	On-call	On-call	Incident record; audit log.	Ch 16 procedure.
11	Grant Break-Glass Access	On-call (self, time-boxed)	Tech Lead (post-hoc review)	Break-glass log; review queue.	Ch 14 §14.5 procedure.
12	Modify Audit Log	FORBIDDEN	FORBIDDEN	N/A (attempt logged as security event).	N/A — see §14.8.
13	Delete Published Artifact	FORBIDDEN except legal-removal	CSA + Legal	Legal-removal record; cryptographic proof preserved.	Ch 14 §14.7 procedure.
14	Override Invariant	FORBIDDEN	FORBIDDEN (CSA cannot override)	N/A (attempt logged).	N/A — see §14.8.

14.3 Key and Credential Rotation

Credentials are the soft underbelly of the platform. A leaked Crossref credential can mint fraudulent DOIs; a leaked ORCID credential can attribute works to the wrong scholar; a leaked preservation credential can corrupt the long-term archive; a leaked database credential can bypass the application's invariants; a leaked TLS private key can impersonate the platform; a leaked signing key can forge release artifacts. Rotation is the discipline by which the platform limits the blast radius of any single credential compromise.

Each credential type has a defined rotation cadence, a defined rotation procedure, and a defined emergency-rotation trigger. The cadence is the maximum interval between scheduled rotations; an emergency rotation SHALL be triggered immediately on suspicion of compromise, without waiting for the cadence interval.

Credential	Cadence	Emergency Trigger	Rotation Window
Crossref API (deposit + query)	90 days	Suspected leak; anomalous deposit.	1 hour.
ORCID API	90 days	Suspected leak.	1 hour.
Preservation (CLOCKSS/LOCKSS) cred	180 days	Suspected leak; vendor notice.	4 hours.
Database application role	90 days	Suspected leak; staff turnover.	2 hours (zero-downtime).
Database break-glass admin	30 days	Each invocation; suspected leak.	1 hour.
TLS private key (public endpoints)	90 days (or per CA)	Suspected leak; CA compromise notice.	2 hours.
Internal service-to-service mTLS	90 days	Suspected leak.	2 hours.
Release signing key	365 days	Suspected leak; key custodian change.	24 hours (with re-sign).
OIDC / SSO client secrets	180 days	Suspected leak; vendor notice.	4 hours.

Procedure: Rotating an External Credential

Step 1. Identify the credential and its dependents Every external credential has a list of services that consume it. The rotation SHALL be coordinated across all dependents; an uncoordinated rotation breaks the dependent that did not get the new value.

Step 2. Generate the new credential at the provider Generate the new credential (API key, secret, token) at the provider's console. Do NOT revoke the old credential yet; dual-credential validity during the rotation window is the zero-downtime technique.

Step 3. Store the new credential in the secret store Store the new credential in the platform's secret store (Vault, AWS Secrets Manager, or equivalent) under a versioned path. The old credential remains at the previous version.

Step 4. Deploy the new credential version to dependents Trigger a rolling redeploy of every dependent service so that each picks up the new credential version. Verify, per service, that the new credential is in use (log line, metric, or health check).

Step 5. Verify the new credential end-to-end Execute a live transaction through each dependent that exercises the credential (a Crossref deposit, an ORCID lookup, a preservation tick). The transaction SHALL succeed.

Step 6. Revoke the old credential Only after every dependent has been verified on the new credential, revoke the old credential at the provider. Revoking earlier breaks dependents that have not yet picked up the new value.

Step 7. Record the rotation Record the rotation in the credential rotation log: credential name, old version, new version, performers, approver, timestamp, verification evidence. The log entry is immutable.

Procedure: Rotating a TLS Certificate

Step 1. Issue the new certificate Issue the new certificate via the platform's certificate authority (ACME / internal CA). The new certificate SHALL use a fresh private key; key reuse across rotations is forbidden.

Step 2. Deploy the new certificate to the load balancer / ingress Upload the new certificate (with its chain) to the load balancer or ingress controller. Most platforms support dual-certificate deployment during the cutover window.

Step 3. Verify the new certificate is served From an external vantage point, verify that the new certificate is served (openssl s_client, SSL Labs). The verification SHALL confirm the chain, the validity dates, and the SAN list.

Step 4. Verify the new certificate is trusted Confirm that the platform's own trust store (and any external monitors) accept the new certificate. A self-issued CA certificate SHALL be distributed to the trust store before the cutover.

Step 5. Remove the old certificate Once the new certificate is verified and trusted, remove the old certificate from the load balancer. The old private key SHALL be securely destroyed (cryptographic erase).

Step 6. Record the rotation Record the rotation in the certificate rotation log: certificate name, old serial, new serial, performer, approver, timestamp, verification evidence.

14.4 Migration Approval and Execution

A database migration is a change to the schema or to a large body of data that the application cannot perform unilaterally. Migrations are high-risk because they can violate invariants (Vol II Ch 26), corrupt the scholarly record, or render the application incompatible with the data it reads. The migration regime is the gate that prevents these outcomes.

Every migration SHALL traverse seven states: proposal, review, approval, scheduling, execution, verification, and rollback-ready. Each state has an owner and an exit criterion; no state MAY be skipped.

State	Owner	Exit Criterion
-------	-------	----------------

Proposal	Implementing engineer	Migration plan documented; up AND down scripts written.
Review	DB Migration Reviewer	Plan reviewed; invariant impact assessed; backup strategy confirmed.
Approval	RCA + Chief Systems Architect	Written approval recorded; rollback window confirmed.
Scheduling	Release Manager	Migration scheduled in a defined maintenance window.
Execution	Release Manager + on-call	Migration applied; down-script verified to reverse it.
Verification	QA + RCA	Five-pillar verification; invariants green; SLO dashboard green.
Rollback-ready	On-call	On-call briefed; down-script tested; rollback window open.

Procedure: Executing a Database Migration

Step 1. Confirm approval artifacts are in place The migration plan, the invariant impact assessment, the RCA + CSA approvals, and the maintenance-window schedule SHALL be present before execution begins. Missing artifacts block execution.

Step 2. Take a verified backup Take a backup of the affected tables (or the full database if the migration is wide). Verify the backup by restoring it to a staging instance and running a smoke test. An unverified backup is no backup.

Step 3. Apply the up migration Apply the migration via the migration tool. The migration SHALL be idempotent where possible and SHALL be applied in a transaction where the database engine permits.

Step 4. Verify the schema change Confirm the schema change is present (describe, information_schema). Confirm no orphaned objects, no broken foreign keys, no missing indexes.

Step 5. Verify the application against the new schema Run the application's smoke test suite against the migrated database. The suite SHALL pass. Invariant monitors SHALL be green.

Step 6. Verify the down migration in staging Apply the down migration in a staging instance restored from the post-migration state. Confirm the down migration reverses the up without data loss. If the down migration is lossy, document the loss explicitly and obtain CSA approval to proceed.

Step 7. Monitor for the rollback window The migration SHALL be monitored for the defined rollback window (typically 24 hours). During this window, the on-call MAY roll back without further approval. After the window, rollback requires a fresh migration proposal.

Step 8. Record the migration Record the migration in the migration log: plan, approvers, timestamp, verification evidence, rollback window expiry. The log entry is the audit artifact.

14.5 Break-Glass Procedures

Break-glass is the controlled invocation of elevated production access outside the normal application-mediated path. Break-glass is not a convenience; it is a last-resort operational tool, used when the application cannot restore service and direct intervention is the only path. Because break-glass bypasses the application's invariants, every invocation is heavily audited and reviewed.

Break-glass is justified only when: (a) an incident is in progress or imminent; (b) the application-mediated path (rollback, feature flag, restart) has been tried and failed; (c) the on-call's normal authority is insufficient; and (d) the action is the minimum necessary to restore service. Break-glass for development, for data exploration, or for ad-hoc fixes is forbidden.

Procedure: Break-Glass Production Access

Step 1. Declare the break-glass intent The on-call declares the break-glass intent in the incident channel: the justification, the target system, the action to be taken, and the expected duration. The declaration is the start of the audit trail.

Step 2. Check out the break-glass credential The on-call checks out the time-boxed break-glass credential from the secret store. The checkout SHALL record: who, when, target, justification, expiry. The credential expires automatically at the time box (default 60 minutes).

Step 3. Perform the minimum necessary action The on-call performs the minimum action necessary to restore service. Every command SHALL be recorded (terminal session log, database audit, infrastructure log). 'Minimum necessary' is judged post-hoc by the Tech Lead in the break-glass review.

Step 4. Restore service and verify Confirm that the action restored service. Verify via the SLO dashboard, the invariant monitors, and the user-facing health checks. If service is not restored, escalate; do not extend the break-glass without re-declaring.

Step 5. Check in the credential On expiry or on completion, the credential is automatically checked in and rotated. The on-call SHALL NOT retain the credential beyond the time box. The rotation is logged.

Step 6. File the break-glass review ticket Every break-glass invocation SHALL be reviewed by the Tech Lead within one business day. The review asks: was the invocation justified? was the action minimum-necessary? what application gap required break-glass? what fix prevents the next invocation?

Step 7. Record the invocation Record the invocation in the break-glass log: declaration, credential checkout, actions, verification, review ticket, reviewer, outcome. The log entry is the audit artifact.

14.6 Disaster Recovery Execution

Disaster recovery (DR) is the response to a failure that exceeds the on-call's normal mitigation authority — a regional outage, a data-center loss, a ransomware event, or a catastrophic data corruption. DR is the

operational form of the Constitution's Recovery Time Objective (RTO 1 hour) and Recovery Point Objective (RPO 15 minutes) for the production database (Vol II Ch 33).

DR activation authority rests with the Chief Systems Architect. The on-call discovers the disaster, declares the incident, and escalates to the CSA; the CSA authorizes DR activation. This separation exists because DR is destructive: a failover to the DR site abandons in-flight transactions on the primary, and an unjustified failover is itself an incident.

DR SHALL be tested quarterly. The test SHALL exercise the full failover path (not a simulation), SHALL measure the actual RTO and RPO against the targets, and SHALL produce a test report reviewed by the CSA. A DR test that does not exercise the full path is a drill, not a test, and SHALL be repeated as a test.

Runbook: Disaster Recovery Activation

Declare

The on-call declares the disaster to the CSA by direct call (not chat). The declaration SHALL state: the affected region, the observed failure, the in-flight user impact, and the estimated RPO gap. The CSA authorizes DR activation in writing (chat message with explicit 'DR-AUTHORIZED' token).

Assemble

The on-call convenes the DR team: CSA, Release Manager, DBA, Platform on-call, and the service owners of every affected service. The war room is the DR bridge; the scribe is the rotation lead.

Assess

The team assesses the DR readiness: is the DR site healthy? is the last backup within RPO? are the DNS, CDN, and TLS certificates ready to point at the DR site? is the application configuration correct for the DR site?

Recover

The team executes the failover: promote the DR replica to primary, redirect DNS / CDN, verify the application reaches the new primary, and verify the SLO dashboard reflects the new topology. Each step is recorded in the DR log.

Verify

Verify end-to-end: user-facing health checks green, SLO dashboard green, invariant monitors green, audit log appending to the new primary, scholarly record (DOI resolution, preservation ticks) functional.

Stand-down

The CSA declares DR stand-down when service is verified. The stand-down does NOT revert to the original primary; that reversion is a separate, scheduled migration (Chapter 14 §14.4) executed in a maintenance window.

Postmortem

A blameless postmortem (Chapter 16 §16.7) SHALL be conducted within 5 business days. The postmortem SHALL cover: root cause of the disaster, DR activation timeliness, actual RTO and RPO vs targets, gaps in the DR plan, corrective actions.

14.7 Legal Removal Process

A published artifact (an article, a book chapter, a dataset) is immutable in the scholarly record. The Constitution forbids deletion of published artifacts (Vol II Ch 34). The single, narrow exception is the legal-removal process: a court order, a regulatory directive, or a documented takedown demand that the platform is legally compelled to honour. Legal removal is the **ONLY** path to removing a published artifact, and it SHALL preserve the cryptographic proof of the artifact's existence even as the artifact itself is removed.

Procedure: Legal Removal of a Published Artifact

Step 1. Intake the legal request The legal request is received by Legal and logged in the legal-removal intake queue with: requestor, jurisdiction, legal basis, target artifact(s), demanded action, response deadline. Intake SHALL occur within one business day of receipt.

Step 2. Legal review Legal reviews the request for validity, scope, and the platform's response obligations. Legal SHALL produce a written recommendation: comply, partially comply, contest, or refer to outside counsel.

Step 3. CSA approval The Chief Systems Architect approves the legal removal in writing. The CSA SHALL NOT approve a removal that lacks Legal's comply recommendation. The CSA's approval SHALL cite the intake ticket, the legal basis, and the specific artifact(s) to be removed.

Step 4. Preserve the cryptographic proof Before the artifact is removed, the platform SHALL preserve: the artifact's SHA-256 hash, its signed manifest, its DOI metadata, its publication timestamp, and its audit log entries. These are stored in the legal-removal archive, which is itself append-only and immutable.

Step 5. Update DOI metadata The DOI metadata SHALL be updated to reflect the removal: the DOI resolves to a takedown notice that states the legal basis (without disclosing privileged details), the date of removal, and a reference to the preserved cryptographic proof. The DOI SHALL NOT be deleted; a deleted DOI is a broken scholarly record.

Step 6. Remove the artifact from public access The artifact is removed from public access: the public pages return the takedown notice, the search index is updated, the OAI-PMH endpoint returns the removal record. The artifact's underlying storage MAY be retained in the legal-removal archive; it is not publicly retrievable.

Step 7. Notify stakeholders The platform notifies the affected authors, the editorial board, and (where required) the requesting party. Notifications SHALL be templated and reviewed by Legal before sending.

Step 8. Record the removal Record the removal in the legal-removal log: intake ticket, legal recommendation, CSA approval, preserved cryptographic proof reference, DOI metadata update,

notification recipients, timestamp. The log entry is the audit artifact and SHALL be retained for the lifetime of the platform.

14.8 Forbidden Actions

The Constitution defines actions that are NEVER permitted, regardless of role, authority, or operational pressure. These forbidden actions are the bedrock protections of the scholarly record and the Constitution's invariants. An attempt to perform a forbidden action SHALL be logged as a security event and reviewed by the CSA; a successful forbidden action constitutes a constitutional violation and triggers the Evolution Governance amendment or, if warranted, the incident process.

Forbidden Action	Why Forbidden	Detection	Response
Modify the audit log	The audit log is the constitutional record; modification destroys verifiability (Vol II Ch 34).	Append-only storage; hash-chained entries; periodic external hash verification.	Treat as security incident; CSA review; legal escalation.
Delete a published artifact (except legal-removal)	The scholarly record is immutable; deletion corrupts the record (Vol II Ch 26).	Storage is append-only; public artifact manifest is hash-chained.	Treat as constitutional violation; CSA + Legal review.
Override an invariant	Invariants are mathematical truths; no operational pressure justifies their suspension (Vol II Ch 26).	Invariant monitors fire on violation; CSA cannot override.	Block the operation; treat as incident.
Skip certification	The Production Acceptance Gate is absolute (Vol I Ch 8).	Release Gate checks for RCA sign-off.	Block release; RCA review.
Deploy without RCA sign-off	RCA sign-off is the certification record.	Deployment console checks for sign-off artifact.	Block deployment; escalate to CSA.
Direct production DB mutation (except break-glass)	DB mutation bypasses application invariants.	DB audit logs non-application sessions; break-glass log cross-check.	Treat as incident; break-glass review.
Bypass the legal-removal process to remove an artifact	Removal outside the process destroys the cryptographic proof chain.	Removal log cross-checks with storage delete events.	Treat as constitutional violation; CSA + Legal review.

Checklist: Operational Action Authorization

- ☐ **OA-01** Action is one of the fourteen in the authority matrix.
- ☐ **OA-02** Performer is the role assigned by the matrix.
- ☐ **OA-03** Approver is the role assigned by the matrix (if any).
- ☐ **OA-04** If action is forbidden, action is NOT performed and the attempt is logged.
- ☐ **OA-05** If action is legal-removal, the §14.7 procedure is followed end-to-end.
- ☐ **OA-06** Credential rotation recorded in rotation log with verification evidence.
- ☐ **OA-07** Migration has proposal, review, approval, scheduling, execution, verification, rollback-ready states all satisfied.
- ☐ **OA-08** Break-glass invocation has declaration, time-boxed credential, recorded actions, review ticket.
- ☐ **OA-09** DR activation has CSA authorization in writing; DR log entries for each step.
- ☐ **OA-10** No audit log modification attempted; audit log hash chain intact.
- ☐ **OA-11** No invariant override attempted; invariant monitors green.
- ☐ **OA-12** No direct DB mutation outside break-glass; DB audit log cross-checked.
- ☐ **OA-13** Every action produces an audit log entry with: actor, action, target, timestamp, justification.
- ☐ **OA-14** Action evidence retained per the retention schedule (Vol II Ch 31 §31.8).
- ☐ **OA-15** Forbidden-action attempts (if any) are in the CSA review queue.

14.9 Roles and Ownership

Role	Owns	Authority	Boundaries
On-Call Engineer	First response; mitigation.	Rollback; shed load; declare incident; break-glass (time-boxed).	Cannot perform forbidden actions; cannot skip matrix approval.
Tech Lead	Service operations; runbook library.	Approves credential rotations; reviews break-glass.	Cannot approve migrations (RCA + CSA); cannot approve DR.
Release Manager	Release and deployment execution.	Approves production deployment (with RCA sign-off); schedules migrations.	Cannot override No-On-Call-No-Release; cannot deploy without RCA.
DB Migration Reviewer	Migration review.	Reviews migration plan; assesses invariant impact.	Cannot approve migration (RCA + CSA); cannot execute migration.

Release Certification Authority	Certification.	Approves DB migration; signs off production deployment.	Cannot override invariant failure; cannot be overridden by CSA on runtime.
Chief Systems Architect	Constitutional posture; DR.	Approves migrations; authorizes DR; approves legal removals.	Cannot override invariants; cannot modify audit log.
Legal	Legal-removal intake and review.	Recommends comply / contest on legal requests.	Cannot approve removal (CSA); cannot execute removal.

14.10 Evidence and Records

Operational governance produces evidence for every action. The evidence is the proof that the action was authorized, performed by the right role, and recorded in the audit log. The evidence SHALL be retained per the Constitution's retention schedule (Vol II Ch 31 §31.8) and SHALL be retrievable for audit, regulatory review, and post-incident analysis.

Record	Produced By	Storage	Retention
Authority matrix entries (audit log)	Action performer	Immutable audit log.	Lifetime of the platform.
Credential rotation log	Platform on-call	Credential rotation log (versioned).	Lifetime + 5 years.
Certificate rotation log	Platform on-call	Certificate rotation log (versioned).	Lifetime + 5 years.
Migration proposal and approval	Engineer + RCA + CSA	Migration log.	Lifetime + 5 years.
Break-glass log	On-call engineer	Break-glass log + audit log.	Lifetime of the platform.
DR activation record	On-call + CSA	DR log + audit log.	Lifetime of the platform.
Legal-removal record	Legal + CSA	Legal-removal archive (append-only).	Lifetime of the platform.
Forbidden-action attempts	Detection systems	Security event log; CSA review queue.	Lifetime of the platform.

Chapter 15 — SLO Management

15.1 Purpose

This chapter operationalizes Volume II Chapter 32 (Production SLO Book) for day-to-day SLO operations. Where Volume II Chapter 32 specifies the ten SLIs, the ten SLOs, the three SLAs, and the multi-window burn-rate alerting regime, this chapter is the operating manual: how the team reads the dashboard, how it manages the error budget, how it responds to burn-rate alerts, and how it amends an SLO when reality demands.

SLOs are not metrics that the team observes; they are commitments that the team manages. A service whose SLO is at risk is a service whose reliability work has been under-invested, and the Playbook redirects engineering capacity from feature work to reliability work when the error budget state demands it. This redirection is not optional; it is the operational form of the Constitution's Production SLO Book.

Constitution Reference: Volume II, Chapter 32 (Production SLO Book) — SLO-001 ... SLO-010; BURN-001 ... BURN-004

15.2 The SLO Dashboard

The SLO dashboard is the single screen that every engineer SHALL be able to read at a glance. It is the operational surface of the Production SLO Book; every other SLO artifact (alerts, reviews, amendments) is derived from it. The dashboard SHALL be the first thing the on-call opens at handoff and the first thing the team reviews at the daily ritual.

The dashboard has four panels per SLO, laid out consistently so that an engineer moving from one SLO to another reads the same shape of information. The panels are:

Panel	What It Shows	Action Trigger
SLI value	Current SLI value (e.g., 99.94% availability).	Glance; context for the other panels.
SLO target	The SLO target (e.g., 99.9%).	Compare with SLI; gap = current risk.
Error budget remaining	Percent of budget remaining in the rolling window (e.g., 64%).	<50% = at-risk; <25% = critical; =0% = exhausted.
Burn rate	Current burn rate (e.g., 2.3x).	Triggers multi-window alerts (§15.4).

The daily review ritual is the operational heartbeat of SLO management. Each working day at 10:30 local time, the team convenes for a 15-minute SLO standup. The standup walks every SLO: its current state, its budget trajectory, any alerts that fired in the last 24 hours, and any action required. The standup is

minuted; the minutes are the evidence that the ritual was performed. A team that does not hold the daily SLO standup is a team that is not managing its SLOs, and the Tech Lead SHALL escalate the gap to the Engineering Leadership Team.

15.3 Error Budget Management

The error budget is the amount of unreliability the SLO permits in the rolling window (typically 30 days). A 99.9% availability SLO over 30 days permits 43.2 minutes of downtime; this is the budget. The budget is consumed when the SLI is below the SLO target; it is replenished when the SLI is above. The budget is the shared resource that aligns feature work and reliability work: feature work consumes budget (every release carries risk); reliability work replenishes it.

The budget has four states, and each state has a defined policy. The policy is normative: the team SHALL act as the state demands.

State	Budget Remaining	Policy	Action
Healthy	> 50%	Normal feature work permitted.	Continue planned releases.
At-risk	25% – 50%	Feature work continues with caution; reliability items prioritized in the next sprint.	Tech Lead reviews the burn-down; reliability tickets pulled forward.
Critical	< 25%	Feature freeze for non-reliability work. Only reliability fixes, invariant repairs, and rollback-only changes permitted.	Release Manager blocks non-reliability releases; CSA notified.
Exhausted	0% (budget breached)	Release freeze. No production deployments except rollback, incident mitigation, and reliability fixes that the RCA approves.	CSA directs reliability work; SLO amendment considered (§15.6).

Critical = Feature Freeze; Exhausted = Release Freeze

The state-driven policy is not advisory. A team that continues feature releases into a Critical state is in violation of the SLO regime and the Tech Lead SHALL halt the release. A team that continues any non-reliability release into an Exhausted state is in violation of the Production Acceptance Gate and the CSA SHALL intervene.

15.4 Burn Rate Alerting

Burn rate is the rate at which the error budget is being consumed relative to the steady-state rate that would exhaust the budget exactly at the end of the window. A burn rate of 1x means the budget will be exhausted exactly at window end; a burn rate of 10x means the budget will be exhausted in 1/10 of the window. Burn-rate alerting is the early-warning system that catches reliability regressions before they exhaust the budget.

The Production SLO Book (Vol II Ch 32) specifies four multi-window, multi-burn-rate alerts. Each alert fires when the burn rate exceeds its threshold over BOTH a short window (for fast detection) and a long window (for noise suppression). The dual-window design catches sustained regressions while ignoring transient spikes.

Alert	Short Window	Long Window	Burn Rate	Meaning	Response
Page (fast)	5 min	1 hour	14.4x	Catastrophic regression; budget exhausted in ~2 hours.	Page on-call immediately; mitigate.
Page (slow)	30 min	6 hours	6x	Serious regression; budget exhausted in ~5 hours.	Page on-call; mitigate within 1 hour.
Ticket (medium)	2 hours	1 day	3x	Sustained regression; budget exhausted in ~10 hours.	Auto-file ticket; team investigates in next shift.
Ticket (slow)	6 hours	3 days	1x	Long-term drift; budget exhausted at window end.	Auto-file ticket; team reviews at weekly burn review.

Procedure: Responding to a Burn Rate Alert

Step 1. Acknowledge the alert Acknowledge pages within the SLA (5 min primary, 10 min secondary). Tickets are acknowledged by the assignee within the next working shift.

Step 2. Confirm the alert is real Open the SLO dashboard. Confirm the burn rate is sustained (both windows crossed). Glance at the SLI value, the recent deployments, and any incident in progress. A burn-

rate alert without a visible regression is a measurement error; file a monitoring ticket and silence the alert (not the SLI).

Step 3. Identify the proximate cause Examine the dashboard's per-SLI breakdown: which user journey, which endpoint, which region is burning the budget? Correlate with recent deployments, dependency state, and external factors (Crossref latency, ORCID outage).

Step 4. Mitigate Mitigate within on-call authority: rollback to the last certified release, shed non-essential load, toggle a feature flag. Mitigation precedes root cause; the budget is the resource being protected.

Step 5. Verify the burn rate falls After mitigation, confirm the burn rate is dropping. A mitigation that does not drop the burn rate is not a mitigation; escalate or roll back further.

Step 6. Transition to root cause Once the burn rate is below the alert threshold, transition from mitigation to root cause. If the alert is a Page (fast) or Page (slow), declare an incident and convene a postmortem (Chapter 16). If a Ticket, file the root-cause ticket.

Step 7. Record the response Record the response in the SLO incident log: alert, acknowledgement time, mitigation, root cause, postmortem link. The log entry is the evidence that the burn-rate regime was applied.

15.5 SLO Review Cadence

SLO management is not a single ritual; it is a layered cadence with three review frequencies, each addressing a different timescale and a different audience. The cadence is normative: each review SHALL occur on its schedule, SHALL produce minutes, and SHALL record its decisions in the SLO review log.

Review	Frequency	Audience	Agenda
Burn review	Weekly (Mondays, 30 min)	Team	Past week's burn; alerts fired; tickets open; reliability work in progress; budget trajectory.
SLO review	Monthly (first Wednesday, 60 min)	Engineering Leadership	Past month's SLO performance; budget state per SLO; reliability work delivered; reliability work planned; amendment candidates.
SLO amendment	Quarterly (end of quarter)	Chief Systems Architect	Proposed SLO changes; evidence for change; approval or rejection; communication plan.

15.6 SLO Amendment

An SLO target is not immutable, but it is not lightly changed. Tightening an SLO (raising the target) commits the team to higher reliability and reduces feature-release capacity; loosening an SLO (lowering the target) reduces user experience and may breach an SLA. The amendment regime ensures that every change is evidence-based, approved, and communicated.

An SLO MAY be amended when one of the following conditions holds: (a) the SLO has been breached for two consecutive windows, demonstrating that the target is not achievable with current capacity; (b) user feedback or SLA obligations demonstrate that the target is too loose and a tighter target is warranted; (c) the service's scope has changed (new user journey, new region, new dependency) such that the original SLI no longer captures the user experience; (d) the cost of meeting the SLO is disproportionate to the user benefit, evidenced by a cost-per-publication analysis (Vol II Ch 35).

Procedure: Amending an SLO

Step 1. Draft the amendment proposal The proposing engineer (typically the Tech Lead) drafts the proposal: current SLO, proposed SLO, justification, evidence (SLO performance data, user feedback, cost analysis), implementation plan, communication plan.

Step 2. Engineering Leadership review The proposal is reviewed at the monthly SLO review. The review SHALL assess: is the evidence sufficient? is the proposed target achievable? what is the user-impact communication?

Step 3. CSA approval The Chief Systems Architect approves or rejects the amendment in writing. SLO amendments that affect an SLA SHALL additionally be reviewed by Legal.

Step 4. Implement the amendment The amendment is implemented in the SLO dashboard, the burn-rate alerts, and the runbooks. The implementation SHALL be deployed in a maintenance window with the on-call briefed.

Step 5. Communicate the amendment The amendment is communicated to the team, to stakeholders, and (if user-visible) to users. The communication SHALL state the old target, the new target, the effective date, and the rationale.

Step 6. Record the amendment The amendment is recorded in the SLO amendment log: proposal, review minutes, CSA approval, implementation evidence, communication record. The log entry is the audit artifact.

15.7 SLA Management

An SLA is an external commitment — a reliability target that the platform guarantees to a third party (Crossref, ORCID, preservation systems, OAI-PMH harvesters, DOI resolvers). SLAs are stricter than SLOs because their breach has external consequences: a Crossref SLA breach can suspend the platform's deposit privileges; an OAI-PMH SLA breach can cause harvesters to drop the platform from their index.

SLA	Counterparty	Target	Breach Consequence
-----	--------------	--------	--------------------

OAI-PMH response time	Harvesters (DOAJ, etc.)	P95 < 5s; 99.5% uptime.	Harvester drops platform from index.
Crossref deposit latency	Crossref	Deposit within 24h of publication.	Delayed DOI resolution; Crossmark stale.
DOI resolution uptime	DOI Foundation / users	99.9% uptime.	Users cannot resolve articles; reputational damage.
ORCID attribution latency	ORCID / authors	Attribution within 24h of publication.	Authors' ORCID profiles stale.
Preservation tick	CLOCKSS / LOCKSS	Quarterly tick; 100% of new artifacts.	Archival gap; long-term preservation risk.

An SLA breach is a serious event. The on-call SHALL treat any SLA breach as a SEV-2 incident at minimum, declare it per Chapter 16, and notify the Engineering Leadership Team. The counterparty SHALL be notified per the SLA's communication clause, typically within one business day. A postmortem SHALL be conducted; a breach that recurs in two consecutive windows SHALL trigger an SLO amendment review (§15.6) and may trigger an SLA renegotiation with the counterparty.

Customer communication during an SLA breach SHALL be honest, timely, and specific. The platform's status page SHALL reflect the breach within 30 minutes of detection; the post-incident communication SHALL state the breach's user impact, the root cause, the corrective action, and any commitment to renegotiate the SLA. Vague or delayed communication is itself a breach of the platform's trust contract with its users.

15.8 SLO Anti-Patterns

SLO management has well-known anti-patterns — practices that appear to support the regime but in fact undermine it. The Playbook enumerates these anti-patterns so that the team can recognize and reject them. Each anti-pattern is forbidden; detection of an anti-pattern SHALL trigger a remediation ticket and a Tech Lead review.

Anti-Pattern	Why Forbidden	Detection
Gaming the SLI	Manipulating the SLI measurement (e.g., excluding error responses, caching successful responses) to make the SLO appear healthy. Destroys the SLI's truth value.	Audit SLI query; cross-check with raw request logs; compare SLI with user-visible behaviour.
SLO too loose	Target set well below user expectation; budget never consumed; reliability regressions	Compare SLO with user feedback and complaints; a service with no budget consumption and frequent

	not detected.	complaints has a loose SLO.
SLO too tight	Target set above what the service can achieve; budget always critical; team in permanent freeze.	Sustained Critical state across windows; amendment review triggered.
No error budget policy	SLO measured but no policy attaches to budget state; feature work continues regardless.	Audit whether Critical and Exhausted states trigger freeze; review release records against budget state.
Alerts on raw SLI, not burn rate	Alerts fire on instantaneous SLI dips, producing noise and alert fatigue; sustained regressions missed.	Audit alert rules; alerts SHALL be burn-rate based per §15.4.
SLO without a user journey	SLI measures an internal metric not tied to a user experience; budget consumption does not reflect user impact.	Each SLI SHALL map to a user journey; review SLI definitions against the journey map.
Set-and-forget SLO	SLO defined once and never reviewed; drifts from service reality.	Quarterly amendment review SHALL consider every SLO, even if no change is proposed.

Checklist: SLO Health (Daily)

- ☐ **SH-01** SLO dashboard reachable; all panels rendering.
- ☐ **SH-02** Every SLO's budget state known (healthy/at-risk/critical/exhausted).
- ☐ **SH-03** Burn-rate alerts firing correctly (no false positives or false negatives observed).
- ☐ **SH-04** Daily SLO standup held; minutes recorded.
- ☐ **SH-05** Tickets from burn-rate alerts assigned and progressing.
- ☐ **SH-06** Critical-state SLOs: feature freeze enforced.
- ☐ **SH-07** Exhausted-state SLOs: release freeze enforced; CSA notified.
- ☐ **SH-08** SLA-breached SLOs: SEV-2 declared; counterparty notified.
- ☐ **SH-09** No anti-patterns observed (gaming, no policy, raw-SLI alerts).
- ☐ **SH-10** SLO incident log up to date.

Checklist: SLO Review (Monthly)

- ☐ **SR-01** Past month's SLO performance summarized per SLO.
- ☐ **SR-02** Budget state trajectory reviewed per SLO.

- ☐ **SR-03** Reliability work delivered in the month reviewed.
- ☐ **SR-04** Reliability work planned for the next month reviewed.
- ☐ **SR-05** Amendment candidates identified and prioritized.
- ☐ **SR-06** SLA performance reviewed; breaches documented.
- ☐ **SR-07** Anti-patterns audited; remediation tickets tracked.
- ☐ **SR-08** Burn-rate alert quality reviewed (precision, recall).
- ☐ **SR-09** Daily standup attendance and minutes reviewed.
- ☐ **SR-10** Review minutes recorded in the SLO review log.

15.9 Roles and Ownership

Role	Owns	Authority	Boundaries
On-Call Engineer	Daily SLO dashboard; burn-rate response.	Mitigate; rollback; declare incident on SLA breach.	Cannot amend SLOs; cannot override budget-state policy.
Tech Lead	Team's SLO performance; daily standup.	Pulls reliability tickets forward; halts non-reliability releases in Critical state.	Cannot amend SLOs (CSA); cannot override release freeze.
Release Manager	Release gate vs budget state.	Blocks non-reliability releases in Critical state; blocks all releases in Exhausted state.	Cannot override budget-state policy; cannot amend SLOs.
Engineering Leadership	Monthly SLO review.	Reviews SLO performance; prioritizes reliability work.	Cannot amend SLOs (CSA); cannot override CSA.
Chief Systems Architect	Quarterly SLO amendment.	Approves SLO amendments; directs reliability work in Exhausted state.	Cannot override invariants; cannot suspend SLA obligations without Legal review.
Legal	SLA obligations.	Reviews SLA-affecting amendments; advises on breach communication.	Cannot amend SLOs (CSA); cannot approve releases.

15.10 Evidence and Records

SLO management produces evidence that SHALL be retained for audit, trend analysis, and amendment review. The evidence is the proof that the regime was applied — that the dashboard was read, the alerts were answered, the budget-state policy was enforced, and the amendments were approved. Without this evidence, the SLO regime is unverifiable.

Record	Produced By	Storage	Retention
SLO dashboard snapshots	Observability platform	Observability platform.	90 days hot; 2 years cold.
Daily standup minutes	Tech Lead	SLO review log.	Lifetime + 5 years.
Burn-rate alert records	Alert routing system	Observability platform.	90 days hot; 2 years cold.
SLO incident log entries	On-call engineer	SLO incident log.	Lifetime + 5 years.
Weekly burn review minutes	Team	SLO review log.	Lifetime + 5 years.
Monthly SLO review minutes	Engineering Leadership	SLO review log.	Lifetime + 5 years.
SLO amendment log	Chief Systems Architect	SLO amendment log (versioned).	Lifetime of the platform.
SLA breach records	On-call + Legal	SLA breach log + audit log.	Lifetime of the platform.

Chapter 16 — Incident Command

16.1 Purpose

This chapter extends Chapter 12 with a deeper treatment of incident command. Where Chapter 12 establishes the incident lifecycle (detect, declare, mitigate, resolve, postmortem), this chapter operationalizes the incident-commander role, the war room, communication cadence, resolution criteria, postmortem culture, and the major-incident review. It is the operational form of Volume II Chapter 34 §34.10 (Incident Command).

An incident is a moment of acute operational pressure. The incident-command regime exists so that the pressure produces structured action rather than chaos. The Incident Commander (IC) is the single authority during the incident; the war room is the single venue; the communication cadence is the single rhythm. These three singletons — one IC, one war room, one cadence — are the architecture of incident response, and the Playbook enforces them.

■ **Constitution Reference: Volume II, Chapter 34 §34.10 (Incident Command) — OP-GOV-010 / IC-001 ... IC-005**

16.2 Incident Commander Role

The Incident Commander (IC) is the engineer who holds operational authority for the duration of the incident. The IC is not necessarily the engineer with the deepest subject-matter expertise on the failing system; the IC is the engineer who can hold the whole picture, make decisions under pressure, and coordinate the subject-matter experts (SMEs) who do the deep work. IC is a role, not a rank.

Any on-call engineer who has completed the on-call training (Chapter 13 §13.3) and the incident-command training MAY serve as IC. Incident-command training covers the war-room convention, the communication cadence, the decision log, the escalation matrix, and the postmortem handoff. An IC who has not completed this training SHALL NOT command a SEV-1 incident; they SHALL escalate to a trained IC at the declaration.

IC Authority

- **Direct** — direct every action taken in the war room, including rollback, load-shedding, feature-flag toggles, and break-glass invocations.
- **Convene** — convene and dismiss the war room; summon SMEs, the Release Manager, the RCA, and the Chief Systems Architect.
- **Declare** — declare and stand down the incident; assign severity; approve the communication cadence.
- **Decide** — the IC's decisions during the incident are not subject to consensus; the IC listens, decides, and the war room executes. Disagreements are recorded and reviewed post-incident.

IC Responsibilities

- **Resolve** — drive the incident toward resolution; never lose sight of the user impact.
- **Document** — maintain the decision log; every significant decision is recorded with timestamp and rationale.
- **Communicate** — hold the communication cadence; ensure updates go out on schedule.
- **Preserve** — preserve evidence for the postmortem; capture logs, traces, timelines before they rotate.
- **Transition** — transition the incident to the postmortem owner at stand-down.

IC Handoff

An IC MAY hand off command mid-incident when the incident exceeds the IC's endurance (typically beyond 4 hours) or the IC's expertise (when a deeper SME is better placed to command). Handoff is a structured event: the outgoing IC briefs the incoming IC on the current state, the decisions taken, the options on the table, and the next planned action. The handoff is recorded in the decision log; the war room is notified. An unrecorded handoff is a non-handoff and SHALL be treated as an incident-process failure.

16.3 Incident Severity in Practice

Severity drives the response. A misclassified severity produces either an under-response (a SEV-1 treated as a SEV-3) or an over-response (a SEV-3 treated as a SEV-1, exhausting the team and the war room). The severity table is the operational reference for classification; when in doubt, the IC classifies up, not down, and reclassifies as the picture clarifies.

Severity	Example Scenarios	IC Required	War Room	Comms Cadence	Escalation
SEV-1	Platform-wide outage; scholarly-record corruption; data loss; Crossref deposit broken; DOI resolution broken.	Yes, trained IC.	Convened immediately.	Internal: every 30 min. External/status page: immediate, then every 30 min.	CSA notified immediately; Major Incident Review required.
SEV-2	Single-service outage; SLA breach; significant user-impact degradation (e.g., submission	Yes.	Convened within 15 min.	Internal: every 2 hours. External/status page: if user-visible, within 30 min.	Engineering Leadership notified within 1 hour.

	portal down).				
SEV-3	Degradation with limited user impact; non-critical path broken; workarounds exist.	Optional; on-call may command.	Optional; chat-channel coordination.	Internal: at acknowledgment and at resolution.	Tech Lead notified next business day.
SEV-4	Minor issue; cosmetic; internal-only impact.	No.	No.	Ticket only.	None.

16.4 The War Room

The war room is the single venue in which the incident is commanded. It is not a physical room; it is a virtual venue comprising a video bridge, a chat channel, and an incident document. The three components together form the war room; without all three, the war room is not convened.

The war room is convened at the declaration of a SEV-1 or SEV-2 incident. It is dismissed at stand-down. Between declaration and stand-down, the war room is the canonical source of incident state; no engineer SHALL act on incident information from any other source.

War-Room Tools

- **Video bridge** — always-on video bridge for voice and face. Mute by default; unmute to speak. The bridge is the low-latency channel.
- **Chat channel** — the incident's dedicated channel. All non-verbal communication, decision logs, and links go here. The channel is created at declaration and archived at stand-down.
- **Incident document** — a real-time collaborative document that holds the timeline, the decisions, the action items, and the evidence links. The scribe owns this document.

War-Room Roles

Role	Responsibility	Boundaries
Incident Commander	Holds authority; directs action; decides.	Does not perform hands-on mitigation; delegates to SMEs.
Scribe	Maintains the incident document; records the timeline and decisions.	Does not command; does not perform mitigation.
Comms Lead	Drafts and sends internal and external communications on the	Does not command; drafts approved by IC.

	cadence.	
Subject-Matter Expert	Performs hands-on mitigation; advises the IC.	Does not command; executes the IC's direction.
Observer	Watches silently; learns; flags concerns to the scribe.	Does not speak on the bridge; does not act.

16.5 Communication During an Incident

Communication during an incident has two audiences: internal (the war room, the engineering organization, the leadership) and external (the status page, the affected users, the stakeholders). The two audiences have different cadences, different content, and different tones, but they share a single principle: communication **SHALL** be honest, timely, and specific. An incident that is silently handled is an incident that the user experiences as a black box; the resulting trust deficit outlasts the incident.

Internal communication cadence is driven by severity. For SEV-1, the war room posts an update every 30 minutes, even if the update is 'still investigating, current mitigation in effect, next checkpoint at HH:MM'. For SEV-2, the cadence is every 2 hours. For SEV-3 and below, the cadence is at acknowledgement and at resolution. The cadence is the rhythm that the rest of the organization can rely on; missing a checkpoint without explanation is itself a process failure.

External communication — the status page — is the platform's public face during the incident. The first external update **SHALL** go out within 30 minutes of a SEV-1 declaration, within 60 minutes of a SEV-2 declaration if user-visible. The update **SHALL** state: what is affected, what the user is experiencing, what the platform is doing, and when the next update will come. Speculation about root cause is forbidden in external updates until the root cause is confirmed; speculation creates liability and erodes trust.

Stakeholder notifications — to the editorial board, to Crossref, to preservation partners — are sent when the incident affects the stakeholder's interest. The Comms Lead drafts; the IC approves; the notification is logged. Stakeholders **SHALL NOT** learn of an incident from a public status page if they have a direct notification relationship with the platform.

16.6 Incident Resolution

An incident is resolved when its user impact has ended and the system has returned to a steady state that the SLO dashboard and the invariant monitors confirm. Resolution is not the absence of pages; it is the verified restoration of service. The IC **SHALL NOT** declare resolution without explicit verification; declaring resolution prematurely produces incident churn (a re-declaration within the hour) that erodes trust in the regime.

Resolution Criterion	How Verified
User impact ended	SLO dashboard shows SLI above target; burn rate

	below 1x for the observation window.
Invariant monitors green	Every invariant in Vol II Ch 26 reads true; no invariant violations in the last 30 minutes.
Alerts quiet	No pages in the last 30 minutes that are attributable to the incident.
Mitigation stable	The mitigation in effect has held for at least one observation window (typically 30 minutes).
Evidence preserved	Logs, traces, and timeline captured for the postmortem.

On verification, the IC declares stand-down. The war room is dismissed; the chat channel is archived; the incident document is finalized. The IC transitions the incident to the postmortem owner — typically the Tech Lead of the owning team — who SHALL convene the blameless postmortem within 5 business days (SEV-1) or 10 business days (SEV-2).

16.7 Postmortem Culture

A postmortem is the structured learning that follows an incident. The postmortem asks: what happened, why did it happen, what did we do about it, what should we do to prevent recurrence, and what did we learn. The postmortem is blameless: it addresses systems and processes, not individuals. The blameless principle is not soft; it is rigorous. A postmortem that concludes 'engineer X made a mistake' has stopped at the surface; the rigorous postmortem asks why the system permitted the mistake, why the review did not catch it, why the tooling did not prevent it.

Psychological safety is the precondition of a blameless postmortem. An engineer who fears punishment will hide the mistake; an engineer who trusts the process will describe it in detail. The Playbook's commitment to psychological safety is operational: no postmortem SHALL name an individual as the cause; no performance review SHALL cite a postmortem as evidence of individual fault; no disciplinary action SHALL follow from a postmortem finding except in the case of demonstrated malice, which is a separate matter handled outside the postmortem regime.

The learning orientation of the postmortem is measured by action-item follow-through. A postmortem without action items is a narrative; a postmortem with action items that are never executed is a ritual. The Playbook requires that every postmortem produce action items, that every action item have an owner and a due date, and that the Tech Lead SHALL track action items to completion. Action items past due are escalated to Engineering Leadership; action items chronically past due are an SLO risk in their own right.

16.8 Major Incident Review

A SEV-1 incident triggers a Major Incident Review (MIR) in addition to the blameless postmortem. The MIR is a formal review with the Chief Systems Architect, the Engineering Leadership Team, and the owning team. The MIR's purpose is to examine the systemic factors that allowed the incident to occur and to authorize the corrective actions, including any that require cross-team coordination, architectural change, or Constitution amendment.

The MIR SHALL be convened within 10 business days of the incident. The MIR agenda covers: incident timeline (from the postmortem), root cause, systemic factors (process, tooling, organizational), corrective actions (with owners and dates), and follow-up review date. The MIR's decisions are recorded in the MIR log; the CSA signs the record. A MIR that does not produce systemic corrective actions has not completed its purpose; another MIR SHALL be convened.

Runbook: SEV-1 Major Incident

Detect

Detection is by alert (burn-rate page, invariant monitor, synthetic probe), by user report (support channel, status page), or by internal observation. The detector — alert or human — declares the incident and pages the on-call. Detection-to-page SHALL be under 5 minutes for alert-based detection.

Declare

The on-call acknowledges, classifies as SEV-1, and declares the incident. Declaration opens the incident record, the war-room chat channel, and the incident document. The on-call assumes IC role (or pages a trained IC if not trained).

Convene

The IC convenes the war room: opens the video bridge, invites the SMEs, the Comms Lead, the scribe, and the observers. The CSA is notified immediately. The war room is convened within 15 minutes of declaration.

Mitigate

The IC directs mitigation. The default first mitigation is rollback to the last certified release; if rollback is not applicable, the IC directs load-shedding, feature-flag toggle, or break-glass as the situation demands. Mitigation precedes root cause.

Communicate

The Comms Lead posts the first external status-page update within 30 minutes. Internal updates go out every 30 minutes. Stakeholder notifications are sent as the incident's impact becomes clear. All communications are IC-approved.

Resolve

The IC verifies resolution against the criteria in §16.6. On verification, the IC declares stand-down, dismisses the war room, and transitions to postmortem.

Postmortem

The postmortem owner convenes the blameless postmortem within 5 business days. The postmortem produces the timeline, root cause, action items, and learning. The postmortem is published to the engineering organization.

Review

The MIR is convened within 10 business days with the CSA. The MIR authorizes systemic corrective actions and records them in the MIR log. The CSA signs the MIR record.

Procedure: Declaring an Incident

Step 1. Acknowledge and classify The on-call acknowledges the alert or report, reads the payload, and classifies the severity. When in doubt, classify up.

Step 2. Open the incident record Open the incident record in the incident management system. The record SHALL capture: detector, severity, affected service, initial user impact, IC, time of declaration.

Step 3. Assume or hand off IC role If the on-call is trained, assume IC. If not, page a trained IC. The IC's name SHALL be recorded in the incident record.

Step 4. Notify per severity For SEV-1, notify the CSA immediately. For SEV-2, notify Engineering Leadership within 1 hour. For SEV-3, notify the Tech Lead next business day.

Step 5. Convene the war room (SEV-1, SEV-2) Open the video bridge, the chat channel, and the incident document. Invite the SMEs, the Comms Lead, the scribe.

Step 6. Post the first internal update Within 15 minutes of declaration, post the first internal update to the incident channel: what is known, what is being done, next checkpoint.

Step 7. Begin the decision log The scribe begins the decision log. Every significant decision SHALL be recorded with timestamp, decider, rationale, and effect.

Procedure: Convening a War Room

Step 1. Open the video bridge Open the dedicated war-room video bridge. The bridge SHALL be a persistent link, known to every on-call, accessible without authentication friction.

Step 2. Create the incident chat channel Create the incident channel with a standard name (e.g., #inc-YYYYMMDD-slug). Pin the incident document, the video bridge link, and the status-page link.

Step 3. Open the incident document Open the incident document from the template. The template SHALL include: timeline, decisions, action items, evidence links, communication log, participants.

Step 4. Invite the participants Invite the SMEs (per the affected service), the Comms Lead, the scribe, and (for SEV-1) the CSA. Observers may join silently.

Step 5. Brief the participants The IC briefs the participants on the current state, the severity, the mitigation in progress, and the immediate next steps. The scribe records the briefing in the incident document.

Step 6. Confirm the cadence The IC confirms the communication cadence (30 min for SEV-1, 2 hours for SEV-2) and the next checkpoint time. The Comms Lead schedules the cadence.

Procedure: Conducting a Blameless Postmortem

Step 1. Schedule the postmortem The postmortem owner schedules the postmortem within 5 business days (SEV-1) or 10 business days (SEV-2). All participants in the incident are invited; the meeting is open to the engineering organization.

Step 2. Assemble the timeline The scribe's incident document is the timeline source. The postmortem owner reconciles the timeline with logs, alerts, and chat records. The timeline SHALL be complete from detection to stand-down.

Step 3. Identify the root cause The postmortem identifies the root cause through structured analysis (5 Whys, fishbone, or equivalent). The root cause SHALL address systems and processes, not individuals. 'Human error' is never an acceptable root cause; the acceptable root cause addresses why the system permitted the error.

Step 4. Identify systemic factors The postmortem identifies the systemic factors that allowed the incident: process gaps, tooling gaps, organizational gaps, architectural gaps. Each factor SHALL be addressed by an action item.

Step 5. Draft action items Each action item SHALL have: description, owner, due date, success criterion. Action items SHALL be specific and verifiable; 'improve monitoring' is not an action item.

Step 6. Publish the postmortem The postmortem is published to the engineering organization within 2 business days of the meeting. Publication is blameless; the postmortem SHALL NOT name individuals as causes.

Step 7. Track action items to completion The Tech Lead SHALL track every action item to completion. Action items past due are escalated to Engineering Leadership. The action-item tracker is the evidence that the postmortem produced change.

Checklist: Incident Commander Readiness

- ☐ **IC-01** On-call training completed (Ch 13 §13.3).
- ☐ **IC-02** Incident-command training completed; war-room convention, cadence, decision log, escalation matrix all covered.
- ☐ **IC-03** Shadowed at least one SEV-2 incident as observer.
- ☐ **IC-04** Authority matrix (Ch 14 §14.2) memorized; knows what the IC MAY and MAY NOT do.
- ☐ **IC-05** Break-glass procedure (Ch 14 §14.5) known; can execute under pressure.
- ☐ **IC-06** War-room tools reachable: video bridge, chat, incident document template.
- ☐ **IC-07** Severity table (Ch 16 §16.3) memorized; can classify under pressure.
- ☐ **IC-08** Communication cadence known for SEV-1, SEV-2, SEV-3.
- ☐ **IC-09** Knows the escalation path: SME, Release Manager, RCA, CSA.

☐ **IC-10** Postmortem handoff procedure known; can transition the incident to the postmortem owner.

☐ **IC-11** Decision-log discipline practiced; can record decisions in real time (via scribe).

☐ **IC-12** Personal readiness: rested, not on a consecutive second shift, no PTO conflict.

16.9 Roles and Ownership

Role	Owns	Authority	Boundaries
Incident Commander	Incident response end-to-end.	Directs war room; declares and stands down; approves communications; authorizes break-glass.	Cannot perform hands-on mitigation (delegates); cannot change Constitution; cannot skip RCA on release.
Scribe	Incident document; decision log; timeline.	Records the canonical incident record.	Does not command; does not perform mitigation.
Comms Lead	Internal and external communications.	Drafts and sends communications on cadence.	Drafts IC-approved; does not command.
Subject-Matter Expert	Hands-on mitigation.	Executes the IC's direction; advises on technical options.	Does not command; does not declare stand-down.
Postmortem Owner	Postmortem process.	Convenes postmortem; drafts action items; publishes.	Cannot name individuals as causes; cannot approve Constitution amendments.
Tech Lead	Action-item follow-through.	Tracks action items; escalates past-due items.	Cannot override IC during incident; cannot suspend postmortem.
Chief Systems Architect	Major Incident Review.	Convenes MIR; authorizes systemic corrective actions; signs MIR record.	Cannot override invariants; cannot name individuals as causes.

16.10 Evidence and Records

Incident command produces evidence that SHALL be retained for audit, regulatory review, and organizational learning. The evidence is the proof that the incident was declared, commanded, mitigated, resolved, and postmortemed. The evidence is also the raw material of organizational learning; patterns across incidents reveal systemic gaps that no single postmortem can see.

Record	Produced By	Storage	Retention
Incident record	On-call / IC	Incident management system.	Lifetime + 5 years.
Decision log	Scribe	Incident document (archived).	Lifetime + 5 years.
War-room chat transcript	Chat system	Chat archive.	Lifetime + 5 years.
Communication log (internal + external)	Comms Lead	Incident document + status-page archive.	Lifetime + 5 years.
Postmortem document	Postmortem owner	Postmortem repository.	Lifetime of the platform.
Action-item tracker	Tech Lead	Action-item tracker (versioned).	Lifetime + 5 years.
Major Incident Review record	CSA	MIR log (signed).	Lifetime of the platform.
Alert and acknowledgement records	Alert routing system	Observability platform.	90 days hot; 2 years cold.

Chapter 17 — Volume I Operationalization Matrix

17.1 Purpose

This chapter is the traceability bridge between Volume I of the Engineering Constitution (Chapters 1-25) and this Engineering Playbook. Every Constitution chapter establishes normative requirements, contracts, and rules; every such requirement is operationalized by one or more Playbook chapters in the form of a procedure, a checklist, a runbook, or a gate. The matrix below is the authoritative map of that operationalization. Where the Constitution says what **MUST** hold, the matrix identifies the Playbook chapter that enforces it and the operational form (clause selection, review checklist item, certification pillar, runbook, SLO dashboard, or governance process) by which the rule is enforced at the moment of implementation.

The matrix is normative. An engineer **SHALL** consult this matrix when implementing a feature governed by a Constitution chapter: the matrix identifies the Playbook chapter that contains the applicable checklist or procedure. A reviewer **SHALL** consult this matrix when reviewing a PR that touches a Constitution chapter: the matrix identifies the review checklist items that **MUST** be satisfied. The Release Certification Authority (RCA) **SHALL** consult this matrix when certifying a release: the matrix identifies the certification pillars and the operational evidence that **MUST** be produced.

Constitution Reference: *Volume I, Preamble §2 (The Constitutional Hierarchy); Volume I, Chapter 24 (Documentation Constitution — cross-reference requirement)*

This matrix exists because operationalization is not a side effect of writing a Constitution; it is a discipline that **MUST** be explicit, auditable, and maintained. A constitutional rule that does not appear in this matrix is, by definition, un-operationalized; such a rule **SHALL** be flagged as a Playbook gap and tracked as Playbook automation debt until the gap is closed by a Playbook amendment.

17.2 The Matrix

The matrix lists every Volume I Constitution chapter (1-25), the key rules or requirements it establishes, the Playbook chapter(s) that operationalize those rules, the operational form the operationalization takes, and the role accountable for enforcement. The columns are:

- **Constitution Chapter | Title** — the Constitution chapter number and its title.
- **Key Rules/Requirements** — the principal rules, requirements, contracts, or principles established by the chapter (non-exhaustive; the Constitution remains the authoritative source).
- **Operationalizing Playbook Chapter(s)** — the Playbook chapter(s) that contain the procedure, checklist, runbook, or gate that enforces the Constitution chapter.
- **Operational Form** — the concrete operational mechanism (e.g., a checklist item, a certification pillar, a runbook, an SLO dashboard) by which the Constitution rule is enforced at implementation time.
- **Owner** — the role accountable for ensuring the operational form is applied and verified.

The matrix SHALL be read alongside Chapter 18 (Volume II Operationalization Matrix). Together the two matrices provide a complete Constitution-to-Playbook cross-reference; Appendix B consolidates the two into a single table.

Constitution Chapter Title	Key Rules/Requirements	Operationalizing Playbook Chapter(s)	Operational Form	Owner
1 Product Mission	Mission, scope, non-goals; the platform is a multi-tenant scholarly-publishing system; no editorial decisions.	PB Ch 2 (lifecycle)	Clause selection in task tickets; mission/non-goal check in review.	Tech Lead
2 Engineering Philosophy	Twelve engineering principles; no fallback hacks; single implementation; you build it, you run it.	PB Ch 5 (review); PB Ch 13 (on-call)	Review checklist enforces the twelve principles; on-call ownership enforces Principle 10.	Reviewer; On-Call
3 System Context	System boundaries; external integrations (Crossref, DOI, OAI-PMH, ORCID); no scope creep across boundaries.	PB Ch 14 (ops governance)	Boundary enforcement in ADRs; integration scope check at review.	Tech Lead
4 System Architecture	Layered architecture; service boundaries; dependency direction; deployment topology.	PB Ch 2, PB Ch 11 (release/deploy)	ADRs required for architectural decisions; deployment pipeline enforces topology.	Architect; Release Manager
5 Data Constitution	Data ownership, lineage, retention, PII handling; canonical schemas; data classification.	PB Ch 6 (QA), PB Ch 9 (certification)	Test data procedures; certification database verification captures before/after persisted state.	QA; RCA
6 Canonical Registry	Single canonical registry of object types and field names; no duplicate or shadow definitions.	PB Ch 9 (certification)	Canonical field lookup executed during certification; new fields rejected if absent from registry.	RCA
7 ADRs	Every architectural decision recorded; ADR template; ADR lifecycle (proposed/accepted/deprecated/superseded).	PB Ch 2 (lifecycle), PB Ch 5 (review)	ADR required in PR for architectural decisions; PR template prompts for ADR link.	Reviewer; Architect

8 Repository Constitution	Trunk-based development; branch naming; CODEOWNERS; protected branches; squash merges.	PB Ch 3 (branching)	Branching model enforced by CODEOWNERS and branch protection; merge strategy enforced by repo policy.	Tech Lead
9 API Constitution	REST/JSON contracts; idempotency keys; ETags; versioning; pagination; error envelope.	PB Ch 5 (review), PB Ch 10 (five pillars)	API spec verified at review; API pillar in certification captures request/response and contract tests.	Reviewer; RCA
10 Workflow Constitution	State machines for editorial workflows; transitions are transactional; invariants protected.	PB Ch 6 (QA), PB Ch 8 (invariants)	Workflow tests assert legal transitions; invariant monitors detect illegal state changes.	QA; On-Call
11 Publication State Machine	Publication lifecycle (draft/submitted/under-review/.../published/retracted); invariant INV-C-07.	PB Ch 8 (invariants)	Invariant INV-C-07 monitor alerts on illegal publication transitions; runbook RB-INV-001 invoked.	On-Call
12 Storage Constitution	Content-addressed storage; SHA-256 hashing; immutable blobs; replicated durability.	PB Ch 10 (five pillars)	Storage pillar in certification verifies SHA-256, immutability, and retrieval.	RCA
13 Rendering Constitution	Deterministic rendering; same input yields same output; visual regression tests.	PB Ch 10 (five pillars)	Visual pillar in certification captures screenshots/recordings of every state.	QA; RCA
14 Metadata Constitution	Metadata schema; Crossref/Dublin Core/FAIR compliance; credential management for external APIs.	PB Ch 10 (five pillars), PB Ch 14 (ops governance)	API pillar captures metadata contracts; credential rotation runbook RB-OPS-001 enforces key lifecycle.	RCA; On-Call
15 Template Constitution	Template schema; variable substitution rules; template versioning; rendering isolation.	PB Ch 6 (QA)	Template tests assert substitution correctness and isolation boundary.	QA
16 Security Constitution	Threat model; authentication;	PB Ch 5 (review), PB Ch 14 (ops)	Security review checklist at PR;	Reviewer; On-Call

	authorization; secrets; audit logs; break-glass.	governance)	credential rotation, break-glass runbook RB-OPS-004 enforce ops.	
17 Performance Constitution	Performance budgets; latency targets; capacity ceilings; no unbounded queries.	PB Ch 6 (QA), PB Ch 15 (SLO management)	Performance tests assert budgets; SLO dashboard tracks latency and capacity.	QA; On-Call
18 Observability Constitution	Structured logs; metrics; traces; invariant dashboard; retention; correlation IDs.	PB Ch 8 (invariants), PB Ch 15 (SLO management)	Invariant dashboard surfaces runtime invariants; SLO dashboard surfaces SLI health.	On-Call
19 Testing Constitution	Test pyramid; coverage thresholds; test data; environments; flaky-test policy.	PB Ch 6 (QA), PB Ch 7 (environments)	Test pyramid enforced in CI; environments gated per Ch 7; flaky tests handled per RB-QA-001.	QA
20 Runtime Verification Constitution	Five pillars of runtime verification; objective evidence; no static-only certification.	PB Ch 8 (invariants), PB Ch 10 (five pillars)	Invariant monitors (Ch 8) and five-pillar certification (Ch 10) enforce runtime verification.	RCA; On-Call
21 Failure Constitution	Failure modes; rollback; blast-radius containment; incident command; postmortem.	PB Ch 12 (rollback), PB Ch 16 (incident command)	Rollback procedure (Ch 12); incident command system (Ch 16); postmortem template.	On-Call; IC
22 Certification Constitution	Certification process; five pillars; RCA role; artifact set; no override of certification failure.	PB Ch 9 (certification)	Certification process executed end-to-end; RCA sign-off required for release.	RCA
23 Knowledge Graph Constitution	Requirement nodes; implementation traces; traceability from code to Constitution clause.	PB Ch 2 (lifecycle)	Clause selection at task creation records the requirement node ID; traceability enforced in PR.	Tech Lead
24 Documentation Constitution	Eight mandatory document fields; eleven document classes; documentation lifecycle; drift	PB Ch 5 (review)	Docs review in PR; PR template requires doc updates when behaviour changes.	Reviewer

	detection.			
25 Future Evolution	RC1/RC2/RC3 roadmap; evolution governance; deprecation; forward compatibility.	PB Ch 14 (ops governance)	Evolution governance execution: amendment process, deprecation runbooks, compatibility checks.	Architect

Reading the Matrix

An engineer implementing a Crossref deposit feature governed by Volume I Chapter 14 (Metadata Constitution) reads the matrix and finds: operationalized by PB Ch 10 (five pillars) and PB Ch 14 (ops governance); operational form is the API pillar in certification plus credential rotation runbook RB-OPS-001; owners are RCA and On-Call. The engineer then consults PB Ch 10 §10.5 (API pillar) and PB Ch 14 §14.7 (credential rotation) for the procedures and checklists. The matrix is the index; the cited Playbook chapters are the operating instructions.

17.3 Using the Matrix

The matrix is consulted at four moments in the implementation lifecycle, by four different roles, for four different purposes. The procedure below is the canonical way to use the matrix; deviations from this procedure (e.g., consulting the Playbook without first locating the Constitution chapter) are a common source of incomplete operationalization and SHALL be avoided.

Procedure: Using the Volume I Operationalization Matrix

Step 1. Identify the Constitution chapter that governs the work Locate the chapter in Volume I (Chapters 1-25) whose domain contains the feature, fix, or operational change being implemented. If multiple chapters apply, list all of them.

Step 2. Locate the chapter row in the matrix Find the row whose 'Constitution Chapter | Title' matches the chapter identified in Step 1. If the row is missing, the matrix is out of date; raise a Playbook amendment request per §17.4.

Step 3. Record the operationalizing Playbook chapter(s) The 'Operationalizing Playbook Chapter(s)' column lists the Playbook chapters that contain the applicable procedures, checklists, runbooks, or gates. Record these in the task ticket.

Step 4. Identify the operational form The 'Operational Form' column names the concrete mechanism (checklist item, certification pillar, runbook ID, SLO dashboard, governance process) by which the Constitution rule is enforced. Record the mechanism; it is the artifact the reviewer and RCA will look for.

Step 5. Confirm the owner The 'Owner' column identifies the role accountable for the operational form. If the owner role is not staffed on the current team, escalate to the Engineering Leadership Team before proceeding.

Step 6. Execute the cited Playbook chapter(s) Open each cited Playbook chapter and execute its procedure, satisfy its checklist, and prepare its evidence. The matrix is the index; the Playbook chapter is the operating manual.

Step 7. Cite the matrix in the PR and release record The PR description SHALL cite the Constitution chapter and the Playbook chapter(s) that operationalize it. The release record SHALL cite the same. This is the operational form of traceability (Volume II Chapter 23).

Checklist: Matrix Consultation

- ☐ **MX-01** Constitution chapter(s) identified and cited.
- ☐ **MX-02** Matrix row(s) located for each cited chapter.
- ☐ **MX-03** Operationalizing Playbook chapter(s) recorded in ticket.
- ☐ **MX-04** Operational form(s) recorded (checklist IDs, pillar, runbook).
- ☐ **MX-05** Owner role confirmed or escalation raised.
- ☐ **MX-06** Cited Playbook chapter(s) executed.
- ☐ **MX-07** Matrix citation present in PR description and release record.

17.4 Matrix Maintenance

The matrix is a living artifact. It SHALL be amended whenever the Constitution is amended, whenever the Playbook is amended, and whenever a gap is discovered (a Constitution chapter that is not operationalized, or a Playbook chapter that operationalizes a Constitution chapter not listed in the matrix). Matrix maintenance is governed by the Evolution Governance process (Volume II Chapter 36) and is the responsibility of the Chief Systems Architect with the Engineering Leadership Team.

The following triggers require matrix amendment:

- **Constitution amendment.** a new Constitution chapter is added (Volume I or Volume II), or an existing chapter is renumbered, retitled, or removed. The matrix SHALL be updated in the same amendment cycle.
- **Playbook amendment.** a new Playbook chapter is added, or an existing chapter is renumbered or removed. The matrix SHALL be updated to reflect the new operationalization map.
- **Gap discovery.** a reviewer, RCA, or on-call engineer discovers a Constitution chapter whose operationalization is incomplete, ambiguous, or absent from the matrix. The gap SHALL be logged as Playbook automation debt and closed by a matrix amendment.
- **Operational form change.** an operational form changes (e.g., a checklist item is automated, a runbook is renamed, a certification pillar is split). The 'Operational Form' column SHALL be updated.
- **Owner change.** an owner role is renamed, split, or consolidated. The 'Owner' column SHALL be updated to reflect the new role.

Matrix amendments follow the standard amendment lifecycle (Volume II Chapter 36 §36.4): proposal, review, discussion, voting, approval, implementation, and certification. A matrix amendment SHALL NOT

be applied ad hoc; it SHALL pass through governance like any other Playbook change. The matrix version SHALL be incremented on each amendment, and the revision history SHALL be recorded in Appendix D (Document Control).

The matrix is reviewed on a monthly cadence (the Playbook review cadence) and on each Constitution amendment. The Chief Systems Architect is the authority of last resort for matrix disputes; however, the Chief Systems Architect SHALL NOT unilaterally remove a row from the matrix without an accompanying Constitution amendment, because the matrix is a derivative artifact of the Constitution and the Constitution remains supreme.

Chapter 18 — Volume II Operationalization Matrix

18.1 Purpose

This chapter is the traceability bridge between Volume II of the Engineering Constitution (Chapters 26-36, the Eleven Missing Chapters) and this Engineering Playbook. Volume II was authored to close the eleven gaps identified in the post-Volume-I review: invariants, system contracts, the canonical object registry, events, UI, engineering mathematics, the SLO book, the database constitution, operational governance, engineering economics, and evolution governance. Each of these chapters establishes normative rules that the Playbook operationalizes.

The structure of this chapter mirrors Chapter 17. The matrix is the authoritative map of Volume II operationalization. Where the Constitution says what **MUST** hold (an invariant, a contract, an SLO, a governance authority), the matrix identifies the Playbook chapter that enforces it and the operational form by which the rule is enforced. The matrix is normative and **SHALL** be maintained in lockstep with Volume II.

Constitution Reference: Volume II, Preamble (The Eleven Missing Chapters); Volume II, Chapter 36 (Evolution Governance)

Volume II is heavy on invariants, contracts, and quantitative thresholds (engineering mathematics, SLOs, error budgets, burn rates). Operationalization of these rules is therefore concentrated in the invariant monitors (PB Ch 8), the five-pillar certification (PB Ch 10), the SLO management regime (PB Ch 15), and the operational governance execution (PB Ch 14). The matrix is the engineer's index into those operational chapters.

18.2 The Matrix

The matrix lists every Volume II Constitution chapter (26-36), the key invariants or rules it establishes, the Playbook chapter(s) that operationalize those rules, the operational form the operationalization takes, and the role accountable for enforcement. The columns are identical to Chapter 17; together the two matrices constitute the complete Constitution-to-Playbook cross-reference consolidated in Appendix B.

Constitution Chapter Title	Key Invariants/Rules	Operationalizing Playbook Chapter(s)	Operational Form	Owner
26 Engineering Invariants	The catalogue of engineering invariants (INV-I-* infrastructure, INV-C-* consistency, INV-S-* safety); every invariant has a monitor, a runbook, and an SLO mapping.	PB Ch 8 (invariants)	Invariant dashboard surfaces all invariants; per-invariant monitors alert on violation; runbooks RB-INV-001/002 invoked.	On-Call

27 System Contracts	Explicit input/output contracts for every service; contract tests; no implicit contracts; versioning and compatibility.	PB Ch 5 (review), PB Ch 10 (five pillars)	Contract verification at review (reviewer checks contract tests) and at certification (API pillar runs contract suite).	Reviewer; RCA
28 Canonical Object Registry	Single registry of object types, field names, and IDs; no duplicate types; lookup-by-ID is the only access pattern.	PB Ch 9 (certification)	Registry lookup executed during certification; new object types or fields rejected if absent from registry.	RCA
29 Event Constitution	Event schema; idempotent consumers; outbox pattern; event ordering; at-least-once delivery.	PB Ch 6 (QA), PB Ch 10 (five pillars)	Event tests assert schema and idempotency; API pillar captures event contracts in certification.	QA; RCA
30 UI Constitution	Component library; visual regression tests; accessibility; no inline styling; deterministic rendering.	PB Ch 6 (QA), PB Ch 10 (five pillars)	Visual regression tests in CI; visual pillar in certification captures screenshots of every state.	QA; RCA
31 Engineering Mathematics	Quantitative thresholds (latency, throughput, error rate, saturation); numerical bounds for SLOs and capacity.	PB Ch 9 (certification), PB Ch 15 (SLO management)	Threshold verification in certification (metrics within bounds); SLO management enforces numerical targets.	RCA; On-Call
32 Production SLO Book	SLI/SLO/SLA catalogue; error budget policy; burn-rate alerting; multi-window multi-burn-rate pattern.	PB Ch 15 (SLO management)	SLO dashboard surfaces SLI health; error budget exhaustion triggers feature freeze; burn-rate alerts page on-call.	On-Call
33 Database Constitution	Schema migrations; transactional integrity; indexing; backup/restore; read/write separation; no ad-hoc SQL in services.	PB Ch 6 (QA), PB Ch 14 (ops governance)	Database tests assert schema and integrity; migration execution runbook RB-OPS-003 governs schema changes.	QA; On-Call
34 Operational Governance	Authority matrix; separation of duties;	PB Ch 14 (ops governance)	Authority matrix executed at every	On-Call; Architect

	approval thresholds; break-glass; emergency powers.		approval point; break-glass runbook RB-OPS-004 governs emergency access.	
35 Engineering Economics	Cost-per-request budgets; resource ceilings; cost review at SLO review; no unbounded cost growth.	PB Ch 15 (SLO management)	Cost review conducted during SLO review; cost-per-request tracked on the SLO dashboard; over-budget features flagged.	On-Call; Architect
36 Evolution Governance	Amendment lifecycle; voting; supermajority for invariants; precedent register; emergency amendments.	PB Ch 14 (ops governance)	Amendment process executed end-to-end; precedent register maintained; emergency amendments tracked for post-hoc review.	Architect

Reading the Matrix

An engineer investigating an alert from the INV-C-07 monitor (illegal publication-state transition) reads the matrix and finds: Volume II Chapter 26 (Engineering Invariants) is operationalized by PB Ch 8 (invariants); operational form is the invariant dashboard plus the per-invariant monitor and runbook; owner is On-Call. The engineer then consults PB Ch 8 §8.4 (per-invariant runbooks) and invokes RB-INV-001 (Invariant Violation SEV-1). The matrix is the index; the cited Playbook chapter and runbook are the response.

18.3 Using the Matrix

The procedure for using the Volume II matrix is identical to the procedure for using the Volume I matrix (§17.3). The engineer identifies the governing Constitution chapter, locates the matrix row, records the operationalizing Playbook chapter(s) and operational form, confirms the owner, executes the cited Playbook chapter, and cites the matrix in the PR and release record.

Two Volume II-specific considerations apply. First, Volume II is invariant-heavy: when a Constitution chapter cites an invariant (INV-I-*, INV-C-*, INV-S-*), the engineer SHALL additionally consult PB Ch 8 (Invariants) and the invariant dashboard; the operational form is the per-invariant monitor and runbook, not merely a checklist item. Second, Volume II is threshold-heavy: when a Constitution chapter cites a numerical threshold (latency bound, error rate, burn rate), the engineer SHALL additionally consult PB Ch 15 (SLO Management) and the SLO dashboard; the operational form is the SLO and its burn-rate alerting, not merely a static assertion.

Procedure: Using the Volume II Operationalization Matrix

Step 1. Identify the Volume II chapter that governs the work Locate the chapter in Volume II (Chapters 26-36) whose domain contains the work. If a Volume I chapter also applies, consult both matrices.

Step 2. Locate the chapter row in the matrix Find the row whose 'Constitution Chapter | Title' matches the Volume II chapter identified in Step 1. If missing, raise a Playbook amendment request per §18.4.

Step 3. Record the operationalizing Playbook chapter(s) and form Record the Playbook chapter(s) and the operational form. If the chapter cites an invariant, record the invariant ID and its runbook. If the chapter cites a threshold, record the SLO ID and its burn-rate alert.

Step 4. Confirm the owner Confirm the owner role is staffed. Volume II owners are frequently On-Call or Architect; if the team does not staff these roles, escalate before proceeding.

Step 5. Execute the cited Playbook chapter(s) Open each cited Playbook chapter and execute its procedure. If an invariant is involved, verify the monitor is green before and after the change. If an SLO is involved, verify the error budget is healthy before and after.

Step 6. Cite the matrix in the PR and release record The PR description and release record SHALL cite the Volume II chapter and the Playbook chapter(s) that operationalize it, including any invariant IDs and SLO IDs.

Checklist: Volume II Matrix Consultation

- ☐ **MX-08** Volume II Constitution chapter(s) identified and cited.
- ☐ **MX-09** Matrix row(s) located for each cited chapter.
- ☐ **MX-10** Operationalizing Playbook chapter(s) recorded in ticket.
- ☐ **MX-11** Operational form(s) recorded (invariant ID, runbook, SLO ID, burn-rate alert).
- ☐ **MX-12** Owner role confirmed or escalation raised.
- ☐ **MX-13** Invariant monitors verified green before and after change (if applicable).
- ☐ **MX-14** SLO error budget verified healthy before and after change (if applicable).
- ☐ **MX-15** Matrix citation present in PR description and release record.

18.4 Matrix Maintenance

The Volume II matrix is maintained in lockstep with the Volume I matrix. The triggers and amendment process are identical (§17.4 applies mutatis mutandis). The Chief Systems Architect is the authority of last resort for matrix disputes; the Chief Systems Architect SHALL NOT unilaterally remove a Volume II row from the matrix without an accompanying Constitution amendment.

Three Volume II-specific maintenance considerations apply. First, invariant amendments require an 80% supermajority and evidence-of-strength (Volume II Chapter 36 §36.13); a matrix amendment that adds, removes, or rewrites an invariant row SHALL follow the same supermajority requirement. Second, SLO amendments follow the SLO review cadence (Volume II Chapter 32 §32.10); a matrix amendment that

changes an SLO row SHALL be synchronized with the SLO review cycle. Third, evolution-governance amendments (Chapter 36) are self-referential: a matrix amendment that changes the Chapter 36 row SHALL itself follow the amended process, with the previous process as the baseline.

The matrix version is incremented on each amendment. The combined Volume I + Volume II matrix is consolidated in Appendix B (Constitution-to-Playbook Cross-Reference); the consolidated table is the single source of truth for traceability audits and SHALL be used in preference to the per-volume matrices when a complete view is required.

Chapter 19 — The Master Checklist Catalog

19.1 Purpose

This chapter collects every checklist referenced in the Playbook into a single catalog for easy reference. The catalog is the engineer's one-stop reference during implementation, review, certification, release, deployment, rollback, incident response, and on-call readiness. Each checklist is presented in its complete form; the cross-references to the originating Playbook chapters are provided so the engineer can consult the surrounding procedure and context.

Constitution Reference: Volume I, Chapter 22 (Certification Constitution); Volume II, Chapter 20 (Runtime Verification Constitution)

The checklists in this catalog are normative. An item marked satisfied SHALL have evidence recorded (a CI run link, a screenshot, a log excerpt, a sign-off). An item marked N/A SHALL have a written justification. An item left blank is a gap; the gate SHALL NOT pass with gaps. The Production Acceptance Gate (§19.10) is the final gate that combines the certification, release, and deployment checklists; it is the ultimate gate that blocks production release.

Each checklist is identified by a stable prefix (IM for Implementation, RV for Review, CT for Certification, RL for Release, DP for Deployment, RB for Rollback, IR for Incident Response, OC for On-Call Readiness). The prefixes are stable across Playbook versions; the item numbers are stable within a version. When an item is amended, the item number is retired (not reused) and a new number is issued, so that historical evidence remains interpretable.

19.2 Implementation Checklist

The Implementation Checklist is applied by the engineer during the Implementation stage (PB Ch 2 §2.5) and verified by the reviewer at the start of the Review stage (PB Ch 2 §2.6). It combines items from PB Ch 2 (IM-01..07), PB Ch 3 (branching), PB Ch 4 (commits and PRs). The checklist is a GATE to opening a pull request; a PR SHALL NOT be opened until every item is satisfied.

Implementation Checklist (GATE to PR)

- ☐ **IM-01** Branch created per the branching model (PB Ch 3); branch name encodes task ID and short slug.
- ☐ **IM-02** Coding Standards followed (Vol I Ch 7): no magic strings, no fallback hacks, single implementation, single ownership.
- ☐ **IM-03** Tests written at every applicable layer (unit, integration, contract, end-to-end) per PB Ch 6.
- ☐ **IM-04** Local smoke test passes (100%); no skipped or quarantined tests in the changed area.
- ☐ **IM-05** Self-review completed against the Review Checklist (PB Ch 19 §19.3); issues found in self-review fixed before requesting peer review.

- ☐ **IM-06** PR opened with required metadata: Constitution clause ref, task ticket, applicable checklists, ADR link (if architectural).
- ☐ **IM-07** CI pipeline green on the PR branch (build, lint, unit, integration, contract tests).
- ☐ **IM-08** Commits follow the conventional format (PB Ch 4); squash merge strategy confirmed.
- ☐ **IM-09** No secrets, credentials, or PII committed; pre-commit hook passed.
- ☐ **IM-10** Feature flag (if applicable) created and defaulted off; flag key recorded in the PR.
- ☐ **IM-11** Database migrations (if any) are reversible; forward and reverse migration tested locally.
- ☐ **IM-12** Documentation updated (PB Ch 5 §5.7): API reference, runbooks, ADRs, README as applicable.
- ☐ **IM-13** Observability added: structured logs, metrics, traces; correlation IDs propagated.
- ☐ **IM-14** Idempotency key (if API change) implemented and tested; retries produce the same end-state.
- ☐ **IM-15** Error handling follows the standard envelope (Vol I Ch 9); no swallowed exceptions; no silent failures.
- ☐ **IM-16** No new dependencies added without ADR (PB Ch 3 §3.6); license compatibility verified.
- ☐ **IM-17** Performance budget checked (Vol I Ch 17); no unbounded queries; no N+1 patterns introduced.
- ☐ **IM-18** Accessibility checked for UI changes (Vol II Ch 30); keyboard navigation, ARIA, contrast.
- ☐ **IM-19** CODEOWNERS path matches the changed files; correct reviewers will be auto-assigned.
- ☐ **IM-20** PR description complete: what, why, how, test plan, rollback plan, screenshot/recording (if UI).
- ☐ **IM-21** Knowledge Graph requirement node ID recorded in the task ticket (Vol II Ch 23).

19.3 Review Checklist

The Review Checklist is applied by the reviewer during the Review stage (PB Ch 2 §2.6, PB Ch 5). It combines items from PB Ch 2 (RV-01..08) and PB Ch 5. The checklist is a GATE to merge; a PR SHALL NOT be merged until every item is satisfied or marked N/A with justification. For risk-class R1 features, two reviewers SHALL each apply the checklist independently.

The Review Checklist is organized into nine categories: Constitution traceability, coding standards, idempotency, error handling, testing, security, observability, documentation, and performance. The categories are not exhaustive; the reviewer SHALL also apply engineering judgement and request changes for any issue not captured by an item.

Review Checklist (GATE to Merge)

- ☐ **RV-01** Reviewer assigned and is not the implementer; for R1 features, two independent reviewers assigned.
- ☐ **RV-02** Constitution clause cited by the reviewer in the approval comment (PROC-004); the clause is the one the implementation satisfies.
- ☐ **RV-03** Playbook checklist item satisfied (PROC-004); the reviewer cites the specific IM/CT/RV item verified.
- ☐ **RV-04** Knowledge Graph requirement node ID present in the task ticket; traceability established.
- ☐ **RV-05** Coding Standards (Vol I Ch 7) verified: no magic strings, no fallback hacks, single implementation, single ownership.
- ☐ **RV-06** Branching model (PB Ch 3) followed; branch name and merge strategy correct.
- ☐ **RV-07** Commit messages follow the conventional format (PB Ch 4); history is clean.
- ☐ **RV-08** Idempotency contract verified (if API change); retries produce the same end-state; idempotency key implemented.
- ☐ **RV-09** Error handling follows the standard envelope; no swallowed exceptions; no silent failures; errors are observable.
- ☐ **RV-10** Tests present at every applicable layer (PB Ch 6); tests are meaningful, not tautological; coverage thresholds met.
- ☐ **RV-11** Test data follows the test-data procedure (PB Ch 6); no production data in tests; PII scrubbed.
- ☐ **RV-12** No new dependencies without ADR; license compatibility verified; supply-chain risk assessed.
- ☐ **RV-13** Security review (Vol I Ch 16): input validation, output encoding, authn/authz checks, no secrets in code.
- ☐ **RV-14** PII handling (Vol I Ch 5): no PII in logs; data classification respected; retention honoured.
- ☐ **RV-15** Observability: structured logs, metrics, traces present; correlation IDs propagated; dashboards updated.
- ☐ **RV-16** Invariant impact assessed (Vol II Ch 26): the change does not violate any engineering invariant; monitors remain valid.
- ☐ **RV-17** SLO impact assessed (Vol II Ch 32): the change does not degrade any SLO; error budget consumption considered.
- ☐ **RV-18** Database changes (if any) follow the Database Constitution (Vol II Ch 33); migrations reversible; indexing reviewed.
- ☐ **RV-19** API changes (if any) follow the API Constitution (Vol I Ch 9); versioning, pagination, error envelope respected.

- ☐ **RV-20** Event changes (if any) follow the Event Constitution (Vol II Ch 29); schema versioned; consumers idempotent.
- ☐ **RV-21** UI changes (if any) follow the UI Constitution (Vol II Ch 30); component library used; visual regression tests added.
- ☐ **RV-22** Documentation updated (PB Ch 5 §5.7): API reference, runbooks, ADRs, README as applicable.
- ☐ **RV-23** Performance budget checked (Vol I Ch 17); no unbounded queries; no N+1 patterns; latency within budget.
- ☐ **RV-24** Accessibility verified for UI changes (Vol II Ch 30): keyboard, ARIA, contrast, screen-reader semantics.
- ☐ **RV-25** All blocking review comments addressed; non-blocking comments either addressed or deferred with a recorded ticket.
- ☐ **RV-26** Re-review (if substantive changes after initial review) completed; trivial changes (whitespace, typo) exempt.
- ☐ **RV-27** Approval recorded; PR merged by the reviewer (not the implementer); merge strategy per PB Ch 3.
- ☐ **RV-28** Post-merge CI green on the release branch; feature flag remains off until certification.

19.4 Certification Checklist

The Certification Checklist is applied by the Release Certification Authority (RCA) during the Certification stage (PB Ch 2 §2.7, PB Ch 9, PB Ch 10). It combines items from PB Ch 2 (CT-01..11), PB Ch 9, and PB Ch 10. The checklist is a GATE to release; the RCA SHALL NOT sign off certification until every item is satisfied. The RCA SHALL NOT be overridden on any runtime-verification item; the Chief Systems Architect SHALL NOT override an RCA certification failure — the implementation SHALL be corrected instead.

Certification Checklist (GATE to Release)

- ☐ **CT-01** PR merged to release branch; feature flag off; build is the certified artifact.
- ☐ **CT-02** Full test suite passes in the production-equivalent environment; no skipped or quarantined tests in the changed area.
- ☐ **CT-03** Runtime evidence captured: logs, traces, invariant monitors green for the duration of the certification run.
- ☐ **CT-04** Visual evidence captured: screenshots/recordings of every user-visible state, including error and empty states.
- ☐ **CT-05** Database verification captured: before/after persisted state; schema migration applied cleanly; no orphaned rows.

- ☐ **CT-06** Storage verification captured: artifacts stored with SHA-256 hashes; retrieval verified; immutability confirmed.
- ☐ **CT-07** API verification captured: request/response pairs; contract tests pass; idempotency, pagination, error envelope verified.
- ☐ **CT-08** Idempotency tested: retries (network, timeout, duplicate) produce the same end-state; no double-charges or duplicate records.
- ☐ **CT-09** Rollback tested: the feature can be rolled back without data loss; reverse migration (if any) applied cleanly.
- ☐ **CT-10** No invariant violations during certification: all engineering invariants (INV-I-*, INV-C-*, INV-S-*) remain green.
- ☐ **CT-11** RCA sign-off recorded in the release record; the RCA certifies the five pillars were independently verified.
- ☐ **CT-12** Canonical Object Registry lookup executed (Vol II Ch 28): all new object types and fields present in the registry.
- ☐ **CT-13** Contract verification executed (Vol II Ch 27): every service contract touched by the change has passing contract tests.
- ☐ **CT-14** Threshold verification (Vol II Ch 31): latency, throughput, error rate, and saturation within the constitutional bounds.
- ☐ **CT-15** SLO impact verified (Vol II Ch 32): no SLO degraded below its target; error budget consumption within the policy window.
- ☐ **CT-16** Cost impact verified (Vol II Ch 35): cost-per-request within budget; no unbounded cost growth introduced.
- ☐ **CT-17** Security verification (Vol I Ch 16): threat-model review for the change; no new attack surface unmitigated.
- ☐ **CT-18** Database integrity verification (Vol II Ch 33): transactional integrity; indexing; backup/restore tested.
- ☐ **CT-19** Event verification (Vol II Ch 29): event schema valid; consumers idempotent; outbox pattern respected.
- ☐ **CT-20** UI verification (Vol II Ch 30): visual regression tests pass; accessibility audits pass; deterministic rendering confirmed.
- ☐ **CT-21** Certification artifact set complete and stored in the certification repository per PB Ch 9 §9.3.
- ☐ **CT-22** Certification run reproducible: a second run from the certified artifact produces the same evidence.

19.5 Release Checklist

The Release Checklist is applied by the Release Manager during the Release stage (PB Ch 2 §2.8, PB Ch 11). It combines items from PB Ch 2 (RL-01..05) and PB Ch 11. The checklist is a GATE to deployment; the Release Manager SHALL NOT stage a release for deployment until every item is satisfied.

Release Checklist (GATE to Deployment)

- ☐ **RL-01** Release artifact built from the release branch by the CI pipeline; artifact is content-addressed (SHA-256) and signed.
- ☐ **RL-02** Staged release hash matches the certified artifact; signature verified; provenance attested.
- ☐ **RL-03** Deployment plan documented: window, staged rollout strategy, rollback procedure, communication plan, on-call assignment.
- ☐ **RL-04** On-call engineer briefed on the release, the rollback procedure, and the alert thresholds; runbooks reviewed.
- ☐ **RL-05** Rollback procedure tested in the production-equivalent environment; rollback time measured and within budget.
- ☐ **RL-06** Feature flag (if any) configured for staged audience; flag key and audience recorded in the release record.
- ☐ **RL-07** Communication plan ready: pre-deploy notice, in-progress notice, post-deploy notice; distribution lists confirmed.
- ☐ **RL-08** Release record created in the release repository; certification artifact set linked.
- ☐ **RL-09** Rollback artifact (previous certified release) staged and verified; rollback can be executed in under the target RTO.
- ☐ **RL-10** SLO dashboard baseline captured; burn-in thresholds defined for the post-deploy observation period.
- ☐ **RL-11** Invariant dashboard baseline captured; any invariants expected to change are pre-annotated.
- ☐ **RL-12** Stakeholder sign-off (Product, Engineering Leadership) recorded for R1 releases.
- ☐ **RL-13** Deployment window confirmed: no conflicting deployments, no scheduled external dependency changes.
- ☐ **RL-14** Compliance and legal review complete for content-affecting changes (retraction, takedown, legal artifact removal).
- ☐ **RL-15** Release notes drafted; user-visible changes documented; breaking changes flagged.
- ☐ **RL-16** Pause/abort criteria defined: the conditions under which the rollout is paused or aborted, and who has authority.

19.6 Deployment Checklist

The Deployment Checklist is applied by the Release Manager and the On-Call engineer during the Deployment stage (PB Ch 2 §2.9, PB Ch 11). It combines items from PB Ch 2 (DP-01..07) and PB Ch 11. The checklist is a GATE to post-release; the deployment SHALL NOT be declared complete until every item is satisfied.

Deployment Checklist (GATE to Post-Release)

- ☐ **DP-01** Deployment window open; on-call engineer paged and present; Release Manager available.
- ☐ **DP-02** Staged rollout executed: first stage (canary) deployed; observation period elapsed; metrics green; next stage proceeded.
- ☐ **DP-03** Each rollout stage observed for the defined period; no stage advanced on red metrics.
- ☐ **DP-04** Feature flag enabled (if applicable) for the staged audience; flag rollout observed.
- ☐ **DP-05** Post-deploy verification executed: runtime checks, invariant monitors, SLO dashboard; all green.
- ☐ **DP-06** Invariant monitors green throughout deployment; no INV-I/INV-C/INV-S violations.
- ☐ **DP-07** SLO dashboard green; error rate, latency, saturation within bounds; error budget not breached.
- ☐ **DP-08** Smoke tests pass in production; key user journeys verified end-to-end.
- ☐ **DP-09** Rollback runbook RB-RB-001 staged and ready; rollback decision criteria confirmed.
- ☐ **DP-10** Communication sent: deployment start, stage advances, deployment complete; stakeholders notified.
- ☐ **DP-11** Observability dashboards updated to reflect the new release version; alerts tuned if needed.
- ☐ **DP-12** On-call handoff (if shift change during deployment) executed per RB-OC-001; handoff record logged.
- ☐ **DP-13** Deployment logs archived to the observability platform; retention per Volume II Chapter 31 §31.8.
- ☐ **DP-14** Post-deploy burn-in declared: 72-hour observation period begins; owning team carries the pager.

19.7 Rollback Checklist

The Rollback Checklist is applied by the On-Call engineer when a deployment must be reversed (PB Ch 12). Rollback is the operational response to a failed deployment, a violated invariant, a breached SLO, or a discovered defect. The checklist is a GATE to declaring the system stable; rollback SHALL NOT be declared complete until every item is satisfied.

Rollback Checklist (GATE to Stable)

- ☐ **RB-01** Rollback trigger confirmed: failed deployment, invariant violation, SLO breach, or discovered defect; trigger recorded.
- ☐ **RB-02** Rollback decision authority confirmed: On-Call may rollback; RCA/CSA required for forward-fix instead of rollback.
- ☐ **RB-03** Rollback artifact (previous certified release) staged and verified; hash and signature match the release record.
- ☐ **RB-04** Rollback procedure RB-RB-001 executed: staged rollout in reverse; each stage observed.
- ☐ **RB-05** Reverse database migration (if any) applied cleanly; no data loss; integrity verified.
- ☐ **RB-06** Feature flag (if applicable) disabled; flag state verified across all environments.
- ☐ **RB-07** Post-rollback verification: runtime checks, invariant monitors, SLO dashboard all green.
- ☐ **RB-08** Rollback time measured; total time-to-stable recorded; within RTO target.
- ☐ **RB-09** Communication sent: rollback initiated, rollback complete; stakeholders notified.
- ☐ **RB-10** Rollback recorded in the release record; RCA notified; incident declared if SEV-1/SEV-2.
- ☐ **RB-11** Rollback evidence (logs, metrics, before/after state) captured for postmortem.
- ☐ **RB-12** Forward-fix plan drafted: the failed change is corrected, re-certified, and re-released; no re-deploy of the failed artifact.

19.8 Incident Response Checklist

The Incident Response Checklist is applied by the Incident Commander (IC) and the on-call engineers during a production incident (PB Ch 12, PB Ch 16). It combines items from PB Ch 12 and PB Ch 16. The checklist is a GATE to incident closure; an incident SHALL NOT be closed until every item is satisfied.

Incident Response Checklist (GATE to Closure)

- ☐ **IR-01** Incident declared: severity (SEV-1..SEV-5) assigned per the severity matrix; IC named; war room opened.
- ☐ **IR-02** Incident commander identified; IC is not the on-call engineer (separation of duties); IC owns coordination, not fix.
- ☐ **IR-03** Stakeholders notified: Engineering Leadership, Product, Support, Communications (for SEV-1/SEV-2).
- ☐ **IR-04** Status page updated (for SEV-1/SEV-2); user-visible impact communicated.
- ☐ **IR-05** Mitigation in progress: rollback, feature-flag off, rate-limit, or failover applied to stop the bleeding.
- ☐ **IR-06** Root cause under investigation: hypothesis recorded; evidence (logs, traces, metrics) preserved.

- ☐ **IR-07** Communication cadence established: updates every 30 min (SEV-1) or 2 h (SEV-2) until resolved.
- ☐ **IR-08** Resolution applied: fix deployed or mitigation made permanent; system verified stable.
- ☐ **IR-09** Incident resolved: all monitors green for the defined observation period; IC declares resolution.
- ☐ **IR-10** Postmortem scheduled (SEV-1 within 5 business days; SEV-2 within 10; SEV-3 within 20); blameless culture affirmed.
- ☐ **IR-11** Postmortem published: root cause, timeline, impact, action items with owners and due dates; distributed to Engineering.
- ☐ **IR-12** Action items tracked to completion; RCA reviews monthly until all action items are closed.
- ☐ **IR-13** Major incident review (SEV-1) conducted with Chief Systems Architect; systemic lessons recorded in the precedent register.
- ☐ **IR-14** Incident record archived; evidence retained per the retention schedule (Vol II Ch 31 §31.8).

19.9 On-Call Readiness Checklist

The On-Call Readiness Checklist is applied by every engineer before taking a shift on the on-call rotation (PB Ch 13). The checklist verifies that the engineer is prepared to respond to incidents and operational requests. It is a GATE to taking the pager; an engineer SHALL NOT take the pager until every item is satisfied.

On-Call Readiness Checklist (GATE to Pager)

- ☐ **OC-01** Current with the on-call runbook index (PB Ch 20); reviewed at least the top-10 most-invoked runbooks.
- ☐ **OC-02** Access verified: production console, observability dashboards, deployment tooling, incident-management tooling.
- ☐ **OC-03** Break-glass procedure (RB-OPS-004) reviewed; break-glass credentials confirmed accessible but not yet used.
- ☐ **OC-04** Escalation matrix current: knowing who to page for Crossref, DOI, database, network, security.
- ☐ **OC-05** Handoff received from the previous on-call (RB-OC-001); open issues, in-flight deployments, watch-items noted.
- ☐ **OC-06** Communication channels monitored: PagerDuty, Slack incident channel, status page.
- ☐ **OC-07** Personal readiness: rested, reachable, not double-scheduled; backup on-call confirmed available.
- ☐ **OC-08** Runbook staleness check: any runbook whose last-reviewed date is older than the quarterly cadence flagged for review.

- ☐ **OC-09** Current SLO state reviewed: error budgets, burn-rate alerts, any SLO in At-risk or Critical state.
- ☐ **OC-10** Current invariant state reviewed: any invariant in warning or violation; any monitor flapping.
- ☐ **OC-11** Recent deployments reviewed: any deployment in its burn-in period; any rollback in progress.
- ☐ **OC-12** On-call shift handoff document prepared for the next engineer; updated at shift end.

19.10 The Production Acceptance Gate

The Production Acceptance Gate is the final gate of the Playbook. It combines the Certification Checklist (§19.4), the Release Checklist (§19.5), and the Deployment Checklist (§19.6) into a single, indivisible gate that blocks production release. The Gate is the operational form of Foundational Principle II (the Production Acceptance Gate) and the Prime Directive of this Playbook (PB Ch 1 §1.2).

The Gate is the ultimate gate. No implementation MAY proceed to production release unless it has passed every applicable item in all three checklists. The Gate SHALL NOT be bypassed; the Gate SHALL NOT be overridden below the Chief Systems Architect; and the Chief Systems Architect SHALL NOT override an RCA certification failure — the implementation SHALL be corrected instead. The Gate is the operational guarantee that the Constitution's rules are enforced, not aspirational.

THE PRODUCTION ACCEPTANCE GATE (FINAL)

No implementation MAY proceed to production release unless ALL of the following are satisfied: (1) Certification Checklist (CT-01..CT-22) passed and RCA sign-off recorded; (2) Release Checklist (RL-01..RL-16) passed and release artifact staged and signed; (3) Deployment Checklist (DP-01..DP-14) passed and post-deploy verification green. The Gate SHALL NOT be bypassed. The Gate SHALL NOT be overridden below the Chief Systems Architect. The Chief Systems Architect SHALL NOT override an RCA certification failure — the implementation SHALL be corrected instead. This is the operational form of Foundational Principle II. (See Appendix F for the single-page summary.)

The Gate is invoked at the end of the Deployment stage (PB Ch 2 §2.9) and is the entry criterion for the Post-Release Ownership stage (PB Ch 2 §2.10). The Gate is jointly enforced by the RCA (for the certification leg), the Release Manager (for the release leg), and the On-Call engineer (for the deployment leg). The Chief Systems Architect is the authority of last resort for Gate disputes but SHALL NOT unilaterally override the certification leg.

19.11 Checklist Maintenance

The checklists in this catalog are versioned artifacts. They are amended through the Evolution Governance process (Volume II Chapter 36) and are reviewed on the monthly Playbook review cadence.

A checklist amendment SHALL follow the standard amendment lifecycle (proposal, review, discussion, voting, approval, implementation, certification) and SHALL increment the catalog version.

Checklist items are stable within a version. When an item is amended, the item number is retired (not reused) and a new number is issued, so that historical evidence (CI run links, screenshots, sign-offs) remains interpretable against the version under which it was produced. A retired item is recorded in the amendment history with its successor number.

Two maintenance principles apply. First, no checklist item is removed without a successor: an item that is no longer relevant is retired and replaced by an explicit N/A item or by an item in a different checklist, so that the historical record is preserved. Second, no checklist item is added without an owner: every item SHALL name the role accountable for verifying it, and the role SHALL be staffed. An item without an owner is a gap and SHALL be flagged.

The catalog version is recorded in Appendix D (Document Control). The version SHALL be cited in every release record and every certification artifact set, so that an auditor can determine which catalog version was in force when the evidence was produced.

Chapter 20 — Runbook Catalog

20.1 Purpose

This chapter collects and indexes every runbook referenced in the Playbook. A runbook is a documented, step-by-step response to a specific failure, scenario, or operational request. Runbooks are the operational form of the Constitution's failure, observability, and operational-governance chapters; they convert abstract rules into executable procedures that an on-call engineer can apply at three o'clock in the morning.

Constitution Reference: *Volume I, Chapter 21 (Failure Constitution); Volume II, Chapter 26 (Engineering Invariants); Volume II, Chapter 34 (Operational Governance)*

The catalog is the index. Each runbook has a stable ID (RB-XXX-NNN), a name, a trigger, a severity, an owner, and a location. The location points to the runbook's full text in the runbook repository. The catalog is normative: an on-call engineer SHALL consult the catalog to locate the correct runbook; a runbook not in the catalog is not an approved runbook and SHALL NOT be invoked in production.

Runbooks are written, reviewed, published, maintained, and retired through a defined lifecycle (§20.4). Every runbook SHALL follow the structure standard (§20.3). Runbook anti-patterns (§20.5) are forbidden and are detected during quarterly review. The catalog is the engineer's assurance that the runbook being invoked is current, tested, and authoritative.

20.2 Runbook Index

The index lists every approved runbook. The columns are: the runbook ID (stable across versions); the runbook name; the trigger (the condition under which the runbook is invoked); the severity (the highest severity the runbook is designed to address); the owner (the role accountable for maintaining the runbook); and the location (the path in the runbook repository).

Runbook ID	Name	Trigger	Severity	Owner	Location
RB-INV-001	Invariant Violation SEV-1	Any INV-I/INV-C/INV-S invariant monitor in violation state with SEV-1 impact (data loss, user- visible outage).	SEV-1	On-Call	/runbooks/inv/ 001-sev1.md
RB-INV-002	Invariant Violation SEV-2	Any invariant monitor in violation state with SEV-2 impact (degraded but not down).	SEV-2	On-Call	/runbooks/inv/ 002-sev2.md

RB-DEP-001	Standard Deployment	Scheduled deployment of a certified release within the deployment window.	SEV-5	Release Manager	/runbooks/dep/001-standard.md
RB-DEP-002	Emergency Hotfix Deployment	Out-of-window deployment required for a critical production defect; RCA + CSA approval required.	SEV-1	Release Manager	/runbooks/dep/002-hotfix.md
RB-RB-001	Rollback Execution	Failed deployment, invariant violation, or SLO breach requiring reversal to the previous certified release.	SEV-1..2	On-Call	/runbooks/rb/001-rollback.md
RB-INC-001	SEV-1 Incident Response	Production outage or data loss affecting users; war room and IC required.	SEV-1	Incident Commander	/runbooks/inc/001-sev1.md
RB-INC-002	SEV-2 Incident Response	Significant degradation affecting a user cohort; coordinated response required.	SEV-2	Incident Commander	/runbooks/inc/002-sev2.md
RB-INC-003	Major Incident Review	Post-incident review for a SEV-1 incident; blameless postmortem with CSA present.	SEV-1	Incident Commander	/runbooks/inc/003-review.md
RB-OC-001	On-Call Handoff	Shift change between on-call engineers; transfer of open issues, in-flight work, and watch-items.	SEV-5	On-Call	/runbooks/oc/001-handoff.md
RB-OC-002	Responding to a	On-call engineer is	SEV-2..5	On-Call	/runbooks/oc/

	Page	paged; the first response procedure (acknowledge, assess, acknowledge).			002-respond.md
RB-OC-003	First 5 Minutes of a Page	The critical first five minutes: acknowledge, open the incident channel, page the right people, capture state.	SEV-1..5	On-Call	/runbooks/oc/003-first5.md
RB-OPS-001	Credential Rotation	Scheduled or emergency rotation of API credentials (Crossref, DOI, ORCID, storage).	SEV-3	On-Call	/runbooks/ops/001-cred-rotation.md
RB-OPS-002	TLS Certificate Rotation	Scheduled or emergency rotation of TLS certificates for public-facing endpoints.	SEV-3	On-Call	/runbooks/ops/002-tls-rotation.md
RB-OPS-003	Database Migration	Execution of a database schema migration; forward and reverse; verification.	SEV-2	On-Call	/runbooks/ops/003-db-migration.md
RB-OPS-004	Break-Glass Access	Emergency elevation of access privileges; break-glass credentials, post-hoc review.	SEV-1	On-Call	/runbooks/ops/004-break-glass.md
RB-OPS-005	Disaster Recovery Activation	Activation of the disaster-recovery site; failover of primary services; data reconciliation.	SEV-1	On-Call	/runbooks/ops/005-dr-activation.md
RB-OPS-006	Legal Artifact Removal	Court-ordered or policy-ordered	SEV-2	On-Call	/runbooks/ops/006-legal-

		removal of a published artifact; takedown with audit trail.			removal.md
RB-SLO-001	Burn Rate Alert Response	A burn-rate alert has fired (multi-window multi-burn-rate pattern); assessment and mitigation.	SEV-2..3	On-Call	/runbooks/slo/001-burn-rate.md
RB-SLO-002	Error Budget Exhaustion	An error budget is exhausted; feature freeze invoked; engineering effort redirected to reliability.	SEV-2	On-Call	/runbooks/slo/002-budget-exhaust.md
RB-QA-001	Flaky Test Handling	A test is identified as flaky; quarantine, root-cause, fix, un-quarantine within the SLA.	SEV-4	QA	/runbooks/qa/001-flaky-test.md
RB-OPS-007	Storage Volume Expansion	Storage volume near capacity; expansion or archival executed without service interruption.	SEV-3	On-Call	/runbooks/ops/007-storage-expand.md
RB-INV-003	Invariant Monitor Flapping	An invariant monitor is flapping (rapid violation/recovery cycles); investigation and stabilization.	SEV-3	On-Call	/runbooks/inv/003-flapping.md

The index above lists twenty-two runbooks. New runbooks SHALL be added when a new failure mode, scenario, or operational request is identified that is not covered by an existing runbook. Runbook additions follow the Evolution Governance process (Volume II Chapter 36) and SHALL be reflected in this index in the same amendment cycle.

20.3 Runbook Structure Standard

Every runbook in the catalog SHALL conform to the structure standard below. The standard ensures that an on-call engineer can open any runbook and find the same sections in the same order, with the same metadata, so that response time is not spent navigating unfamiliar layouts. A runbook that does not conform is not an approved runbook and SHALL be returned for revision.

The structure standard has eight required sections:

- **Trigger** — the condition under which the runbook is invoked. The trigger SHALL be specific (a monitor name, an alert threshold, a user report type) so that the on-call engineer can determine with certainty whether the runbook applies.
- **Severity** — the highest severity the runbook is designed to address. The severity SHALL be drawn from the severity matrix (SEV-1 through SEV-5) and SHALL determine the escalation path.
- **Prerequisites** — what MUST be true before the runbook is invoked (access, credentials, runbooks already executed, approvals obtained). The prerequisites SHALL be checkable.
- **Steps** — the numbered, ordered actions the on-call engineer takes. Each step SHALL be a single action with a single owner and a single verification.
- **Verification** — how the on-call engineer confirms the runbook achieved its objective. Verification SHALL be objective (a monitor is green, a query returns expected results, a user confirms resolution).
- **Escalation** — when and to whom to escalate if the runbook does not resolve the situation within the expected time. The escalation SHALL name roles, not individuals.
- **Post-Action** — what the on-call engineer does after the runbook is complete (close the incident, file a follow-up ticket, update the runbook, schedule a postmortem).
- **Last-Reviewed Date** — the date the runbook was last reviewed by its owner. A runbook whose last-reviewed date is older than the quarterly cadence is stale and SHALL be flagged.

Runbook Structure Standard

Every approved runbook SHALL contain, in order: Trigger, Severity, Prerequisites, Steps, Verification, Escalation, Post-Action, and Last-Reviewed Date. A runbook missing any section is non-conformant and SHALL NOT be invoked in production. The structure standard is enforced at runbook review (quarterly) and at runbook publication (Evolution Governance).

20.4 Runbook Lifecycle

Runbooks follow a five-state lifecycle: draft, reviewed, published, maintained, and retired. The lifecycle is the operational form of the documentation lifecycle established in Volume I Chapter 24, specialized for runbooks. The lifecycle is governed by the Evolution Governance process (Volume II Chapter 36).

State	Entry Criteria	Owner Action	Exit Criteria
Draft	A new failure mode or	Author drafts the runbook	Draft complete; structure

	scenario is identified; an author is assigned.	per the structure standard; walkthrough-tested in a non-production environment.	standard satisfied; ready for review.
Reviewed	Draft complete.	Reviewer (not the author) validates the runbook against the structure standard and the operational reality; tabletop exercise executed.	Reviewer sign-off recorded; ready for publication.
Published	Reviewer sign-off recorded.	Runbook added to the catalog (this chapter); indexed by ID; announced to the on-call rotation.	Runbook in the catalog; on-call engineers notified.
Maintained	Runbook published.	Owner reviews the runbook on the quarterly cadence; updates for accuracy; reflects any operational changes since the last review.	Last-Reviewed Date updated; runbook remains current.
Retired	Runbook no longer applicable (failure mode eliminated, scenario merged with another runbook, system retired).	Owner marks the runbook retired; the catalog entry is annotated; a successor runbook is named if applicable.	Runbook removed from active use; catalog updated; historical record retained.

The review cadence is quarterly. Every published runbook SHALL be reviewed by its owner at least once per quarter. The review verifies that the runbook still reflects the operational reality (monitor names, escalation paths, tooling, ownership). A runbook whose Last-Reviewed Date is older than ninety days is stale and SHALL be flagged for review; an on-call engineer who encounters a stale runbook SHALL record the staleness and SHALL proceed with caution, treating the runbook as a starting point rather than an authoritative procedure.

Staleness detection is automated where possible. The runbook repository SHALL emit a weekly report of runbooks whose Last-Reviewed Date is older than ninety days; the report is reviewed by the Engineering Leadership Team and stale runbooks are assigned for review. A runbook that is stale for more than one hundred eighty days is automatically moved to the 'maintained (at-risk)' state and is flagged in the catalog; an at-risk runbook SHALL NOT be the sole source of truth for a SEV-1 response, and a fresh runbook SHALL be drafted as a replacement.

20.5 Runbook Anti-Patterns

The following anti-patterns are forbidden. A runbook exhibiting any of these anti-patterns SHALL be returned for revision at review and SHALL NOT be published. The detection column names the mechanism by which the anti-pattern is identified during the lifecycle.

Anti-Pattern	Why Forbidden	Detection
Runbook never tested	An untested runbook is a hypothesis, not a procedure. The first time it is invoked should not be during a SEV-1.	Tabletop exercise required at the Reviewed state; the reviewer confirms the exercise was executed.
Runbook out of date	An out-of-date runbook misleads the on-call engineer, lengthens the incident, and erodes trust in the catalog.	Staleness detection (Last-Reviewed Date > 90 days); weekly report; at-risk flag at 180 days.
Runbook too abstract	An abstract runbook ('investigate the issue', 'consider rollback') does not provide actionable steps and is not a runbook.	Reviewer checks that each step is a single concrete action with a single owner and a single verification; abstract steps are rejected.
Runbook without escalation	A runbook without an escalation path leaves the on-call engineer without recourse when the runbook fails; this extends incidents.	Structure-standard check: the Escalation section is present and names roles, not individuals.
Runbook without verification	A runbook without verification cannot confirm resolution; the on-call engineer declares success on gut feeling, which is unreliable.	Structure-standard check: the Verification section is present and is objective (monitor, query, user confirmation).
Runbook that hides an incident	A runbook whose steps mask the underlying defect (e.g., silently restarting a service without RCA) forbids learning and guarantees recurrence.	Reviewer checks that the Post-Action section requires a follow-up ticket and, for SEV-1/SEV-2, a postmortem.
Runbook that bypasses the Gate	A runbook whose steps bypass the Production Acceptance Gate (e.g., direct production edits without certification) violates Foundational Principle II.	Reviewer checks that no step bypasses the Gate; emergency runbooks (RB-DEP-002, RB-OPS-004, RB-OPS-005) explicitly invoke the emergency-amendment process with post-hoc review.
Runbook with named individuals	A runbook that names individuals (instead of roles) breaks when the individual leaves the team or changes role.	Reviewer checks that all names are roles (On-Call, IC, RCA, CSA, Release Manager); named individuals are replaced with roles.

The anti-patterns above are detected at runbook review (the Reviewed state), at the quarterly maintenance review, and at post-incident review (where the runbook's effectiveness is assessed). A runbook that exhibits an anti-pattern in production SHALL be corrected within five business days for SEV-1/SEV-2 runbooks and within twenty business days for SEV-3..5 runbooks. Repeated anti-pattern violations by the same owner SHALL be escalated to the Engineering Leadership Team.

Appendices

The following appendices provide the reference material that supports the Playbook: the glossary of operational terms, the consolidated Constitution-to-Playbook cross-reference, the runbook templates, the document-control record, the references, and the single-page summary of the Production Acceptance Gate. The appendices are normative reference material; they are cited by the operational chapters and SHALL be maintained in lockstep with the chapters they support.

Appendix A — Glossary of Operational Terms

This glossary defines the operational terms used throughout the Playbook. The terms are drawn from the Constitution, RFC 2119, the Google SRE Workbook, NIST publications, and the Playbook itself. Where a term has a Constitution-specific meaning, the Constitution's definition prevails; the glossary entry is the operational interpretation.

Term	Definition
GATE	A hard block in the implementation lifecycle. Progress cannot proceed until the gate is satisfied. Enforced automatically (CI) or manually (reviewer/RCA).
CHECK	A checkable item in a Playbook checklist. Marked satisfied (with evidence) or N/A (with justification).
PROCEDURE	A sequence of steps that MUST be followed in order. Verified by the reviewer at review time.
RUNBOOK	A documented, step-by-step response to a specific failure, scenario, or operational request. Invoked by the on-call engineer during incidents.
ALERT	An automated signal that triggers human attention. Routed per the escalation matrix; paging the on-call engineer.
EVIDENCE	An artifact that proves a procedure was followed or a gate was satisfied. Stored in the certification repository; retained per schedule.
RCA	Release Certification Authority. The role that signs off certification; distinct from the implementing team and from QA. Cannot be overridden on runtime verification.
IC	Incident Commander. The role that coordinates an incident response; owns coordination, not the fix. Separation of duties from the on-call engineer.
SEV-1	Severity 1. Production outage or data loss affecting users. War room, IC, CSA notification, postmortem within 5 business days.
SEV-2	Severity 2. Significant degradation affecting a user cohort. Coordinated response, postmortem within 10 business days.
SEV-3	Severity 3. Moderate impact; workaround available. Resolved during business hours; postmortem within 20 business days.
SEV-4	Severity 4. Minor impact; no user effect. Tracked and resolved as capacity allows.
SEV-5	Severity 5. Cosmetic or informational. Resolved at leisure.
SLO	Service Level Objective. The target for an SLI over a compliance window (e.g., 99.9% availability over 28 days).

SLI	Service Level Indicator. A quantitative measure of service health (e.g., success rate, latency percentile).
SLA	Service Level Agreement. An external contract with users, with consequences if breached. Stricter than the SLO.
Error Budget	The allowable amount of unreliability over the compliance window (1 - SLO target). When exhausted, feature freeze is invoked.
Burn Rate	The rate at which the error budget is being consumed, relative to the sustainable rate. Multi-window multi-burn-rate alerting pattern.
Canary	A staged rollout stage in which a change is exposed to a small fraction of traffic before broader rollout. Monitored for the observation period.
Feature Flag	A runtime toggle that enables or disables a feature without redeployment. Defaulted off until certification is complete.
Break-Glass	Emergency elevation of access privileges, used when normal access is insufficient to resolve an incident. Post-hoc review required.
War Room	A dedicated incident-response channel (physical or virtual) for SEV-1 incidents. IC coordinates; stakeholders join.
Postmortem	A blameless, written analysis of an incident: root cause, timeline, impact, action items. Published and distributed.
CODEOWNERS	A repository file mapping paths to the teams responsible for review. Enforces the branching and review model (PB Ch 3).
ADR	Architecture Decision Record. A recorded decision with context, alternatives, and consequences. Required for architectural decisions (Vol I Ch 7).
PR	Pull Request. The unit of review; the artifact that travels through the Review gate to merge.
CI	Continuous Integration. The automated pipeline that builds, lints, and tests every PR. Enforces the Implementation Gate.
CD	Continuous Delivery/Deployment. The automated pipeline that stages and deploys certified releases. Enforces the Deployment Gate.
Trunk-Based	A branching model in which all developers merge to a single trunk (main) branch frequently, behind feature flags.
Squash Merge	A merge strategy that combines all commits on a branch into a single commit on the target branch. The Playbook's default strategy.
Idempotency Key	A client-supplied key that allows a server to deduplicate

	retries; the same key produces the same end-state.
ETag	An entity tag identifying a specific version of a resource; used for conditional requests and optimistic concurrency.
Outbox	A pattern in which database writes and event publication are coordinated through an outbox table, ensuring atomicity (Vol II Ch 29).
Invariant	A property that MUST hold at all times in a running system. Catalogued as INV-I-*, INV-C-*, INV-S-* (Vol II Ch 26).
Reconciliation	A periodic job that detects and corrects drift between the expected and actual state of a system, used to enforce invariants that cannot be enforced transactionally.

Appendix B — Constitution-to-Playbook Cross-Reference

This appendix consolidates the Volume I matrix (Chapter 17) and the Volume II matrix (Chapter 18) into a single cross-reference table. The consolidated table is the single source of truth for traceability audits and SHALL be used in preference to the per-volume matrices when a complete view is required. The consolidated table is maintained in lockstep with the per-volume matrices; an amendment to either per-volume matrix SHALL be reflected here in the same amendment cycle.

Const. Ch.	Title	Operationalizing PB Ch.	Operational Form	Owner
Vol I 1	Product Mission	PB Ch 2	Clause selection in task tickets.	Tech Lead
Vol I 2	Engineering Philosophy	PB Ch 5, 13	Review checklist; on-call ownership.	Reviewer; On-Call
Vol I 3	System Context	PB Ch 14	Boundary enforcement in ADRs.	Tech Lead
Vol I 4	System Architecture	PB Ch 2, 11	ADRs; deployment pipeline.	Architect; Release Manager
Vol I 5	Data Constitution	PB Ch 6, 9	Test data; certification DB verification.	QA; RCA
Vol I 6	Canonical Registry	PB Ch 9	Canonical field lookup in certification.	RCA
Vol I 7	ADRs	PB Ch 2, 5	ADR required in PR for architectural decisions.	Reviewer; Architect
Vol I 8	Repository Constitution	PB Ch 3	Branching; CODEOWNERS.	Tech Lead
Vol I 9	API Constitution	PB Ch 5, 10	API spec in review; API pillar in certification.	Reviewer; RCA
Vol I 10	Workflow Constitution	PB Ch 6, 8	Workflow tests; invariant monitors.	QA; On-Call
Vol I 11	Publication State Machine	PB Ch 8	Invariant INV-C-07 monitor.	On-Call
Vol I 12	Storage Constitution	PB Ch 10	Storage pillar in certification.	RCA
Vol I 13	Rendering Constitution	PB Ch 10	Visual pillar in certification.	QA; RCA
Vol I 14	Metadata Constitution	PB Ch 10, 14	API pillar; credential rotation.	RCA; On-Call
Vol I 15	Template Constitution	PB Ch 6	Template tests.	QA

Vol I 16	Security Constitution	PB Ch 5, 14	Security review checklist; credential rotation.	Reviewer; On-Call
Vol I 17	Performance Constitution	PB Ch 6, 15	Performance tests; SLO management.	QA; On-Call
Vol I 18	Observability Constitution	PB Ch 8, 15	Invariant dashboard; SLO dashboard.	On-Call
Vol I 19	Testing Constitution	PB Ch 6, 7	Test pyramid; environments.	QA
Vol I 20	Runtime Verification Constitution	PB Ch 8, 10	Invariant monitors; five pillars.	RCA; On-Call
Vol I 21	Failure Constitution	PB Ch 12, 16	Rollback; incident command.	On-Call; IC
Vol I 22	Certification Constitution	PB Ch 9	Certification process.	RCA
Vol I 23	Knowledge Graph Constitution	PB Ch 2	Clause selection traces to requirement node.	Tech Lead
Vol I 24	Documentation Constitution	PB Ch 5	Docs review in PR.	Reviewer
Vol I 25	Future Evolution	PB Ch 14	Evolution governance execution.	Architect
Vol II 26	Engineering Invariants	PB Ch 8	Invariant dashboard; monitors; runbooks.	On-Call
Vol II 27	System Contracts	PB Ch 5, 10	Contract verification in review and certification.	Reviewer; RCA
Vol II 28	Canonical Object Registry	PB Ch 9	Registry lookup in certification.	RCA
Vol II 29	Event Constitution	PB Ch 6, 10	Event tests; API pillar.	QA; RCA
Vol II 30	UI Constitution	PB Ch 6, 10	Visual regression tests; visual pillar.	QA; RCA
Vol II 31	Engineering Mathematics	PB Ch 9, 15	Threshold verification; SLO management.	RCA; On-Call
Vol II 32	Production SLO Book	PB Ch 15	SLO dashboard; error budget management.	On-Call
Vol II 33	Database Constitution	PB Ch 6, 14	DB tests; migration execution.	QA; On-Call
Vol II 34	Operational Governance	PB Ch 14	Authority matrix execution.	On-Call; Architect

Vol II 35	Engineering Economics	PB Ch 15	Cost review in SLO review.	On-Call; Architect
Vol II 36	Evolution Governance	PB Ch 14	Amendment process execution.	Architect

Appendix C — Runbook Templates

This appendix provides three runbook templates that conform to the Runbook Structure Standard (PB Ch 20 §20.3). The templates are the starting point for any new runbook; an author copies the appropriate template, fills in the placeholders, and submits the result for review. The templates are normative: a runbook that does not conform to one of these templates SHALL be returned for revision.

C.1 Standard Runbook Template

The standard runbook template is the default. It applies to operational scenarios that are not incidents (e.g., scheduled deployments, credential rotations, database migrations). The template is below; placeholders are marked with angle brackets.

#	RB-OPS-NNN:	<Runbook	Name>
##			Trigger
	<The specific condition under which this runbook is invoked. Name the monitor, alert, or scenario.>		
##			Severity
	<SEV-1..SEV-5, drawn from the severity matrix.>		
##			Prerequisites
-	<Access requirement, e.g., production console access.>		
-	<Approval requirement, e.g., RCA sign-off.>		
-	<Prior runbook, e.g., RB-OC-003 First 5 Minutes.>		
##			Steps
1.	<Step 1. Single action, single owner, single verification.>		
2.	<Step 2.>		
3.	<Step 3.>		
##			Verification
	<Objective confirmation that the runbook achieved its objective: a monitor is green, a query returns expected results, a user confirms resolution.>		
##			Escalation
	<When and to whom to escalate if the runbook does not resolve the situation within the expected time. Name roles, not individuals.>		
##			Post-Action
-	<Close the ticket.>		
-	<File a follow-up ticket for any deferred work.>		
-	<Update this runbook if any step was inaccurate.>		
##		Last-Reviewed	Date
	<YYYY-MM-DD> by <Role>		

C.2 Incident Runbook Template

The incident runbook template applies to SEV-1 and SEV-2 incidents. It adds an Incident Commander coordination section and a communication cadence that the standard template does not require. The template is below.

#	RB-INC-NNN:	<Incident	Name>
##			Trigger
	<The specific alert, monitor, or user report that indicates a SEV-1/SEV-2 incident.>		
##			Severity
	SEV-<1 2>		
##	Incident		Commander
	<The IC is named at declaration. The IC owns coordination, not the fix. Separation of duties from the on-call engineer.>		
##			Prerequisites
-	War room opened (Slack channel or physical).		
-	IC declared; on-call engineer engaged; stakeholders notified.		
-	Status page ready to update (SEV-1 / SEV-2).		
##			Steps
1.	<Acknowledge the page; open the incident channel.>		
2.	<IC declares the incident; severity confirmed.>		
3.	<Mitigation: rollback, feature-flag off, rate-limit, failover.>		
4.	<Investigation: hypothesis, evidence preservation.>		
5.	<Resolution: fix deployed or mitigation made permanent.>		
6.	<Verification: monitors green for the observation period.>		
7.	<IC declares resolution; incident closed.>		
##			Verification
	<All monitors green for the defined observation period; user impact confirmed resolved; SLOs recovered.>		
##	Communication		Cadence
-	Updates every 30 min (SEV-1) or 2 h (SEV-2) until resolved.		
-	Stakeholders: Engineering Leadership, Product, Support, Communications.		
-	Status page updated at start, at each cadence, and at resolution.		
##			Escalation
	<Escalate to the Chief Systems Architect if: SEV-1 not mitigated within 60 min; data loss suspected; legal or compliance implications.>		
##			Post-Action
-	Postmortem scheduled (SEV-1: 5 business days; SEV-2: 10).		
-	Blameless culture affirmed; RCA reviews monthly until action items are closed.		
-	Major incident review (SEV-1) with CSA; systemic lessons recorded in the precedent		

register.

Last-Reviewed Date
<YYYY-MM-DD> by <Role>

C.3 Operational Action Runbook Template

The operational action runbook template applies to scheduled or user-initiated operational requests that are not incidents and not deployments (e.g., legal artifact removal, disaster recovery activation, break-glass access). The template emphasizes approval, audit trail, and post-hoc review.

#	RB-OPS-NNN:	<Operational	Action	Name>
##				Trigger
	<The scheduled trigger (e.g., quarterly credential rotation) or the user-initiated request (e.g., court-ordered takedown) that invokes this runbook.>			
##				Severity
	<SEV-2..SEV-5, per the severity matrix.>			
##				Approvals
-	<Required approval, e.g., RCA sign-off.>			
-	<Required approval, e.g., Legal sign-off for takedowns.>			
-	<Required approval, e.g., CSA sign-off for break-glass.>			
##				Prerequisites
-	<Access, credentials, prior runbooks.>			
-	<Audit trail logging enabled; all actions recorded.>			
##				Steps
1.	<Step 1. Record the request, the approver, and the timestamp.>			
2.	<Step 2. Execute the action.>			
3.	<Step 3. Verify the action; capture before/after evidence.>			
4.	<Step 4. Notify stakeholders.>			
##				Verification
	<Objective confirmation: the action achieved its objective; the audit trail is complete; no side effects.>			
##				Escalation
	<Escalate if the action cannot be completed within the expected time, if approvals are not obtained, or if side effects are detected.>			
##				Post-Action
-	Post-hoc review (for break-glass, emergency DR activation): review the action within 5 business days; CSA present.			
-	Update this runbook if any step was inaccurate.			
-	File a follow-up ticket for any deferred work.			

# #	Last-Reviewed	Date
<YYYY-MM-DD> by <Role>		

Appendix D — Document Control

This appendix records the document-control metadata for the Engineering Playbook and the revision history. The metadata is the operational form of the Documentation Constitution (Volume I Chapter 24), specialized for the Playbook. The metadata SHALL be updated on every Playbook amendment.

Field	Value
Document ID	OP-PLAY-RC1-001
Document Type	Engineering Playbook (L3 in the Constitutional Hierarchy)
Version	1.0
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Owner	Chief Systems Architect
Last-Reviewed	09 August 2026
Next-Review	Monthly cadence; or on any Constitution amendment
Supersedes	None (first issuance)
Superseded By	Not applicable (current)
Classification	Mandatory — Operational Execution Authority
Distribution	All engineers, reviewers, QA, on-call, operators, build agents
Retention	Lifetime of the platform + 5 years
Storage	Version control (chapters/); release repository (compiled)
Governing Documents	Engineering Constitution Volume I (OP-CONST-RC1-001) + Volume II (OP-CONST-RC1-002)
Amendment Authority	Evolution Governance (Vol II Ch 36)

Revision History

Version	Date	Author	Summary of Changes
0.1 (draft)	Pre-RC1	Office of the CSA	Initial draft of the Playbook front matter and Chapter 1 (How to Use This Playbook).
0.2 (draft)	Pre-RC1	Office of the CSA	Chapter 2 (Implementation Lifecycle) added; eight-stage lifecycle established; lifecycle checklists IM/RV/CT/RL/DP

			first defined.
0.9 (RC1 candidate)	Pre-RC1	Office of the CSA	Chapters 3-16 added (Daily Disciplines, QA, Certification, Release, Operational Ownership); invariants, SLOs, incident command operationalized.
1.0 (RC1)	09 August 2026	Office of the CSA	Chapters 17-20 + Appendices A-F added; Volume I and Volume II operationalization matrices published; Master Checklist Catalog published; Runbook Catalog published; Production Acceptance Gate codified as the final gate.

Subsequent revisions SHALL be recorded in this table on each Playbook amendment. Each revision SHALL cite the Evolution Governance proposal ID, the voting outcome, and the summary of changes. Revisions SHALL NOT be applied without an accompanying entry in this table; an undocumented revision is a process violation and SHALL be flagged.

Appendix E — References

This appendix lists the references cited or implied by the Playbook. The references are the authoritative sources for the disciplines the Playbook operationalizes. Where a reference is normative (the Constitution, RFC 2119), it prevails over the Playbook; where a reference is informative (Google SRE, NIST publications, 12-Factor), it provides the conceptual foundation for the Playbook's procedures and checklists.

E.1 Normative References

- Opus Publica RC1 Engineering Constitution, Volume I (Chapters 1-25), OP-CONST-RC1-001. The supreme engineering authority; prevails over this Playbook.
- Opus Publica RC1 Engineering Constitution, Volume II: The Eleven Missing Chapters (Chapters 26-36), OP-CONST-RC1-002. Closes the eleven gaps identified in the post-Volume-I review; prevails over this Playbook.
- IETF RFC 2119 — Key words for use in RFCs to indicate requirement levels (MUST, SHALL, SHOULD, MAY). <https://datatracker.ietf.org/doc/html/rfc2119>
- IETF RFC 8259 — The JavaScript Object Notation (JSON) Data Interchange Format. The wire format for all Opus Publica APIs. <https://datatracker.ietf.org/doc/html/rfc8259>

E.2 Informative References

- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly. The conceptual foundation for SLI/SLO/error-budget discipline. <https://sre.google/sre-book/table-of-contents/>
- Beyer, B., Murphy, N. R., Rensin, D., Kawahara, K., & Thorne, S. (Eds.). (2018). The Site Reliability Workbook: Practical Approaches to Implementing Reliability. O'Reilly. The conceptual foundation for burn-rate alerting and error-budget policy. <https://sre.google/workbook/table-of-contents/>
- NIST SP 800-92 — Guide to Computer Security Log Management. The conceptual foundation for the Observability Constitution (Vol I Ch 18) and the audit trail requirements. <https://csrc.nist.gov/pubs/sp/800/92/final>
- NIST SP 800-61 (Revision 2) — Computer Security Incident Handling Guide. The conceptual foundation for the incident command system (PB Ch 16). <https://csrc.nist.gov/pubs/sp/800/61/r2/final>
- Wiggins, A. (2012). The Twelve-Factor App. The conceptual foundation for configuration management, environment parity, and disposability. <https://12factor.net/>
- Fowler, M. (2003). Patterns of Enterprise Application Architecture. Addison-Wesley. Source of the outbox, unit-of-work, and repository patterns referenced in the Event Constitution (Vol II Ch 29) and the Database Constitution (Vol II Ch 33).
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly. The conceptual foundation for the canonical-registry, idempotency, and reconciliation disciplines.

- Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly. The conceptual foundation for service boundaries, contract testing, and the API Constitution (Vol I Ch 9).

E.3 Internal Cross-References

- Playbook Chapter 1 — How to Use This Playbook (operational vocabulary, document organization).
- Playbook Chapter 2 — The Implementation Lifecycle (eight-stage lifecycle, gates summary).
- Playbook Chapter 3 — Branching and Version Control.
- Playbook Chapter 4 — Commits and Pull Requests.
- Playbook Chapter 5 — Code Review Regime.
- Playbook Chapter 6 — Quality Assurance and Testing.
- Playbook Chapter 7 — Environments and Test Data.
- Playbook Chapter 8 — Runtime Invariants.
- Playbook Chapter 9 — The Certification Process.
- Playbook Chapter 10 — The Five Pillars of Certification.
- Playbook Chapter 11 — Release and Deployment.
- Playbook Chapter 12 — Rollback and Recovery.
- Playbook Chapter 13 — On-Call Engineering.
- Playbook Chapter 14 — Operational Governance Execution.
- Playbook Chapter 15 — SLO Management.
- Playbook Chapter 16 — Incident Command.
- Playbook Chapter 17 — Volume I Operationalization Matrix.
- Playbook Chapter 18 — Volume II Operationalization Matrix.
- Playbook Chapter 19 — The Master Checklist Catalog.
- Playbook Chapter 20 — Runbook Catalog.

Appendix F — The Production Acceptance Gate (Final)

This appendix is the single-page summary of the Production Acceptance Gate. It is the operational form of Foundational Principle II and the Prime Directive of this Playbook. It is the ultimate gate that blocks production release. This page SHALL be the first page an engineer consults when asked to bypass a gate; the answer is on this page.

— THE PRODUCTION ACCEPTANCE GATE —

No implementation MAY proceed to production release unless ALL three legs are satisfied:

1. CERTIFICATION LEG (PB Ch 9, Ch 10, Ch 19 §19.4). Every item in the Certification Checklist (CT-01..CT-22) SHALL be satisfied. The Release Certification Authority (RCA) SHALL have signed off in the release record. The five pillars (Runtime, Visual, Database, Storage, API) SHALL each have objective evidence captured and stored in the certification repository. No invariant SHALL be in violation.
2. RELEASE LEG (PB Ch 11, Ch 19 §19.5). Every item in the Release Checklist (RL-01..RL-16) SHALL be satisfied. The release artifact SHALL be content-addressed (SHA-256) and signed. The staged release SHALL match the certified artifact. The deployment plan SHALL be documented; the on-call SHALL be briefed; the rollback SHALL be tested.
3. DEPLOYMENT LEG (PB Ch 11, Ch 19 §19.6). Every item in the Deployment Checklist (DP-01..DP-14) SHALL be satisfied. The staged rollout SHALL be executed; each stage SHALL be observed; the feature flag SHALL be enabled for the staged audience; post-deploy verification SHALL be green; invariant monitors SHALL be green; the SLO dashboard SHALL be green.

THE GATE SHALL NOT BE BYPASSED. THE GATE SHALL NOT BE OVERRIDDEN BELOW THE CHIEF SYSTEMS ARCHITECT. THE CHIEF SYSTEMS ARCHITECT SHALL NOT OVERRIDE AN RCA CERTIFICATION FAILURE — THE IMPLEMENTATION SHALL BE CORRECTED INSTEAD.

This is the operational form of Foundational Principle II. The Gate is jointly enforced by the RCA (certification leg), the Release Manager (release leg), and the On-Call engineer (deployment leg). The Gate is the entry criterion for the Post-Release Ownership stage (PB Ch 2 §2.10). The Gate is the operational guarantee that the Constitution's rules are enforced, not aspirational.

The Gate is not negotiable. An engineer who is asked to bypass the Gate SHALL refuse and SHALL escalate to the Engineering Leadership Team. An engineer who is asked to override an RCA certification failure SHALL refuse, regardless of the requester's seniority, and SHALL escalate to the Chief Systems Architect. The Chief Systems Architect, in turn, SHALL NOT override the certification leg; the implementation SHALL be corrected and re-certified. This is the contract that the Playbook makes with every engineer, every reviewer, every certifier, and every operator of Opus Publica.

**— Issued under the authority of the Chief Systems Architect, Opus Publica — Engineering Playbook
complete —**